

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Навчально-науковий інститут комп'ютерних систем

Кафедра інженерії програмного забезпечення

Нікіта КАБАКОВ

(група [ПОТРЕБУЄ УТОЧНЕННЯ: група])

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА

**Вебсервіс для потокової передачі медіаконтенту з функцією захоплення
окремих застосунків**

Спеціальність:

121 Інженерія програмного забезпечення

Освітньо-професійна програма:

Інженерія програмного забезпечення

Керівник:

Андрій ЛАПАЄВ

Одеса – [ПОТРЕБУЄ УТОЧНЕННЯ: рік]

АНОТАЦІЯ

Кабаков Н. С. Вебсервіс для потокової передачі медіаконтенту з функцією захоплення окремих застосунків : кваліфікаційна робота бакалавра за спеціальністю «121 Інженерія програмного забезпечення» / Нікіта Сергійович Кабаков ; керівник Андрій Валерійович Лапаєв. – Одеса : Нац. ун-т «Одес. політехніка», [ПОТРЕБУЄ УТОЧНЕННЯ: рік]. – 107 с.

Кваліфікаційна робота містить основну текстову частину на 80 сторінках, список використаних джерел з 30 найменувань на 3 сторінках, додаток на 17 сторінках.

Метою кваліфікаційної роботи є підвищення ефективності трансляції окремих застосунків шляхом розроблення вебсервісу потокової передачі медіаконтенту з настільним застосунком для захоплення, який передає глядачам у браузері зображення та власний звук лише обраного застосунку і без дій стримера перемикає трансляцію на нове вікно цього застосунку протягом 1,5 с. Об'єктом роботи є процес трансляції окремих застосунків глядачам у реальному часі, предметом – методи та програмні засоби захоплення зображення й звуку окремого застосунку, автоматичного перемикання трансляції між його вікнами та доставки медіапотоку глядачам через WebRTC.

У роботі проаналізовано наявні рішення, оцінено трудомісткість методом точок варіантів використання, спроектовано й реалізовано вебзастосунок, сервер сигналізації з медіасервером SFU і настільний застосунок на основі Next.js, mediasoup і Electron. Система передає глядачам у браузері зображення та власний звук лише обраного застосунку й перемикає трансляцію на його нове вікно за 1,5 с. Із 30 автоматизованих функціональних прикладів 27 дали очікуваний результат, дефекти авторизації визначено як напрям доопрацювання.

Ключові слова: вебсервіс, потокова передача медіаконтенту, захоплення застосунку, захоплення звуку процесу, WebRTC, SFU, mediasoup, Electron, Next.js.

ABSTRACT

Kabakov N. S. Web service for streaming media content with the function of capturing individual applications : bachelor's qualification work in the specialty «121 Software Engineering» / Nikita Serhiiovych Kabakov ; supervisor Andrii Valeriiovych Lapaiev. – Odesa : Odesa Polytech. Nat. Univ., [ПОТРЕБУЄ УТОЧНЕННЯ: year]. – 107 p.

The qualification work contains the main text part on 80 pages, a list of used sources with 30 titles on 3 pages, an appendix on 17 pages.

The purpose of the qualification work is to improve the efficiency of broadcasting individual applications by developing a media streaming web service with a desktop capture application that delivers to viewers in the browser the picture and own sound of the selected application only and, without any action from the streamer, switches the broadcast to a new window of that application within 1.5 s. The object of the work is the process of broadcasting individual applications to viewers in real time, the subject is the methods and software means of capturing the picture and sound of an individual application, automatically switching the broadcast between its windows and delivering the media stream to viewers over WebRTC.

The work analyses existing solutions, estimates the effort with the use case points method, designs and implements a web application, a signaling server with an SFU media server and a desktop application based on Next.js, mediasoup and Electron. The system delivers to viewers in the browser the picture and own sound of the selected application only and switches the broadcast to its new window within 1.5 s. Of 30 automated functional test cases, 27 produced the expected result; the authorization defects are identified as a direction for further work.

Keywords: web service, media streaming, application capture, process audio capture, WebRTC, SFU, mediasoup, Electron, Next.js.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	6
ВСТУП.....	8
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА СПЕЦИФІКАЦІЯ ВИМОГ.....	11
1.1 Аналіз предметної області.....	11
1.2 Аналіз наявних програмних рішень.....	14
1.3 Функціональні вимоги до програмної системи.....	17
1.4 Нефункціональні вимоги.....	26
Висновки до розділу 1.....	28
2 ПЛАНУВАННЯ ПРОГРАМНОГО ПРОЄКТУ.....	29
2.1 Оцінювання тривалості розробки.....	29
2.2 Розробка плану виконання проєкту.....	33
2.3 Аналіз ризиків та планування реакцій.....	35
Висновки до розділу 2.....	36
3 ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ.....	37
3.1 Архітектура програмної системи.....	37
3.2 Проєктування процесів у системі.....	40
3.3 Проєктування структури бази даних.....	48
3.4 Проєктування класів.....	51
3.5 Проєктування інтерфейсу користувача.....	54
Висновки до розділу 3.....	55
4 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ.....	56
4.1 Вибір інструментів розробки.....	56
4.2 Реалізація захоплення окремих застосунків.....	58
4.3 Реалізація потокової передачі на основі WebRTC та SFU.....	63
4.4 Реалізація автентифікації та авторизації.....	67
4.5 Реалізація рівня даних і програмного інтерфейсу сервера.....	70
4.6 Реалізація інтерфейсу користувача та розгортання.....	71
Висновки до розділу 4.....	72

5 ТЕСТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ.....	73
5.1 Принципи вибору тестових прикладів.....	73
5.2 Функціональне тестування.....	75
5.3 Нефункціональне тестування.....	77
5.4 Виправлені дефекти та аналіз результатів.....	78
Висновки до розділу 5.....	80
6 ЕКСПЛУАТАЦІЯ ПРОГРАМНОЇ СИСТЕМИ.....	81
6.1 Контрольний приклад застосування програмної системи.....	81
6.2 Експлуатаційні випробування програмної системи.....	84
Висновки до розділу 6.....	85
ЗАГАЛЬНІ ВИСНОВКИ.....	86
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	88
ДОДАТОК А Лістинг програми.....	91

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

люд.-год – людино-година

ОС – операційна система

СКБД – система керування базами даних

ФВ – функціональна вимога

API – Application Programming Interface, прикладний програмний інтерфейс

CI – Continuous Integration, безперервна інтеграція

CSS – Cascading Style Sheets, каскадні таблиці стилів

DTLS – Datagram Transport Layer Security, протокол захисту транспортного рівня для датаграм

EF – Environmental Factor, коефіцієнт середовища

FK – Foreign Key, зовнішній ключ

HLS – HTTP Live Streaming, протокол потокової передачі через HTTP

HTTP – HyperText Transfer Protocol, протокол передачі гіпертексту

HTTPS – HyperText Transfer Protocol Secure, захищений протокол передачі гіпертексту

ICE – Interactive Connectivity Establishment, протокол встановлення інтерактивного з'єднання

ID – Identifier, ідентифікатор

IPC – Inter-Process Communication, міжпроцесна взаємодія

ISO/IEC – International Organization for Standardization / International Electrotechnical Commission, Міжнародна організація зі стандартизації / Міжнародна електротехнічна комісія

JSON – JavaScript Object Notation, текстовий формат обміну даними

JWT – JSON Web Token, вебтокен у форматі JSON

OAuth – Open Authorization, відкритий протокол авторизації

OBS – Open Broadcaster Software, програма для запису та трансляцій

ORM – Object-Relational Mapping, об'єктно-реляційне відображення

PF – Productivity Factor, коефіцієнт продуктивності

PID – Process Identifier, ідентифікатор процесу

PK – Primary Key, первинний ключ

PKCE – Proof Key for Code Exchange, ключ підтвердження для обміну кодом

REST – Representational State Transfer, передача стану представлення

RFC – Request for Comments, серія документів IETF

RTMP – Real-Time Messaging Protocol, протокол обміну повідомленнями в реальному часі

RTP – Real-time Transport Protocol, транспортний протокол реального часу

SFU – Selective Forwarding Unit, вузол селективного пересилання

SRTP – Secure Real-time Transport Protocol, захищений транспортний протокол реального часу

TCF – Technical Complexity Factor, технічний коефіцієнт

TCP – Transmission Control Protocol, протокол керування передачею

UAW – Unadjusted Actor Weight, нескоригована вага дійових осіб

UCP – Use Case Points, точки варіантів використання

UDP – User Datagram Protocol, протокол користувацьких датаграм

UML – Unified Modeling Language, уніфікована мова моделювання

UUCW – Unadjusted Use Case Weight, нескоригована вага варіантів використання

UUID – Universally Unique Identifier, універсальний унікальний ідентифікатор

WebRTC – Web Real-Time Communication, технологія обміну медіаданими в реальному часі у браузері

ВСТУП

Актуальність теми роботи. Пряма трансляція вмісту екрана стала звичним способом показати іншим людям роботу з програмою, ігровий процес чи навчальний матеріал. Більшість наявних рішень розв'язує це завдання одним із двох способів. Платформи трансляцій доставляють потік великій аудиторії через HTTP-технології із затримкою в секунди [1] і покладають захоплення на сторонній кодувальник. Сервіси відеоконференцій та месенджери, навпаки, прив'язують глядача до зустрічі чи голосового каналу [2, 3]. Водночас трансляція саме окремого застосунку, а не всього екрана, має на практиці кілька особливостей: потрібно передавати лише звук цього застосунку, а не весь системний звук із повідомленнями та голосами інших програм, і не втрачати зображення, коли застосунок відкриває нове вікно.

Вебсторінка таких можливостей не має. Специфікація Screen Capture вимагає, щоб користувач щоразу сам обирав поверхню захоплення, забороняє застосунку обмежувати цей вибір, а звук розглядає як необов'язковий [4]. Звук окремого процесу доступний лише через системні інтерфейси операційної системи [5]. Отже, актуальним є створення вебсервісу, у якому перегляд відбувається у браузері з низькою затримкою, а захоплення окремого застосунку разом із його власним звуком виконує допоміжний настільний застосунок стримера.

Технологічну основу для такого сервісу становить стек WebRTC, стандартизований W3C та IETF [6, 7]. Він забезпечує обов'язкове шифрування медіаданих [8] і відтворення потоку у браузері без додаткових програм. Для доставки одного потоку багатьом глядачам доцільна топологія з вузлом селективного пересилання (SFU), який не декодує медіадані, вирізняється надзвичайно низькою затримкою і масштабується на велику кількість користувачів [9]. Серед розглянутих наявних рішень жодне не поєднує захоплення окремого застосунку з його власним звуком, автоматичний перехід до нових вікон застосунку та перегляд у браузері без приєднання до зустрічі чи

голосового каналу. Саме це поєднання визначає новизну практичного завдання, яке розв'язується в роботі.

Мета і задачі роботи. Метою кваліфікаційної роботи є підвищення ефективності трансляції окремих застосунків шляхом розроблення вебсервісу потокової передачі медіаконтенту з настільним застосунком для захоплення, який передає глядачам у браузері зображення та власний звук лише обраного застосунку і без дій стримера перемикає трансляцію на нове вікно цього застосунку протягом 1,5 с. Показником досягнення мети є відсутність ручних дій стримера під час появи нового вікна застосунку та час перемикавання трансляції на нього, який не перевищує одного циклу опитування (1,5 с).

Для досягнення мети необхідно розв'язати такі задачі:

- проаналізувати предметну область і наявні програмні рішення та сформулювати функціональні й нефункціональні вимоги до програмної системи;
- спланувати програмний проект: оцінити його трудомісткість, скласти календарний план і проаналізувати ризики;
- спроектувати архітектуру програмної системи, взаємодію її компонентів, структуру даних і користувацький інтерфейс;
- реалізувати вебзастосунок, сервер сигналізації з медіасервером SFU та настільний застосунок із захопленням окремих застосунків і їхнього звуку;
- протестувати програмну систему на відповідність сформульованим вимогам;
- описати порядок експлуатації програмної системи та провести її експлуатаційні випробування.

Об'єкт роботи. Об'єктом роботи є процес трансляції окремих застосунків глядачам у реальному часі.

Предмет роботи. Предметом роботи є методи та програмні засоби захоплення зображення й звуку окремого застосунку, автоматичного перемикавання трансляції між його вікнами та доставки медіапотоку глядачам через WebRTC.

Методи та засоби розроблення. Під час роботи застосовано порівняльний аналіз наявних програмних рішень за їхньою офіційною документацією, моделювання вимог і проектних рішень мовою UML (діаграми варіантів використання, компонентів, розгортання, послідовності, діяльності, станів і класів), оцінювання трудомісткості методом точок варіантів використання (UCP),

статичний аналіз і перевірку типів вихідного коду, функціональне тестування за сценаріями використання та експлуатаційні випробування. Програмну систему розроблено мовою TypeScript із використанням фреймворку Next.js для вебзастосунку, сервера Fastify з бібліотекою mediasoup для сигналізації та пересилання медіапотоків, платформи Electron для настільного застосунку, СКБД PostgreSQL з бібліотекою Drizzle ORM для зберігання даних і Docker Compose для розгортання. Вибір цих засобів обґрунтовано під час опису програмної реалізації.

Практична значущість. Розроблений вебсервіс розгорнуто й доступно за адресою <https://streamshare.space>. Стример може розпочати трансляцію безпосередньо у браузері або в настільному застосунку, який дозволяє обрати окреме вікно, передає звук лише процесу, якому це вікно належить, і автоматично переходить до нових вікон того самого застосунку. Глядач переглядає трансляцію у браузері без встановлення програм, зокрема як гість без облікового запису, а приватну трансляцію можна передати за посиланням. Вхід до сервісу виконується через обліковий запис Google, причому в настільному застосунку вибір облікового запису відбувається в системному браузері користувача. Інсталятори настільного застосунку для Windows, macOS і Linux публікуються в розділі випусків репозиторію GitHub; автоматичне оновлення підтримується у Windows і Linux.

Апробація матеріалів роботи. [ПОТРЕБУЄ УТОЧНЕННЯ: доповіді на конференціях або публікації за темою роботи з повними бібліографічними даними; якщо їх немає, абзац вилучається]

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА СПЕЦИФІКАЦІЯ ВИМОГ

1.1 Аналіз предметної області

Потокова передача передбачає доставлення аудіо- та відеоданих отримувачу в міру їх надходження, а пряма трансляція – передачу контенту, що формується в момент передавання. Для прямої трансляції визначальною є затримка доставки – проміжок часу між подією на стороні джерела та її відтворенням у глядача, від якого залежить можливість взаємодії в реальному часі. Затримка визначається насамперед технологією доставки.

Протокол HTTP Live Streaming (HLS) передає потік через HTTP як список відтворення з послідовних медіасегментів, і вже сама модель доставки закладає затримку в секунди [1]. Протокол RTMP поверх TCP слугує переважно для приймання потоку від кодувальника платформою, а WebRTC призначена для двобічного обміну медіаданими в реальному часі безпосередньо з браузера (табл. 1.1).

Таблиця 1.1 – Технології доставки медіапотоків

Технологія	Транспорт	Роль у ланцюгу трансляції	Затримка (за джерелами)
RTMP	TCP	приймання потоку від кодувальника платформою [10]	не нормується, залежить від подальшої доставки
HLS	HTTP, сегменти	доставка глядачам через HTTP-інфраструктуру [1]	секунди: старт не ближче трьох цільових тривалостей від кінця списку [1]
WebRTC	UDP (RTP/SRTP), ICE	двобічна передача між браузерами та серверами [6, 7]	реальний час; у топології SFU – «надзвичайно низька» [9]

З табл. 1.1 видно, що технології на основі HTTP оптимізовано для масштабованої доставки ціною затримки в секунди, тоді як WebRTC розрахована на інтерактивну взаємодію. Оскільки глядач спостерігає за роботою застосунку стримера і реагує на неї, у роботі як транспорт обрано WebRTC.

Розглянемо складові WebRTC докладніше. Технологія поєднує програмний інтерфейс браузера, стандартизований W3C [6], і набір протоколів IETF, огляд яких наведено в RFC 8825 [7]. Медіадані передаються у форматі RTP, причому реалізація захищеного профілю SRTP є обов'язковою [7]. Архітектура безпеки WebRTC забороняє передавання медіа через незашифрований RTP і вимагає пропонувати для кожного медіаканалу узгодження ключів DTLS-SRTP [8]. Встановлення з'єднання крізь трансляцію мережевих адрес (NAT) забезпечує протокол ICE, а описи сеансу учасники узгоджують за моделлю «пропозиція/відповідь», причому спосіб обміну цими описами визначає сам застосунок.

Тому в stream-share сигналізацію реалізовано власним протоколом повідомлень поверх WebSocket, типи яких спільні для клієнта й сервера, а медіасервер оголошує в ICE-кандидатах публічну адресу, задану в конфігурації сервера; цей протокол описано під час проєктування системи.

Наступним питанням є топологія, за якою один стример передає потік кільком глядачам. RFC 7667 описує три основні варіанти [9] (рис. 1.1). У топології mesh відправник надсилає окрему копію потоку кожному отримувачу, тож вихідний канал стримера зростає пропорційно кількості глядачів. Мікшер (MCU) декодує, змішує та повторно кодує потоки, що потребує значних обчислювальних ресурсів і погіршує якість [9]. Вузол селективного пересилання (SFU) лише пересилає RTP-потоки без обробки сигналу, тому має надзвичайно низьку затримку і масштабується на велику кількість користувачів [9].

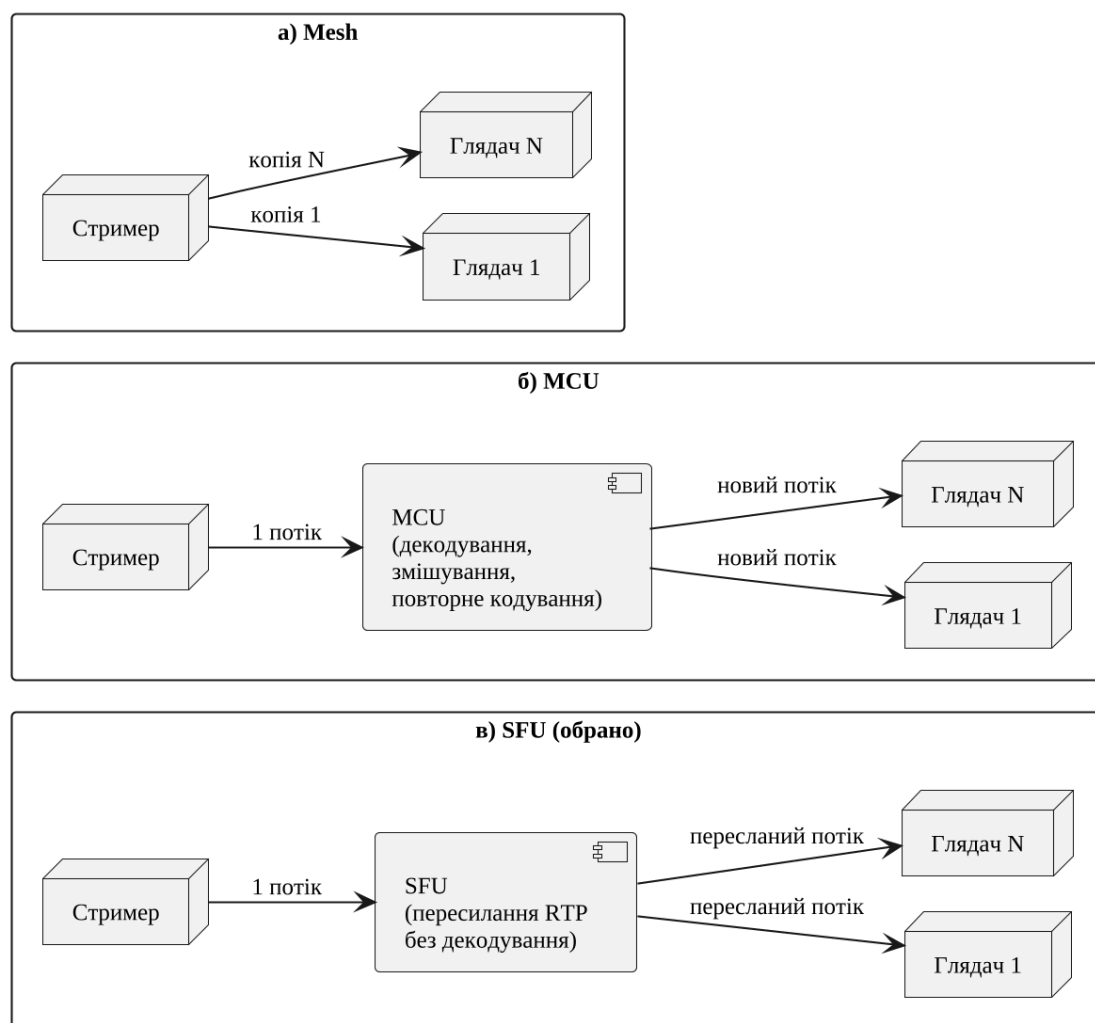


Рисунок 1.1 – Топології передачі медіапотоку від одного стримера кільком глядачам

На рис. 1.1 показано, що в топології SFU стример відправляє рівно один потік незалежно від кількості глядачів, а сервер розмножує його без декодування. Відеоконференції та трансляції є двома найпоширенішими сценаріями застосування SFU [11]. Бібліотека mediasoup реалізує SFU як модуль Node.js і не нав'язує сигнальний протокол [12]; у stream-share вона працює в складі сервера сигналізації.

Останнім розглянемо захоплення вмісту екрана, від якого залежить, що саме побачить глядач. У браузері його виконує метод `getDisplayMedia` специфікації Screen Capture, яка визначає три типи поверхонь: монітор, вікно окремого застосунку та вкладку браузера [4]. Специфікація вимагає, щоб користувач

щоразу сам обирав поверхню з усіх доступних, і забороняє застосунок обмежувати цей вибір; джерела захоплення не перелічуються сторінці, оскільки це розкрило б забагато інформації про систему [4]. Звук під час захоплення є необов'язковим, а параметри `systemAudio` і `windowAudio` є лише підказками браузеру [4].

Отже, вебсторінка не може програмно обрати потрібне вікно, отримати звук лише цього застосунок чи перейти до його нового вікна. Ці обмеження долаються в настільному застосунку. Модуль `desktopCapturer` платформи Electron перелічує джерела типів «екран» і «вікно», а метод `setDisplayMediaRequestHandler` дозволяє самостійно відповісти на запит захоплення сторінки обраним джерелом [13]. Його аудіорежим `loopback` захоплює лише весь системний звук і підтримується тільки у Windows [14]. Звук окремого процесу дають системні інтерфейси WASAPI Process Loopback (Windows 10 2004+), ScreenCaptureKit (macOS 13+) і PipeWire (Linux), які об'єднує нативна бібліотека `electron-native-screenshare`; вона також визначає ідентифікатор процесу за дескриптором вікна [5].

Саме цей підхід використовує настільний застосунок `stream-share`.

1.2 Аналіз наявних програмних рішень

Для визначення можливостей, які має поєднувати `stream-share`, проаналізовано найвідоміші рішення трьох класів, що дозволяють показати іншим вміст екрана: платформу трансляцій Twitch разом із кодувальником OBS Studio, месенджер Discord із функцією Go Live і сервіси відеоконференцій Google Meet та Zoom. Аналіз ґрунтується лише на офіційній документації цих продуктів.

Платформа Twitch приймає потік від зовнішнього кодувальника за протоколом RTMP [10], тому захоплення застосунок на стороні стримера виконує окрема програма, найчастіше OBS Studio – вільна програма для запису та прямих трансляцій для Windows, macOS і Linux. Її джерело «Window Capture» захоплює вміст конкретного вікна навіть тоді, коли воно перекрите іншими, а параметр «Window Match Priority» визначає, як шукати вікно, якщо попереднє не знайдено:

за збігом заголовка або вікно того самого типу [15]. Починаючи з версії 28, OBS Studio може захоплювати звук окремих застосунків, але лише у Windows 10 версії 2004 і новіших та Windows 11 [16]. Глядачі переглядають трансляції на сайті twitch.tv у браузері, зокрема без входу до облікового запису: такі глядачі отримують відео з роздільністю до 1080p [17].

Месенджер Discord дозволяє за допомогою Go Live транслювати окреме вікно застосунку або весь екран у голосовому каналі сервера чи у виклику. Звук застосунку захоплюється лише в настільних клієнтах для Windows і macOS, у браузері Chrome та в мобільних клієнтах, а в Linux недоступний. Переглядати трансляцію можуть лише учасники голосового каналу чи виклику, тобто користувачі Discord, а без підписки Nitro діють обмеження якості й кількості глядачів [3].

Сервіс Google Meet дозволяє показати вкладку, окреме вікно або весь екран; звук передається або разом із вкладкою Chrome, або як системний звук під час показу вікна чи екрана [2]. До зустрічі приєднуються в браузері на сайті meet.google.com; учасник без облікового запису Google може приєднатися лише після того, як його впустить організатор або інший учасник [18].

Сервіс Zoom також дозволяє показати весь екран чи окремі вікна застосунків, проте функція спільного доступу до звуку передає весь звук, який відтворює комп'ютер, а в сеансах Linux на Wayland можна показати тільки весь робочий стіл [19]. Учасникові зустрічі обліковий запис Zoom не потрібен, якщо організатор не вимагає автентифікації, а приєднатися можна з браузера без встановлення програми [20].

Для stream-share наведені далі можливості встановлено за вихідним кодом системи: захоплення окремого вікна, звуку лише його процесу та слідування за новими вікнами застосунку виконує настільний застосунок, а перегляд відбувається на сторінці вебзастосунку, де для відвідувача без облікового запису автоматично створюється гостьовий сеанс. Порівняння функціональних можливостей stream-share та розглянутих рішень наведено в табл. 1.2.

Таблиця 1.2 – Порівняння наявних програмних рішень

Функціональна можливість	stream-share	Twitch + OBS Studio	Discord Go Live	Google Meet	Zoom
Захоплення окремого вікна застосунку	+	+	+	+	+/-
Передавання звуку лише обраного застосунку	+	+/-	+/-	-	-
Автоматичний перехід до нових вікон застосунку	+	+/-	-	-	-
Перегляд у браузері без встановлення програм	+	+	+	+	+
Перегляд без облікового запису	+	+	-	+/-	+
Перегляд без приєднання до зустрічі чи голосового каналу	+	+	-	-	-

Позначка «+/-» означає часткову підтримку: OBS Studio захоплює звук окремого застосунку лише у Windows і за зникнення вікна лише повторно шукає вікно за заголовком чи типом, Discord не передає звук застосунку в Linux, Zoom у сеансах Linux на Wayland показує лише весь робочий стіл, а гість Google Meet потрапляє до зустрічі тільки з дозволу її учасників; позначка «-» означає, що можливість в офіційній документації не описано.

Жодне з розглянутих рішень не поєднує всіх можливостей, потрібних для трансляції окремого застосунку: найближча зв'язка Twitch і OBS Studio покладає захоплення на окремий кодувальник і не переходить до нових вікон, а Discord, Google Meet і Zoom роблять глядача учасником голосового каналу чи зустрічі. Отже, нішу розроблюваного вебсервісу визначає поєднання захоплення окремого застосунку з його власним звуком, слідування за новими вікнами цього застосунку, доставки з низькою затримкою через WebRTC і SFU та перегляду у браузері без встановлення програм, зокрема гостем без облікового запису. Ці можливості покладено в основу функціональних вимог до системи.

1.3 Функціональні вимоги до програмної системи

Функціональні вимоги до вебсервісу сформульовано за результатами аналізу предметної області та наявних рішень і зафіксовано у вигляді моделі варіантів використання мови UML [21]. [ПОТРЕБУЄ УТОЧНЕННЯ: звірити перелік вимог із формулюваннями індивідуального завдання на кваліфікаційну роботу]

Система взаємодіє з двома категоріями людей і однією зовнішньою системою.

Глядач – відвідувач, який переглядає трансляції: незареєстрований глядач лише відкриває трансляцію за посиланням в автоматично створеному гостьовому сеансі, а глядач, що зареєструвався через Google, додатково обирає трансляції з переліку активних публічних трансляцій.

Стример – зареєстрований користувач, який розпочинає та завершує трансляції у браузері або в настільному застосунку й водночас має всі можливості глядача.

Google – зовнішня система, що підтверджує особу глядача під час авторизації за протоколом OAuth 2.0.

Проаналізувавши можливості дійових осіб, складено діаграму варіантів використання, яку наведено на рис. 1.2.

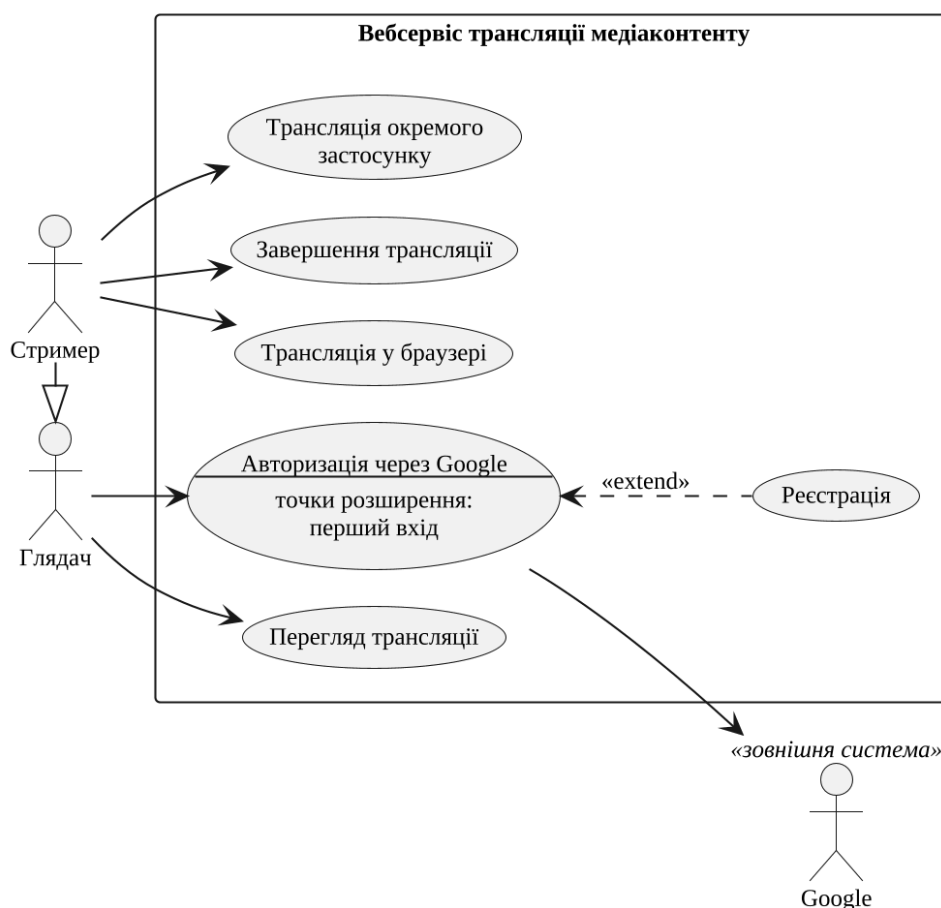


Рисунок 1.2 – Діаграма варіантів використання вебсервісу

Діаграма містить шість варіантів використання. Глядачу доступні «Перегляд трансляції» та «Авторизація через Google», у якій бере участь зовнішня система Google. Стример успадковує ці можливості, оскільки узагальнює глядача, і додатково виконує «Трансляцію у браузері», «Трансляцію окремого застосунку» та «Завершення трансляції». Отже, незареєстрований глядач може лише переглядати доступні трансляції, а для показу екрана чи застосунку потрібна реєстрація. «Трансляція у браузері» використовує стандартне захоплення екрана, вікна чи вкладки, а «Трансляція окремого застосунку» доступна лише в настільному застосунку, де система захоплює звук обраного застосунку й переходить до його нових вікон. «Реєстрація» розширює авторизацію (відношення «extend») у точці розширення «перший вхід»: окремої форми реєстрації система не має, а запис користувача створюється лише тоді, коли підтверджений Google обліковий запис ще не пов'язаний із жодним

користувачем. Наступні входи завершуються без реєстрації, тому вона є умовним доповненням авторизації, а не обов'язковим включенням «include». Вхід не включено й до варіантів стримера, бо система вимагає його лише за відсутності чинного сеансу, тобто він є передумовою, а не кроком цих варіантів.

Узагальнений перелік функціональних вимог наведено в табл. 1.3. Кожну вимогу позначено кодом, за яким її простежено під час проєктування, реалізації та тестування, і частиною системи, що її реалізує.

Таблиця 1.3 – Функціональні вимоги до програмної системи

Код	Вимога	Частина системи
ФВ-01	Вхід через обліковий запис Google з вибором облікового запису під час кожного входу; вихід із системи	вебзастосунок
ФВ-02	Вхід у настільному застосунку через системний браузер із передаванням сеансу в застосунок за схемою РКСЕ	настільний застосунок, вебзастосунок
ФВ-03	Автоматичне створення гостьового сеансу з випадковим іменем та згенерованим аватаром під час відкриття сторінки перегляду; повторне використання гостьового запису в тому самому браузері	вебзастосунок
ФВ-04	Обмеження доступу: сторінки, крім головної, сторінки входу та сторінки перегляду, доступні лише користувачам, які увійшли через Google	вебзастосунок
ФВ-05	Перелік активних публічних трансляцій із мініатюрами та посторінковим переглядом по 12 елементів	вебзастосунок, сервер сигналізації
ФВ-06	Трансляція у браузері з вибором екрана, вікна чи вкладки та звуку стандартними засобами браузера	вебзастосунок
ФВ-07	Перелік екранів і вікон із мініатюрами та піктограмами застосунків у настільному застосунку	настільний застосунок
ФВ-08	Захоплення звуку обраного застосунку; за неможливості захоплення – трансляція лише відео з повідомленням користувачу	настільний застосунок
ФВ-09	Режим слідування: автоматичне перемикання трансляції на нові вікна того самого застосунку та повернення до початкового вікна	настільний застосунок, вебзастосунок
ФВ-10	Вибір якості (360p, 480p, 720p, 1080p, 1440p,	вебзастосунок

Код	Вимога	Частина системи
	вихідна) і частоти кадрів (5, 30, 60 кадр/с) до початку та під час трансляції	
ФВ-11	Позначення трансляції як приватної: вона не потрапляє до переліку, але доступна за посиланням	вебзастосунок, сервер сигналізації
ФВ-12	Формування та копіювання посилання на перегляд активної трансляції	вебзастосунок
ФВ-13	Вимкнення та ввімкнення звуку трансляції без її зупинення	вебзастосунок
ФВ-14	Перегляд трансляції з керуванням гучністю, режимом «картинка в картинці» та повноекранним режимом	вебзастосунок, сервер сигналізації
ФВ-15	Завершення трансляції стримером зі сповіщенням глядачів	вебзастосунок, сервер сигналізації
ФВ-16	Збереження трансляції під час втрати з'єднання стримера (до 5 спроб з інтервалом 6 с) і відновлення її з того самого або іншого вікна браузера	сервер сигналізації, вебзастосунок
ФВ-17	Періодичне оновлення мініатюри трансляції (кожні 5 хв); завантаження мініатюри дозволено лише власнику трансляції	вебзастосунок, сервер сигналізації
ФВ-18	Завантаження настільного застосунку для операційної системи відвідувача; перевірка й установлення оновлень для Windows і Linux	вебзастосунок, настільний застосунок
ФВ-19	Щотижневе видалення неактивних гостей записів, не пов'язаних із жодною трансляцією	сервер сигналізації

Вимоги ФВ-01–ФВ-05 визначають доступ до системи, ФВ-06–ФВ-13 – підготовку й ведення трансляції, ФВ-14–ФВ-17 – доставлення потоку глядачам і життєвий цикл трансляції, ФВ-18 і ФВ-19 є службовими. Вимоги ФВ-07–ФВ-09 реалізують захоплення окремих застосунків і доступні лише в настільному застосунку, оскільки вебсторінка не може ні обрати звук окремого процесу, ні відстежувати появу його вікон.

Для кожного з наведених варіантів використання складемо сценарій [22] (сценарії 1.1 – 1.6). Назви кнопок у сценаріях наведено так, як їх показує інтерфейс системи.

Сценарій 1.1 – Авторизація через Google

Основна дійова особа: глядач.

Результат: глядач увійшов до системи як зареєстрований користувач і отримав сеанс.

Тригер: глядач відкриває сторінку входу або сторінку, доступну лише після входу.

Основний успішний сценарій:

1. Глядач переходить до сторінки входу.
2. Система відображає кнопку «Continue with Google».
3. Глядач натискає кнопку «Continue with Google».
4. Система перенаправляє глядача до сервісу автентифікації Google із запитом на вибір облікового запису.
5. Глядач обирає обліковий запис Google і підтверджує вхід.
6. Система знаходить запис користувача, пов'язаний з обраним обліковим записом Google.
7. Система встановлює сеансовий файл cookie та перенаправляє глядача на головну сторінку з переліком активних трансляцій.

Альтернативні сценарії:

- За. Вхід розпочато в настільному застосунку.
- За.1. Система відкриває сторінку автентифікації в системному браузері, де виконуються кроки 4–6.
- За.2. Система передає настільному застосунку одноразовий код через локальну адресу 127.0.0.1.
- За.3. Настільний застосунок обмінює код на сеанс. Перехід до п. 7.
- Зб. Глядач скасовує вхід у настільному застосунку або не завершує його протягом 5 хв.
- Зб.1. Система припиняє очікування, сеанс не створюється. Повернення до п. 3.
- Ба. Сервіс автентифікації Google відхиляє вхід.
- Ба.1. Система повертає глядача на сторінку входу. Повернення до п. 3.

6а. Обліковий запис Google ще не пов'язаний із жодним користувачем (точка розширення «перший вхід»).

6а.1. Варіант використання «Реєстрація». Перехід до п. 7.

Сценарій 1.2 – Реєстрація

Основна дійова особа: незареєстрований глядач.

Результат: створено запис користувача, пов'язаний з обліковим записом Google.

Тригер: підтверджений Google обліковий запис не пов'язаний із жодним користувачем.

Основний успішний сценарій:

1. Система отримує від Google ім'я, адресу електронної пошти та зображення профілю глядача.
2. Система перевіряє, що користувача з такою адресою електронної пошти не існує.
3. Система створює запис користувача з роллю зареєстрованого користувача.
4. Система зберігає зв'язок запису користувача з обліковим записом Google і повертається до п. 7 сценарію 1.1.

Альтернативні сценарії:

2а. Користувач із такою адресою електронної пошти вже існує, але не пов'язаний із цим обліковим записом Google.

2а.1. Система відмовляє у вході та повертає глядача на сторінку входу.

Сценарій 1.3 – Перегляд трансляції

Основна дійова особа: глядач.

Результат: глядач бачить і чує трансляцію в реальному часі.

Тригер: глядач відкриває посилання на трансляцію або обирає її в переліку активних трансляцій.

Основний успішний сценарій:

1. Глядач переходить за посиланням на сторінку перегляду трансляції.

2. Система перевіряє, що трансляція існує та не завершена.
3. Система створює гостьовий сеанс, якщо глядач не увійшов до системи, або відновлює створений раніше в цьому браузері.
4. Система реєструє глядача, з'єднується із сервером сигналізації та отримує відео- та аудіопотоки.
5. Система відтворює трансляцію.
6. Глядач керує гучністю, режимом «картинка в картинці» та повноекранним режимом.

Альтернативні сценарії:

- 1а. Зареєстрований глядач відкриває головну сторінку.
 - 1а.1. Система показує перелік активних публічних трансляцій із мініатюрами.
 - 1а.2. Глядач обирає трансляцію. Перехід до п. 2.
 - 2а. Трансляцію не знайдено або її завершено.
 - 2а.1. Система показує відповідне повідомлення.
 - 5а. Браузер заблокував автоматичне відтворення.
 - 5а.1. Система показує кнопку запуску відтворення.
 - 5а.2. Глядач натискає кнопку, і система відтворює трансляцію.
 - 5б. Стример втратив з'єднання із сервером.
 - 5б.1. Система залишає сторінку перегляду відкритою та очікує відновлення трансляції.
 - 5б.2. Після відновлення трансляції система автоматично отримує новий потік.
- Повернення до п. 5.
-

Сценарій 1.4 – Трансляція у браузері

Основна дійова особа: стример.

Результат: трансляція має статус «наживо», а глядачі можуть переглядати її за посиланням.

Тригер: стример відкриває сторінку трансляції.

Основний успішний сценарій:

1. Стример переходить до сторінки трансляції.

2. Система відображає перемикачі якості, частоти кадрів і приватного режиму та кнопку «Pick source».
3. Стример обирає якість і частоту кадрів, за потреби вмикає приватний режим і натискає кнопку «Pick source».
4. Система відображає стандартне вікно браузера для вибору екрана, вікна або вкладки.
5. Стример обирає джерело захоплення.
6. Система показує попередній перегляд джерела.
7. Стример натискає кнопку «Start stream».
8. Система створює трансляцію, з'єднується із сервером сигналізації та передає відео- та аудіопотоки.
9. Система встановлює трансляції статус «наживо» та відображає посилання на перегляд.

Альтернативні сценарії:

- 5а. Стример закриває вікно вибору, не обравши джерело.
 - 5а.1. Система не розпочинає трансляцію. Повернення до п. 3.
 - 9а. Стример змінює якість або частоту кадрів під час трансляції.
 - 9а.1. Система застосовує нові параметри кодування без перезапуску трансляції.
 - 9б. Стример припиняє показ засобами браузера.
 - 9б.1. Система завершує трансляцію.
-

Сценарій 1.5 – Трансляція окремого застосунку

Основна дійова особа: стример.

Результат: глядачі отримують зображення та звук лише обраного застосунку, зокрема з його нових вікон.

Тригер: стример відкриває сторінку трансляції в настільному застосунку.

Основний успішний сценарій:

1. Стример натискає кнопку «Pick source».
2. Система відображає перелік екранів і вікон застосунків із мініатюрами та піктограмами.

3. Стример обирає вікно застосунку та за потреби вмикає перемикач «Follow app».
4. Система запускає захоплення звуку процесу, якому належить вікно, і показує попередній перегляд.
5. Стример обирає якість і частоту кадрів та натискає кнопку «Start stream».
6. Система розпочинає трансляцію, як у кроках 8–9 сценарію 1.4.
7. Стример відкриває в застосунку, що транслюється, нове вікно.
8. Система перемикає трансляцію на нове вікно та показує його назву.
9. Стример закриває нове вікно.
10. Система повертає трансляцію до початкового вікна.

Альтернативні сценарії:

- 3а. Стример обирає екран.
 - 3а.1. Система робить перемикач «Follow app» недоступним, і трансляція відбувається без слідування за вікнами.
 - 4а. Захопити звук застосунку не вдалося.
 - 4а.1. Система повідомляє, що звук недоступний, і транслює лише відео.
 - 8а. Режим слідування вимкнено.
 - 8а.1. Система продовжує транслювати початкове вікно.
 - 10а. Початкове вікно застосунку закрито, і воно не з'являється знову протягом 20 с.
 - 10а.1. Система повідомляє про закриття застосунку та завершує трансляцію.
-

Сценарій 1.6 – Завершення трансляції

Основна дійова особа: стример.

Результат: трансляцію завершено, а її глядачів сповіщено.

Тригер: стример вирішує завершити трансляцію, що має статус «наживо».

Основний успішний сценарій:

1. Стример натискає кнопку «Stop stream».
2. Система встановлює трансляції статус «завершено» з причиною «зупинено стримером».
3. Система звільняє ресурси сервера, виділені трансляції.

4. Система надсилає глядачам подію завершення трансляції.
5. Система показує глядачам повідомлення про завершення трансляції.

Альтернативні сценарії:

- 1а. З'єднання стримера із сервером розірвано до натискання кнопки.
 - 1а.1. Система сповіщає глядачів про втрату з'єднання стримера та очікує повторного підключення до 5 спроб з інтервалом 6 с.
 - 1а.2. Стример відкриває сторінку трансляції, на якій система показує повідомлення «You have an unfinished stream», і натискає кнопку «Reconnect».
 - 1а.3. Система відновлює передавання потоку тим самим глядачам.
 - 1б. Стример не підключився протягом очікування.
 - 1б.1. Система встановлює трансляції статус «завершено» з причиною «тайм-аут».
- Перехід до п. 3.
- 1в. Сервер сигналізації перезапущено.
 - 1в.1. Під час запуску система встановлює всім незавершеним трансляціям статус «завершено».
-

Кроки 3, 4 і 8 сценарію 1.5 відрізняють систему від аналогів (табл. 1.2). Сценарії є основою для оцінювання трудомісткості та розроблення тестових прикладів.

1.4 Нефункціональні вимоги

Нефункціональні вимоги згруповано за моделлю якості ISO/IEC 25010 [23], а їхні числові значення взято з коду та конфігурації системи.

Вимоги до надійності:

- 1) після розриву з'єднання стримера система повинна очікувати його повторного підключення протягом 5 спроб з інтервалом 6 с і лише потім завершувати трансляцію;
- 2) під час запуску сервер сигналізації повинен завершувати трансляції, що залишилися незавершеними після його зупинки;

3) якщо звук застосунку захопити не вдалося, система повинна транслювати лише відео й повідомляти про це стримера.

Вимоги до продуктивності:

1) система повинна без дій стримера перемикає трансляцію на нове вікно застосунку протягом одного циклу опитування вікон (1,5 с), а після його закриття – повертати до початкового вікна;

2) система повинна передавати зображення в профілях від 360p до роздільності джерела (до 3840×2160) з частотою 5, 30 або 60 кадр/с, понад 1440p – до 30 кадр/с;

3) бітрейт відео повинен становити від 300 кбіт/с до 18 Мбіт/с залежно від профілю, початковий вихідний бітрейт сервера – не менше 4 Мбіт/с, вхідний – до 21,6 Мбіт/с;

4) сервер повинен запускати стільки процесів пересилання медіапотоків, скільки ядер має процесор, і призначати трансляцію найменш завантаженому з них;

5) затримка доставки зображення глядачеві не повинна перевищувати [ПОТРЕБУЄ УТОЧНЕННЯ: цільове значення затримки].

Вимоги до безпеки:

1) закриті сторінки та маршрути WebSocket повинні бути недоступні без дійсного сеансу, а керувати трансляцією повинен мати змогу лише її власник – зареєстрований стример;

2) сеанс повинен зберігатися як JWT у cookie з атрибутами HttpOnly і SameSite=Lax, у виробничому середовищі – також Secure і префіксом __Secure-, і перевірятися сервером WebSocket;

3) вхід із настільного застосунку повинен виконуватися в системному браузері за схемою PKCE з перевіркою параметра state [24, 25], бо Google блокує вхід у вбудованих вебпереглядах [26]; запит дійсний 15 хв, одноразовий код – 120 с;

4) сторінки настільного застосунку повинні працювати з ізоляцією контексту, без доступу до Node.js, а дані каналів IPC – перевірятися схемами.

Вимоги до сумісності:

- 1) перегляд трансляції повинен працювати в браузері без встановлення додаткових програм;
- 2) звук повинен передаватися кодеком Opus, а відео – насамперед кодеком H.264, що дає змогу апаратного кодування, з резервним кодеком VP8;
- 3) настільний застосунок повинен постачатися для Windows, macOS і Linux з автоматичним оновленням у Windows і Linux та ручним – у macOS [ПОТРЕБУЄ УТОЧНЕННЯ: чи випробувано застосунок на macOS].

Вимоги до супроводжуваності:

- 1) змінні середовища кожного застосунку повинні перевірятися схемами під час запуску, а помилка конфігурації – зупиняти запуск;
- 2) протокол обміну повідомленнями, схеми даних і розрахунок бітрейту повинні бути спільними модулями вебзастосунку й сервера сигналізації;
- 3) код повинен писатися мовою TypeScript із суворою перевіркою типів.

Вимоги до зручності використання:

- 1) англomовний інтерфейс повинен мати світлу й темну теми відповідно до налаштувань операційної системи;
- 2) перед трансляцією система повинна показувати мініатюри вікон і екранів та попередній перегляд обраного джерела.

Висновки до розділу 1

Аналіз предметної області обґрунтував вибір WebRTC у топології SFU для трансляцій з одним джерелом і багатьма глядачами, а порівняння з аналогами – відсутність рішення, що поєднує захоплення окремого застосунку з його звуком, перехід до його нових вікон і перегляд у браузері без приєднання до зустрічі.

На цій основі визначено дійових осіб «Глядач» і «Стример», шість варіантів використання зі сценаріями, 19 функціональних вимог і нефункціональні вимоги до надійності, продуктивності, безпеки, сумісності, супроводжуваності та зручності використання.

2 ПЛАНУВАННЯ ПРОГРАМНОГО ПРОЄКТУ

2.1 Оцінювання тривалості розробки

Для оцінювання трудомісткості обрано метод точок варіантів використання (Use Case Points, UCP), запропонований Г. Карнером у 1993 р. [27]. Вихідними даними методу є модель варіантів використання (рис. 1.2) та їхні сценарії (сценарії 1.1 – 1.6).

Розглянемо спочатку дійових осіб. Їх (табл. 2.1) поділяють на прості (зовнішня система з програмним інтерфейсом), середні (система, що взаємодіє через протокол, наприклад HTTP, або сховище даних) і складні (людина, що працює через графічний або вебінтерфейс). Нескоригована вага дійових осіб обчислюється за формулою (2.1):

$$UAW = 1 \cdot a_s + 2 \cdot a_a + 3 \cdot a_c, \quad (2.1)$$

де a_s , a_a , a_c – кількість простих, середніх і складних дійових осіб відповідно.

Таблиця 2.1 – Класифікація дійових осіб

Дійова особа	Спосіб взаємодії із системою	Тип	Вага
Глядач	вебінтерфейс у браузері	складний	3
Стример	вебінтерфейс і настільний застосунок	складний	3
Google (OAuth 2.0)	протокол OAuth 2.0 поверх HTTPS	середній	2

Отже, $UAW = 2 \cdot 3 + 1 \cdot 2 = 8$.

Перейдемо до варіантів використання. Їхню складність визначають кількістю транзакцій – кроків основного сценарію та альтернативних гілок: до трьох транзакцій – простий (вага 5), від чотирьох до семи – середній (10), понад сім – складний (15). Нескоригована вага варіантів використання обчислюється за формулою (2.2):

$$UUCW = 5 \cdot u_s + 10 \cdot u_a + 15 \cdot u_c, \quad (2.2)$$

де u_s , u_a , u_c – кількість простих, середніх і складних варіантів використання.

Транзакції кожного з шести варіантів використання підраховано за їхніми сценаріями (табл. 2.2).

Таблиця 2.2 – Класифікація варіантів використання

Варіант використання	Транзакцій	Тип	Вага
Авторизація через Google	$7 + 4 = 11$	складний	15
Реєстрація	$4 + 1 = 5$	середній	10
Трансляція у браузері	$9 + 3 = 12$	складний	15
Трансляція окремого застосунку	$10 + 4 = 14$	складний	15
Перегляд трансляції	$6 + 4 = 10$	складний	15
Завершення трансляції	$5 + 3 = 8$	складний	15
Разом	–	–	85

Нескориговані точки варіантів використання становлять $UUCP = UAW + UUCW = 8 + 85 = 93$.

Наступним розглянемо технічний коефіцієнт, що враховує 13 факторів (табл. 2.3) з оцінкою впливу від 0 до 5. Коефіцієнт обчислюється за формулою (2.3):

$$TCF = 0,6 + 0,01 \cdot \sum_{i=1}^{13} w_i \cdot f_i, \quad (2.3)$$

де w_i – вага i -го технічного фактора; f_i – оцінка його впливу.

Таблиця 2.3 – Оцінка технічних факторів

№	Фактор	Вага	Оцінка	Внесок	Обґрунтування
T1	Розподілена система	2	5	10	вебзастосунок, сервер SFU, настільний застосунок, СКБД
T2	Вимоги до швидкодії	2	5	10	доставлення відео в реальному часі
T3	Ефективність роботи користувача	1	3	3	запуск трансляції в кілька дій
T4	Складність обробки	1	4	4	параметри кодування,

№	Фактор	Вага	Оцінка	Внесок	Обґрунтування
					відстеження процесів
T5	Повторне використання коду	1	3	3	спільні типи, модулі перевірки оточення й доступу до бази даних
T6	Простота встановлення	0,5	3	1,5	інсталятори, автооновлення
T7	Простота використання	0,5	4	2	гостьовий перегляд без реєстрації
T8	Переносимість	2	4	8	браузери; Windows, macOS, Linux
T9	Простота змін	1	3	3	шари контролерів, сервісів і доступу до даних
T10	Паралельність	1	4	4	багато глядачів, кілька робочих процесів
T11	Безпека	1	3	3	сеанси JWT, РКСЕ, обмеження маршрутів
T12	Доступ для третіх сторін	1	1	1	відкритого API немає
T13	Потреба в навчанні	1	1	1	не потрібне
	Разом			53,5	

За формулою (2.3) $T C F = 0,6 + 0,01 \cdot 53,5 = 1,135$.

Далі оцінимо вісім факторів середовища, що характеризують виконавця й умови розробки; два останні мають від'ємну вагу. Коефіцієнт обчислюється за формулою (2.4):

$$E F = 1,4 - 0,03 \cdot \sum_{i=1}^8 v_i \cdot e_i, \quad (2.4)$$

де v_i – вага i -го фактора середовища; e_i – оцінка його впливу.

Фактори середовища оцінено для автора роботи з обґрунтуванням кожної оцінки (табл. 2.4).

Таблиця 2.4 – Оцінка факторів середовища

№	Фактор	Вага	Оцінка	Внесок	Обґрунтування оцінки
E1	Знайомство з процесом розробки RUP	1,5	1	1,5	процес RUP формально не застосовувався
E2	Досвід розроблення подібних застосунків	0,5	0	0	система такого класу розробляється вперше
E3	Досвід об'єктно-орієнтованої розробки	1	0	0	досвіду розроблення подібної системи немає
E4	Кваліфікація аналітика	0,5	3	1,5	середній рівень навичок збирання й формалізації

№	Фактор	Вага	Оцінка	Внесок	Обґрунтування оцінки
					вимог
E5	Мотивація	1	5	5	висока зацікавленість у результаті
E6	Стабільність вимог	2	3	6	вимоги розширювалися: захоплення звуку застосунку та режим слідування додано на завершальних етапах
E7	Зайнятість розробника	-1	0	0	розробник повністю зайнятий у проєкті
E8	Складність мови програмування	-1	2	-2	TypeScript середньої складності
	Разом			12	

За формулою (2.4) $EF = 1,4 - 0,03 \cdot 12 = 1,04$.

Залишається обчислити скориговані точки варіантів використання за формулою (2.5):

$$UCP = (UAW + UUCW) \cdot TCF \cdot EF = 93 \cdot 1,135 \cdot 1,04 \approx 109,8. \quad (2.5)$$

Трудомісткість визначається добутком UCP на коефіцієнт продуктивності за формулою (2.6):

$$E = PF \cdot UCP, \quad (2.6)$$

де PF – кількість людино-годин на одну точку.

Карнер запропонував $PF = 20$ люд.-год, а Шнайдер і Вінтерс – вибір між 20, 28 і 36 люд.-год залежно від кількості несприятливих факторів середовища: оцінок, менших за 3, серед E1–E6 і більших за 3 серед E7–E8 [27]. У табл. 2.4 таких оцінок три (E1, E2, E3), що відповідає $PF = 28$. Отже, оцінна трудомісткість $E = 28 \cdot 109,78 \approx 3074$ люд.-год.

Така трудомісткість перевищує можливості одного розробника в межах часу, відведеного на кваліфікаційну роботу, тому план зосереджено на реалізації шести варіантів використання, а значну частину функцій покладено на готові

компоненти (mediasoup, Auth.js, Next.js, Electron), так що повністю самостійно розробляються лише захоплення окремих застосунків і режим слідування.

2.2 Розробка плану виконання проєкту

Для структурування проєкту загальний обсяг робіт поділено на дев'ять робіт, кожна з яких має визначений результат.

Робота «Створення каркаса системи та базової трансляції» передбачає створення вебзастосунку, сервера сигналізації із сервером пересилання медіапотоків, входу через Google та модуля доступу до бази даних. Результат – трансляція у браузері для глядачів.

Робота «Гостьовий доступ і керування трансляцією» передбачає гостьові сеанси для перегляду без реєстрації, приватні трансляції, вимкнення звуку стримером, сповіщення, мініатюри трансляцій та очищення неактивних сеансів. Результат – функціонально завершене ядро вебсервісу.

Робота «Підготовка розгортання» включає опис контейнерів сервісів, налаштування зворотного проксі Caddy та перевірку змінних оточення під час запуску. Результат – система, розгорнута на сервері з доступом через HTTPS.

Робота «Доопрацювання інтерфейсу» спрямована на вдосконалення сторінок вебзастосунку. Результат – оновлений інтерфейс.

Робота «Захоплення окремих застосунків» передбачає вибір джерела захоплення, передавання власного звуку застосунку, пакування й випуск настільного застосунку та кнопку його завантаження на головній сторінці. Результат – настільний застосунок, що транслює зображення та звук лише обраного застосунку.

Робота «Керування трансляцією, автооновлення та вхід через системний браузер» охоплює керування трансляцією з настільного застосунку, його автоматичне оновлення та вхід через Google у системному браузері. Результат – настільний застосунок, готовий до самостійного використання.

Робота «Оптимізація кодування відео» передбачає узгодження бітрейту й апаратне кодування H.264. Результат – перший випуск настільного застосунку 0.0.1.

Робота «Режим слідування» передбачає автоматичне перемикання трансляції на нові вікна обраного застосунку та повернення до попереднього вікна. Результат – випуск настільного застосунку 1.0.0.

Робота «Тестування системи та оформлення пояснювальної записки» включає функціональне й нефункціональне тестування та опис системи. Результат – пояснювальна записка.

Терміни та попередники робіт наведено в табл. 2.5.

Таблиця 2.5 – Тривалість та залежності робіт проекту

№	Робота	Терміни	Тривалість, дн.	Попередники
1	Створення каркаса системи та базової трансляції	27.05–13.07	48	–
2	Гостьовий доступ і керування трансляцією	14.07–12.08	30	1
3	Підготовка розгортання	13.08–29.08	17	2
4	Доопрацювання інтерфейсу	30.08–03.09	5	3
5	Захоплення окремих застосунків	04.09–14.09	11	1, 4
6	Керування трансляцією, автооновлення та вхід через системний браузер	15.09–19.09	5	5
7	Оптимізація кодування відео	20.09	1	6
8	Режим слідування	21.09–25.09	5	5, 7
9	Тестування системи та оформлення пояснювальної записки	з 05.10	[ПОТРЕБУЄ УТОЧНЕННЯ: дата завершення]	8

Робота 1 створює основу для всіх подальших робіт, робота 2 розширює її модель трансляцій і сеансів, робота 3 розгортає завершене ядро, а інтерфейс у роботі 4 доопрацьовується на розгорнутій системі. Робота 5 використовує конвеєр трансляції роботи 1, робота 6 – настільний застосунок роботи 5, випуск роботи 7 можливий після робіт 5 і 6, робота 8 спирається на захоплення застосунків і перший випуск, а робота 9 потребує заведеної версії системи.

Роботи на діаграмі Ганта (рис. 2.1) згруповано в етапи: розробка вебсервісу (1–4), настільного застосунку (5–8), тестування й документування (9).

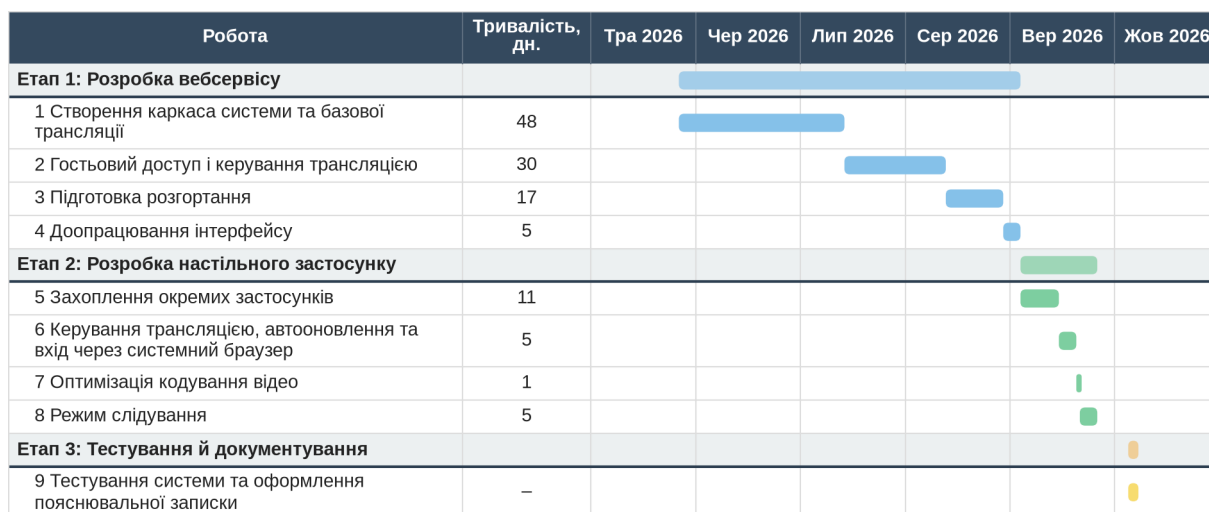


Рисунок 2.1 – Діаграма Ганта проекту

Роботи виконуються послідовно, бо їх виконує один розробник. Найтривалішою та найтрудомісткішою є робота 1, далі – робота 2. Критичною є робота 5: захоплення окремих застосунків становить новизну системи, від нього залежать роботи 6–8. Розробка програмної частини триває 122 календарні дні (27.05–25.09.2026).

2.3 Аналіз ризиків та планування реакцій

Імовірність і вплив ризиків оцінено експертно за трирівневою шкалою, стратегією реагування є уникнення, пом'якшення або прийняття (табл. 2.6).

Таблиця 2.6 – Реєстр ризиків проекту

№	Ризик	Імовірність	Вплив	Реагування
R1	Медіаз'єднання не встановлюється через NAT або неправильну публічну адресу сервера	середня	високий	пом'якшення: обов'язкова змінна оточення MEDIASOUP_ANNOUNCED_IP, без якої сервер не запускається; інструкція з розгортання
R2	Google відхиляє	висока	високий	уникнення: вхід у системному

№	Ризик	Імовірність	Вплив	Реагування
	вхід із вбудованого вікна Electron			браузері з РКСЕ і локальною адресою повернення
R3	Нативний модуль захоплення звуку не збирається для версії Electron або ОС	середня	високий	пом'якшення: перекомпіляція під час встановлення, збирання в CI для кожної ОС; трансляція лише відео
R4	Немає апаратного кодувальника H.264, режими 1080p60 і вище деградують	середня	середній	пом'якшення: порядок кодеків H.264 Baseline, резервний VP8, вибір пріоритету частоти кадрів чи роздільності
R5	Інсталювальники без цифрового підпису: попередження ОС, немає автооновлення на macOS	висока	середній	прийняття: необов'язковий крок підпису в CI, ручне оновлення на macOS
R6	Перевантаження одного сервера кількістю глядачів	середня	середній	пом'якшення: кілька робочих процесів mediasoup, вибір найменш завантаженого
R7	Регресії через відсутність автоматизованих тестів	висока	середній	пом'якшення: перевірки lint, typecheck, build і ручне тестування
R8	Зміна й розширення вимог під час розробки	середня	середній	пом'якшення: поетапне додавання функцій, модульна структура
R9	Зрив графіка через малу кількість виконавців	середня	високий	пом'якшення: пріоритет основних варіантів використання, короткі етапи

Ризик R2 [24, 26] усунуто в роботі 6, R3 і R4 – у роботах 5 і 7, R8 і R9 – планом робіт 5–8.

Висновки до розділу 2

Трудомісткість оцінено методом точок варіантів використання: $UCP \approx 109,8$, або близько 3074 люд.-год. Дев'ять робіт виконуються з 27.05 до 25.09.2026, критична – захоплення окремих застосунків; заходи щодо ключових ризиків реалізовано в коді.

3 ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

3.1 Архітектура програмної системи

Програмну систему організовано як монорепозіторій `pprt`, у якому три виконувані модулі спираються на три спільні модулі. Модулі та зв'язки між ними наведено на діаграмі компонентів (рис. 3.1), побудованій у нотації UML [21]. Вихідний код ключових модулів наведено в додатку А.

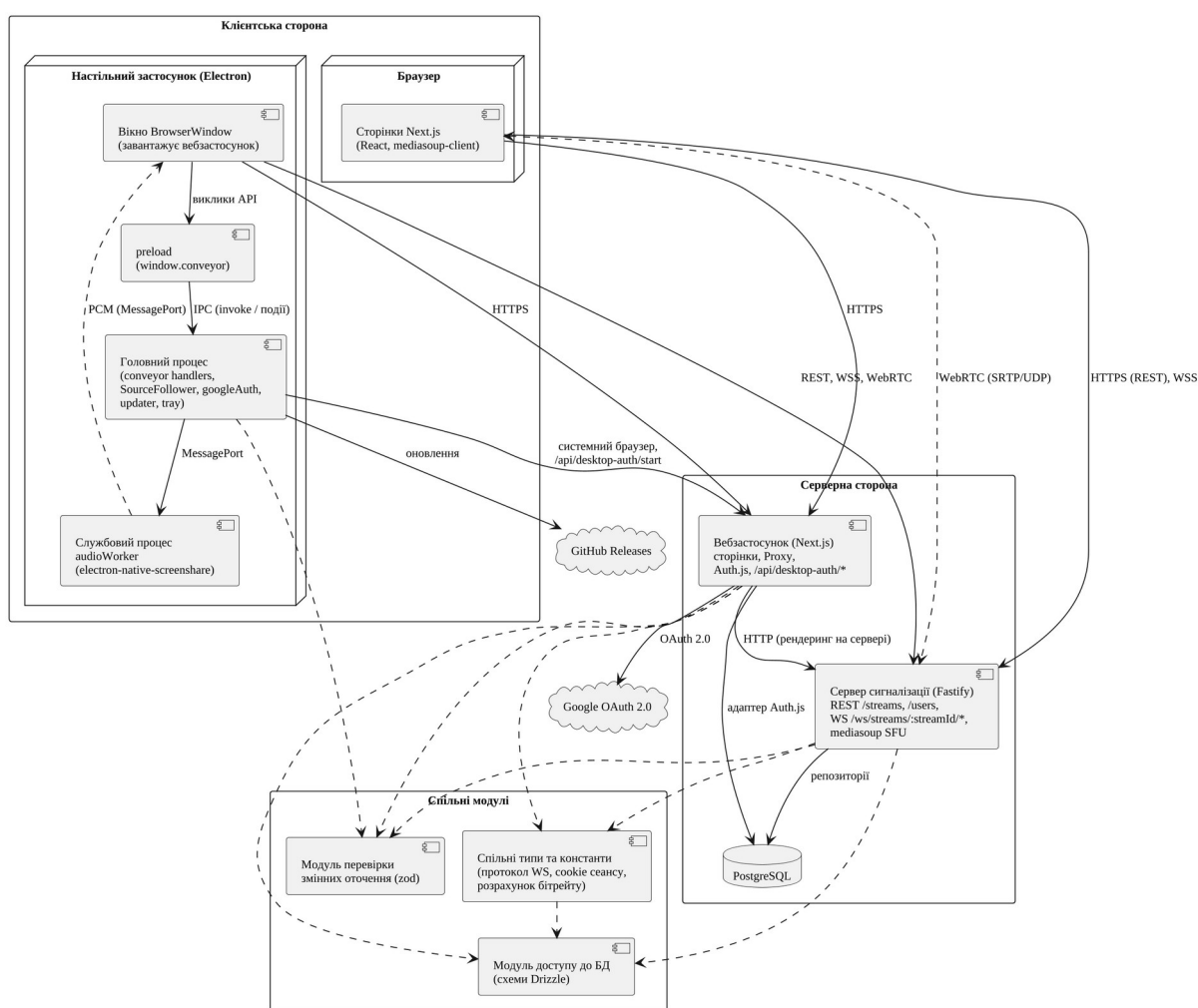


Рисунок 3.1 – Діаграма компонентів програмної системи

Вебзастосунок (Next.js) формує сторінки, виконує автентифікацію через Auth.js і під час серверного рендерингу отримує дані про трансляції від сервера сигналізації внутрішнім HTTP-запитом. Сервер сигналізації (Fastify) надає REST-

інтерфейс і WebSocket-канали для сигнальних повідомлень, а також пересилає медіапотоки як SFU на основі mediasoup [12]: потік стримера надходить на сервер один раз і розсилається кожному глядачеві без перекодування.

Настільний застосунок (Electron) не дублює інтерфейс: його вікно завантажує розгорнутий вебзастосунок, а головний процес [28] надає сторінці через об'єкт `window.conveyor` лише недоступне браузеру – перелік вікон з ідентифікаторами процесів, захоплення звуку застосунку в утилітарному процесі, слідування за вікнами та вхід через системний браузер.

Модуль доступу до бази даних містить схеми таблиць Drizzle ORM, спільні типи та константи – типи повідомлень протоколу, ім'я cookie сеансу та розрахунок бітрейту, модуль перевірки змінних оточення – перевірку конфігурації за схемами zod. Тому зміна протоколу чи схеми даних перевіряється компілятором в усіх модулях. Зовнішні системи – Google OAuth 2.0 для підтвердження особи та GitHub Releases для інсталяторів і автоматичного оновлення настільного застосунку.

Виробниче середовище описано засобами Docker Compose; його діаграму розгортання наведено на рис. 3.2.

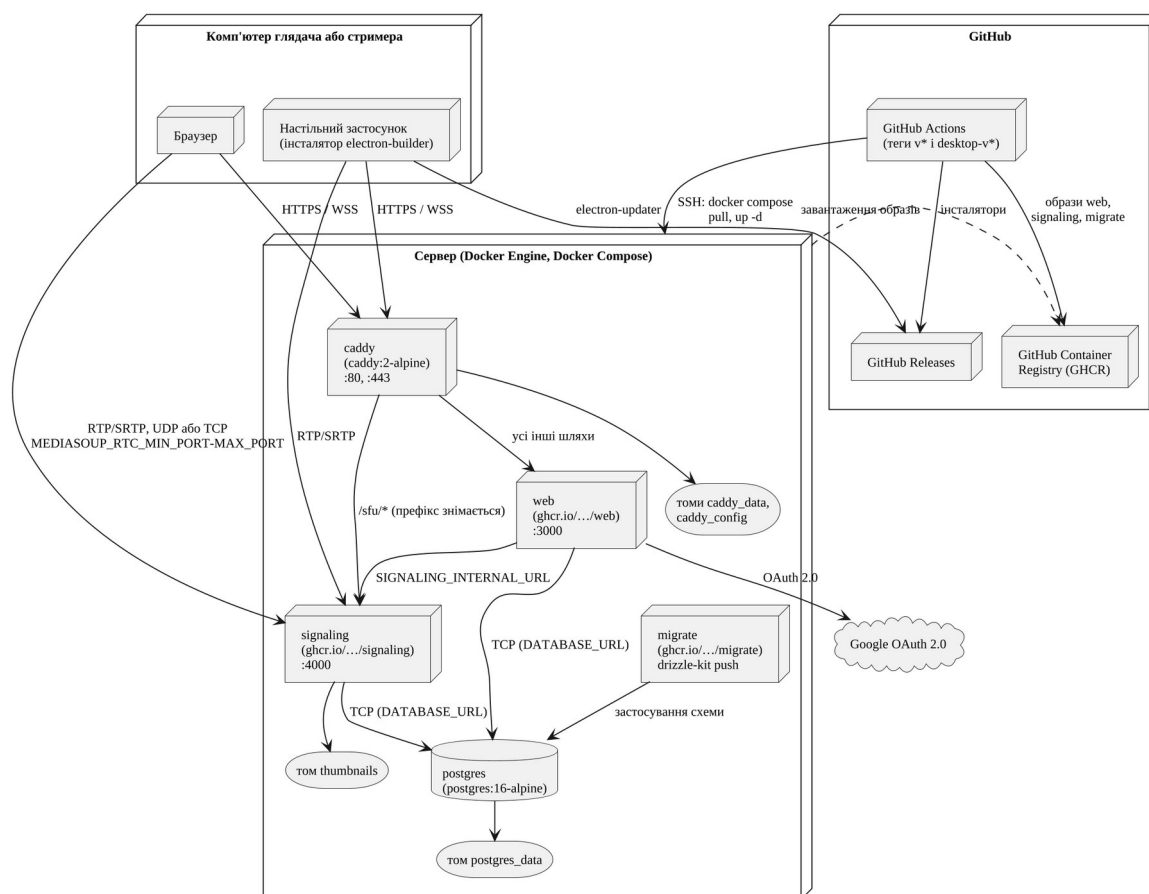


Рисунок 3.2 – Діаграма розгортання програмної системи

На сервері працюють п'ять сервісів. Сервіс caddy (порти 80, 443) автоматично отримує TLS-сертифікат і спрямовує запити з префіксом /sfu до сервера сигналізації, знімаючи префікс, а решту – до сервісу web, який запускає вебзастосунок Next.js з образу в GitHub Container Registry; змінні оточення AUTH_URL (публічна HTTPS-адреса сервера) і AUTH_TRUST_HOST налаштовують Auth.js на роботу за зворотним проксі. Сервіс signaling (порт 4000) додатково публікує діапазон портів UDP і TCP, заданий змінними оточення MEDIASOUP_RTC_MIN_PORT і MEDIASOUP_RTC_MAX_PORT, через який медіадані йдуть до клієнтів в обхід Caddy. Сервіс postgres на основі образу PostgreSQL 16 зберігає дані, а сервіс migrate одноразово застосовує схему бази даних засобами drizzle-kit. Мініатюри, дані СУБД і сертифікати зберігаються в томах; доставку змін на сервер автоматизовано засобами GitHub Actions.

3.2 Проєктування процесів у системі

Клієнти та сервер обмінюються JSON-повідомленнями трьох типів: запит методу з ідентифікатором, відповідь з тим самим ідентифікатором, що містить результат або помилку, та подія сервера. Методи й події протоколу сигналізації наведено в табл. 3.1.

Таблиця 3.1 – Методи та події протоколу сигналізації

Метод / подія	Канал	Напрямок	Параметри → результат
getRtpCapabilities	broadcast	клієнт → сервер	– → можливості RTP маршрутизатора
createTransport	обидва	клієнт → сервер	напрямок (send / recv) → ідентифікатор, параметри ICE і DTLS, кандидати ICE
connectTransport	обидва	клієнт → сервер	параметри DTLS клієнта → –
produce	broadcast	клієнт → сервер	вид доріжки, параметри RTP, ознака last, максимальний бітрейт → ідентифікатор продюсера
endStream	broadcast	клієнт → сервер	– → –
joinStream	watch	клієнт → сервер	– → ідентифікатори продюсерів, можливості RTP
consume	watch	клієнт → сервер	можливості RTP клієнта, ідентифікатор продюсера → ідентифікатор споживача, вид доріжки, параметри RTP
streamEnd	watch	сервер → клієнт	трансляцію завершено
streamerDisconnect	watch	сервер → клієнт	з'єднання зі стримером втрачено
streamerReconnected	watch	сервер → клієнт	ідентифікатор нового продюсера

Методи відповідають етапам встановлення WebRTC-сеансу [6, 7]: для кожного напрямку створюється окремий транспорт, і після обміну параметрами DTLS медіадані йдуть протоколом SRTP [8].

Публікацію трансляції наведено на діаграмі послідовності (рис. 3.3). Вона деталізує кроки 8–9 сценарію 1.4, які використовує й сценарій 1.5.

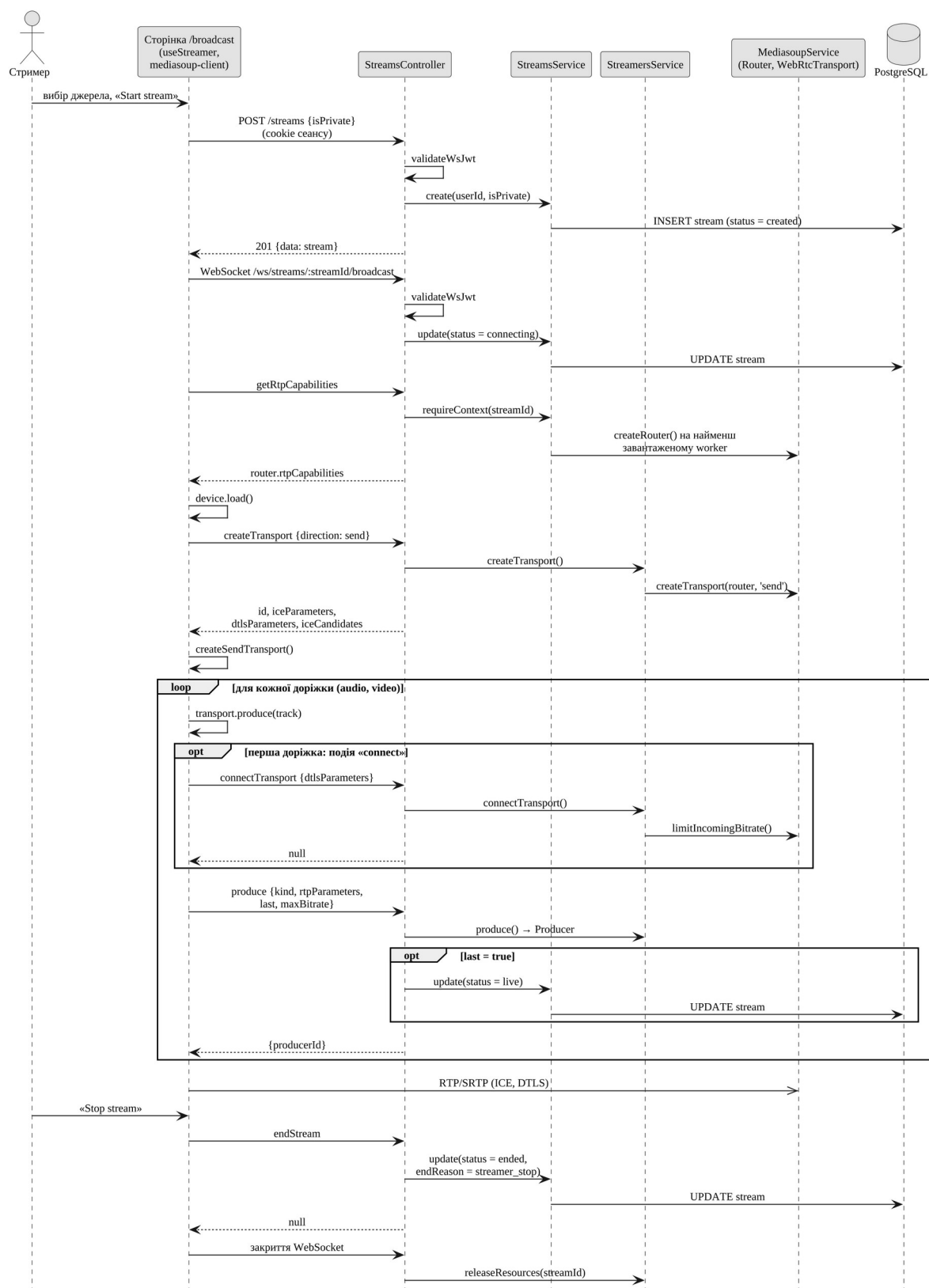


Рисунок 3.3 – Діаграма послідовності публікації трансляції

Сторінка трансляції створює запис трансляції REST-запитом і відкриває WebSocket-канал стримера. Обидва звернення проходять перевірку функцією `validateWsJwt`, яка розшифровує cookie сеансу `Auth.js` спільним секретом, тож сервер сигналізації перевіряє сеанс без звернення до вебзастосунку. Підключення переводить трансляцію в стан `connecting`. Запит `getRtpCapabilities` створює маршрутизатор `mediasoup` на найменш завантаженому процесі-обробнику (`worker`). Далі клієнт викликає `produce` для кожної доріжки, і ознака `last` останньої доріжки переводить трансляцію в стан `live`. Запит `endStream` фіксує завершення з причиною `streamer_stop`, а закриття сокета звільняє ресурси `mediasoup`.

Контекст активної трансляції (маршрутизатор, транспорти, продюсери й споживачі) зберігається лише в пам'яті сервера, а в базі даних – тільки статус і час трансляції.

Наступним розглянемо процес перегляду, що відповідає сценарію 1.3 (рис. 3.4).

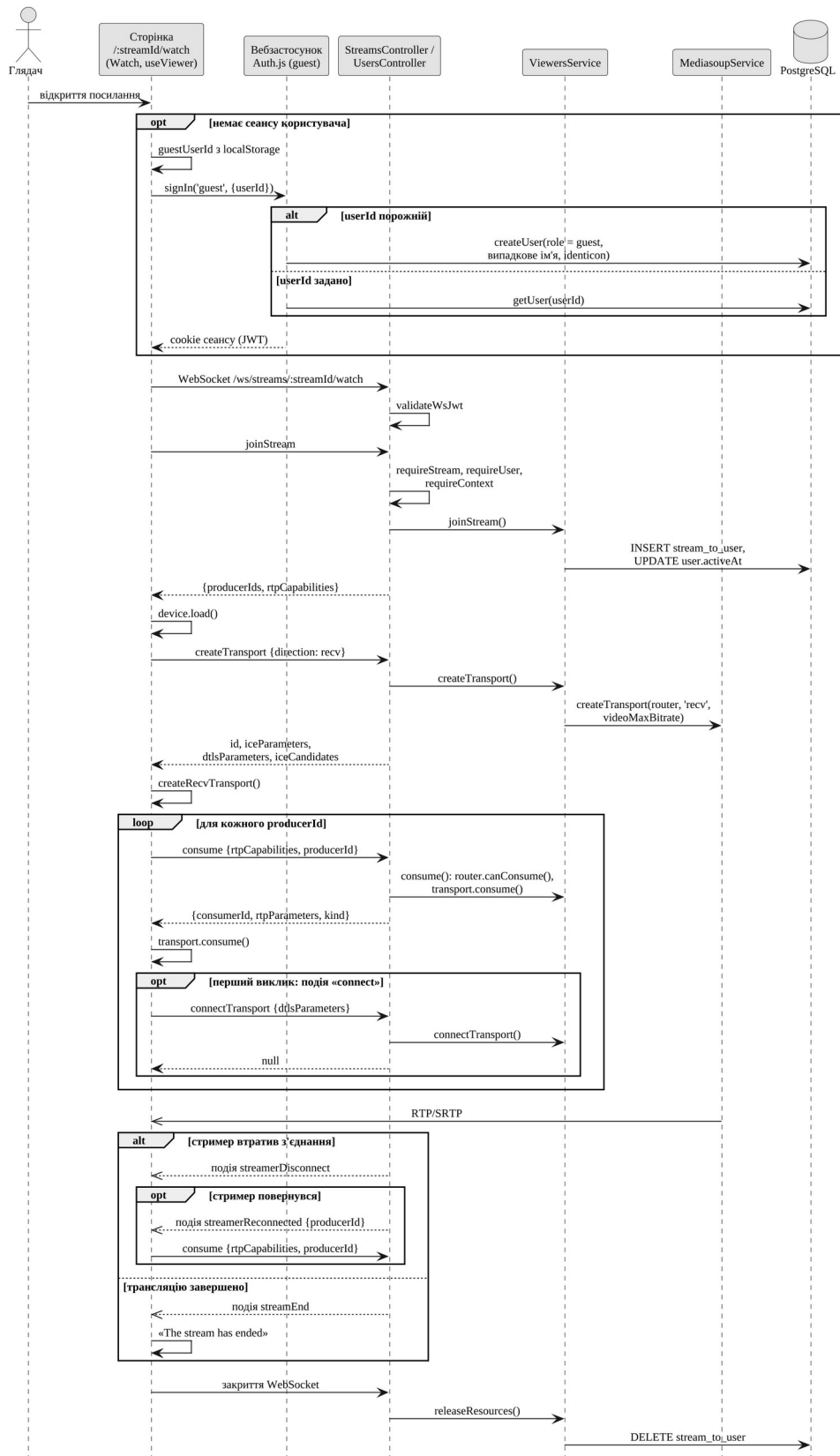


Рисунок 3.4 – Діаграма послідовності перегляду трансляції

Сторінку перегляду виключено з переадресації на сторінку входу. Відвідувачу без сеансу вона виконує вхід через гостьовий провайдер (ФВ-03),

який повторно використовує запис гостя, збережений у локальному сховищі браузера, або створює новий з роллю guest, випадковим іменем та аватаром. Запит `joinStream` додає глядача до таблиці `stream_to_user` і повертає ідентифікатори продюсерів; клієнт створює приймальний транспорт, початковий бітрейт якого сервер обчислює з максимального бітрейту відео трансляції, і виконує `consume` для кожного продюсера. Подію `streamerReconnected` клієнт обробляє споживанням нового продюсера, а `streamEnd` – заміною плеєра повідомленням про завершення; подію `streamerDisconnect` сервер надсилає, але клієнт її не обробляє. Закриття сокета видаляє запис глядача з таблиці `stream_to_user`.

Окремий процес становить вхід у настільному застосунку. Google блокує OAuth-вхід у вбудованих вебпереглядачах [26], тому нативним застосункам рекомендовано використовувати системний браузер і локальну адресу зворотного виклику [24]. Взаємодію, що реалізує ФВ-02, наведено на рис. 3.5.

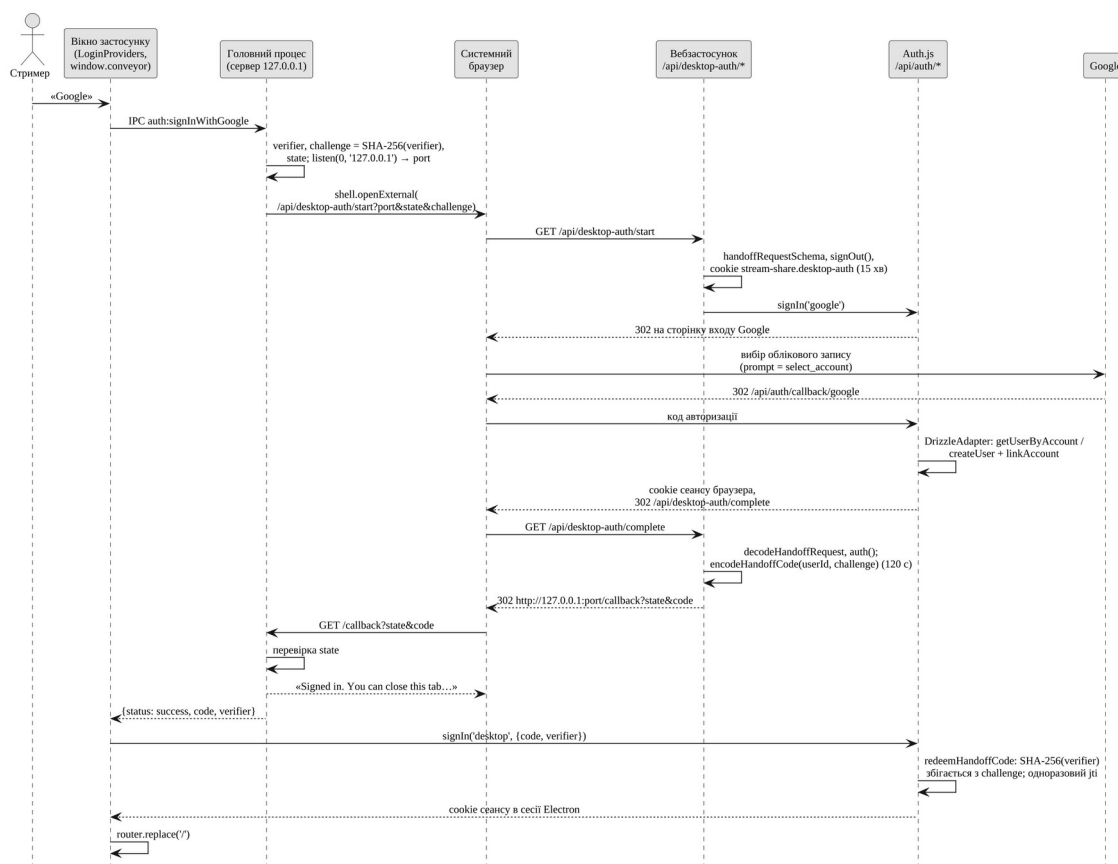


Рисунок 3.5 – Діаграма послідовності входу в настільному застосунку через системний браузер

Головний процес за схемою РКСЕ [25] генерує перевірочний рядок і його хеш SHA-256, запускає локальний HTTP-сервер і відкриває в системному браузері вхід вебзастосунку. Після входу через Google вебзастосунок видає пов'язаний із хешем одноразовий код (дійсний 120 с) і перенаправляє браузер на локальний сервер, а сторінка обмінює код і перевірочний рядок на сеанс. Перехоплений код без перевірконого рядка непридатний, а повторне використання коду блокується.

Вимога ФВ-09 передбачає перехід трансляції на нові вікна застосунку, зокрема відкриті його дочірніми процесами, і повернення до початкового вікна. Алгоритм класу SourceFollower, що реалізує цей режим, наведено на діаграмі діяльності (рис. 3.6).

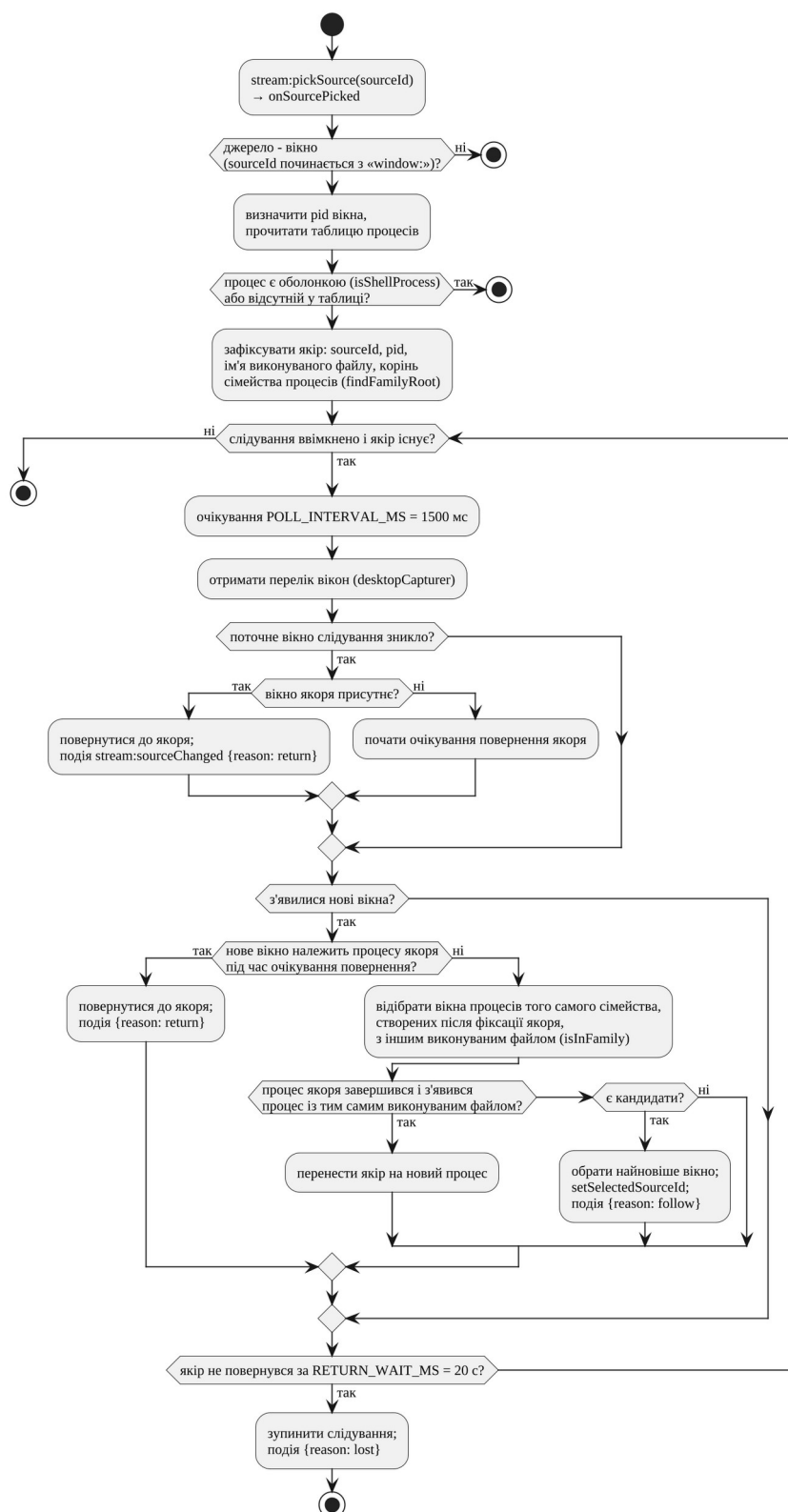


Рисунок 3.6 – Діаграма діяльності режиму слідування за застосунком

Обране вікно стає «якорем» (вікна процесів-оболонки якорем не стають). Кожні 1500 мс клас порівнює перелік вікон від desktopCapturer [13] з попереднім і

переходить на найновіше вікно процесу того самого сімейства з іншим виконуваним файлом; після його зникнення трансляція повертається до якоря, а якщо зникло й воно, слідування вимикається через 20 с. Сторінка замінює доріжку без перестворення трансляції.

Стани трансляції (поле status таблиці stream) і причини завершення (поле endReason) наведено на діаграмі станів (рис. 3.7).

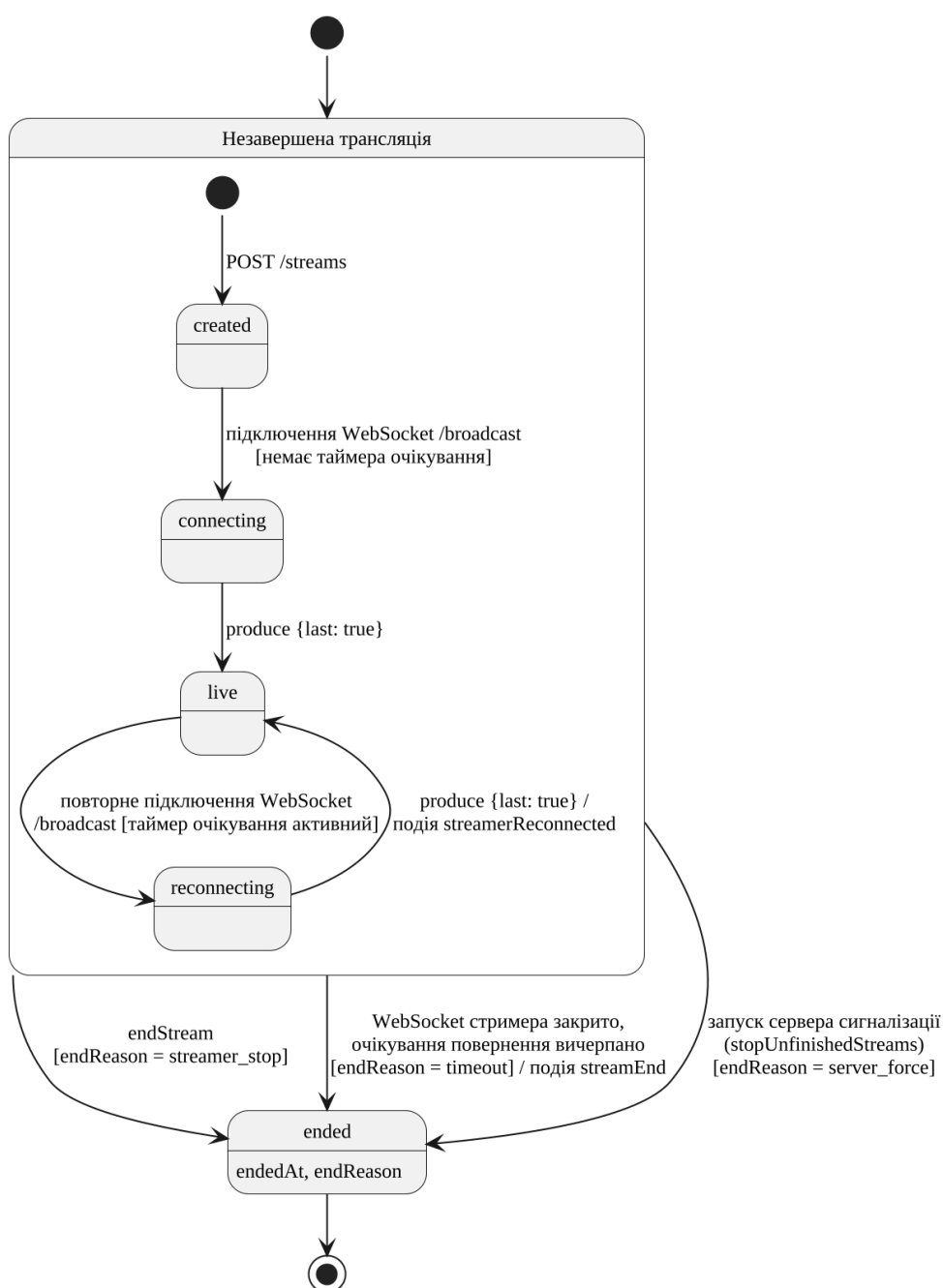


Рисунок 3.7 – Діаграма станів трансляції

Якщо сокет стримера закривається до запиту `endStream`, сервер надсилає глядачам подію `streamerDisconnect` і запускає таймер з інтервалом 6 с та обмеженням у п'ять спроб (ФВ-16), не змінюючи статусу в базі даних. Повторне підключення стримера за активного таймера дає стан `reconnecting`, а `produce` з ознакою `last` – знову `live`; інакше трансляція завершується з причиною `timeout`. Під час запуску сервер завершує незавершені трансляції з причиною `server_force`, оскільки їхній контекст у пам'яті втрачено.

3.3 Проєктування структури бази даних

Схему бази даних, визначену в модулі доступу до бази даних і застосовану до PostgreSQL засобами Drizzle, наведено на діаграмі «сутність – зв'язок» у нотації IE (рис. 3.8).

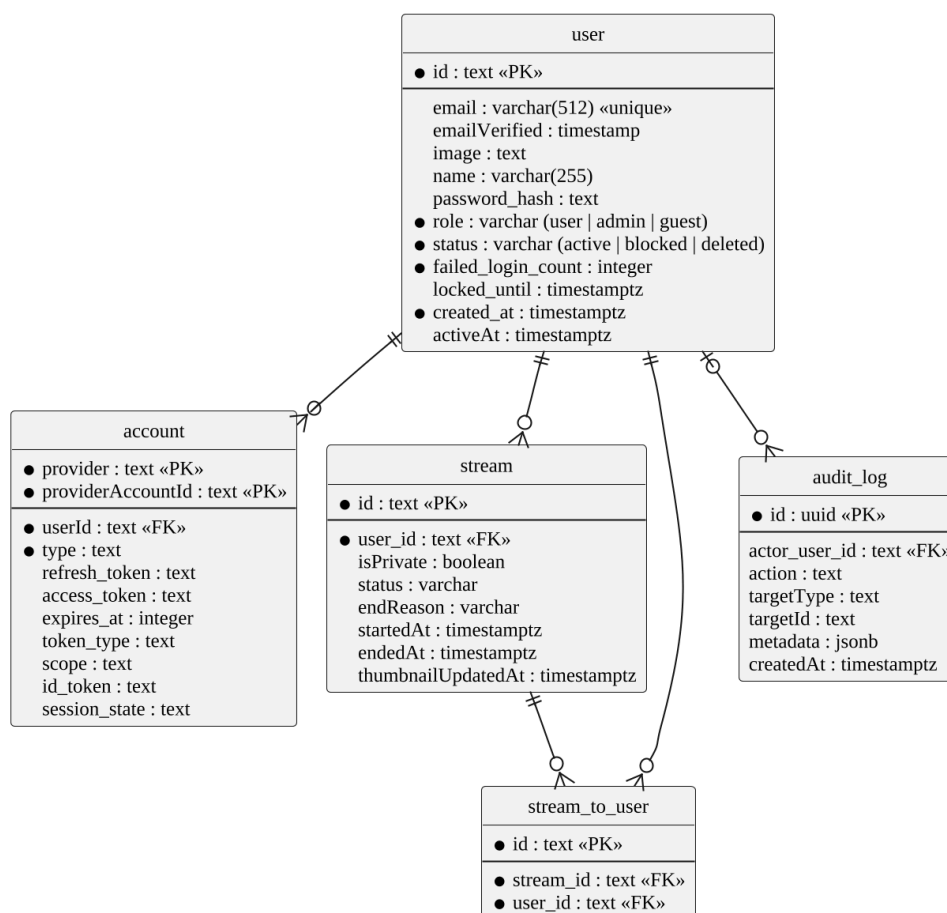


Рисунок 3.8 – Діаграма «сутність – зв'язок» бази даних

Схема містить п'ять таблиць, опис яких наведено в табл. 3.2–3.6. Порожніх значень не допускають поля з обмеженнями PK і NOT NULL, а також усі зовнішні ключі (FK), крім actor_user_id. Усі зовнішні ключі мають правило ON DELETE CASCADE, тож щотижневе очищення гостей (ФВ-19), яке видаляє гостей без активності понад 7 днів, що не дивляться жодної трансляції, автоматично видаляє й пов'язані записи. Сеанси зберігаються в зашифрованому JWT у cookie [29], тому таблиць сеансів у схемі немає.

Таблиця user (табл. 3.2) є центральною: вона зберігає як користувачів, що увійшли через Google, так і гостей, яких розрізняє поле role. Поле role набуває значень user, admin і guest, поле status – active, blocked і deleted; новий запис типово отримує роль user і стан active, а поля created_at і activeAt – поточний час. Разом із таблицею account таблиця має структуру, якої очікує адаптер Auth.js: власний адаптер Drizzle створює користувача під час першого входу. Поля password_hash, failed_login_count і locked_until, стани blocked і deleted та роль admin закладено для входу за паролем і модерації; у поточній версії їх не заповнює жоден модуль.

Таблиця 3.2 – Опис таблиці user

Поле	Тип даних	Обмеження	Опис
id	text	PK	Ідентифікатор (UUID)
email	varchar(512)	UNIQUE	Електронна пошта
emailVerified	timestamp	–	Підтвердження пошти
name	varchar(255)	–	Ім'я користувача
image	text	–	Адреса аватара
role	varchar	NOT NULL	Роль
status	varchar	NOT NULL	Стан запису
password_hash	text	–	Хеш пароля
failed_login_count	integer	NOT NULL	Невдалі спроби входу
locked_until	timestampz	–	Кінець блокування
created_at	timestampz	NOT NULL	Час створення запису
activeAt	timestampz	–	Остання активність

Таблиця account (табл. 3.3) пов'язує користувача з обліковим записом провайдера OAuth і зберігає токени, отримані під час входу через Google.

Первинний ключ складено з полів provider і providerAccountId, тому той самий обліковий запис провайдера не може бути прив'язаний двічі.

Таблиця 3.3 – Опис таблиці account

Поле	Тип даних	Обмеження	Опис
userId	text	FK (user)	Власник запису
type	text	NOT NULL	Тип за Auth.js
provider	text	PK	Провайдер
providerAccountId	text	PK	Код у провайдера
refresh_token	text	—	Токен оновлення
access_token	text	—	Токен доступу
expires_at	integer	—	Кінець дії токена
token_type	text	—	Тип токена
scope	text	—	Області доступу
id_token	text	—	ID-токен
session_state	text	—	Стан сеансу

Таблиця stream (табл. 3.4) зберігає трансляції; її ідентифікатор входить до посилання на перегляд. Значення полів status (типово created) і endReason відповідають діаграмі станів (рис. 3.7); причину admin_force оголошено, але не використовується. Приватні трансляції, позначені полем isPrivate, не потрапляють до загального переліку.

Таблиця 3.4 – Опис таблиці stream

Поле	Тип даних	Обмеження	Опис
id	text	PK	Ідентифікатор (UUID)
user_id	text	FK (user)	Стример
isPrivate	boolean	—	Ознака приватності
status	varchar	—	Стан трансляції
endReason	varchar	—	Причина завершення
startedAt	timestamptz	—	Час початку
endedAt	timestamptz	—	Час завершення
thumbnailUpdatedAt	timestamptz	—	Оновлення мініатюри

Таблиця stream_to_user (табл. 3.5) реалізує зв'язок «багато до багатьох» між трансляціями та їхніми поточними глядачами.

Таблиця 3.5 – Опис таблиці stream_to_user

Поле	Тип даних	Обмеження	Опис
id	text	PK	Ідентифікатор (UUID)
stream_id	text	FK (stream)	Трансляція
user_id	text	FK (user)	Глядач

Таблиця audit_log (табл. 3.6) призначена для журналу дій модерації; у поточній версії її не заповнює жоден модуль.

Таблиця 3.6 – Опис таблиці audit_log

Поле	Тип даних	Обмеження	Опис
id	uuid	PK	Ідентифікатор (UUID)
actor_user_id	text	FK (user)	Автор дії
action	text	–	Дія
targetType	text	–	Тип об'єкта дії
targetId	text	–	Ідентифікатор об'єкта
metadata	jsonb	–	Додаткові дані
createdAt	timestampz	–	Час дії

3.4 Проєктування класів

Код сервера сигналізації поділено на три шари: контролери, сервіси та репозиторії. Діаграму класів наведено на рис. 3.9.

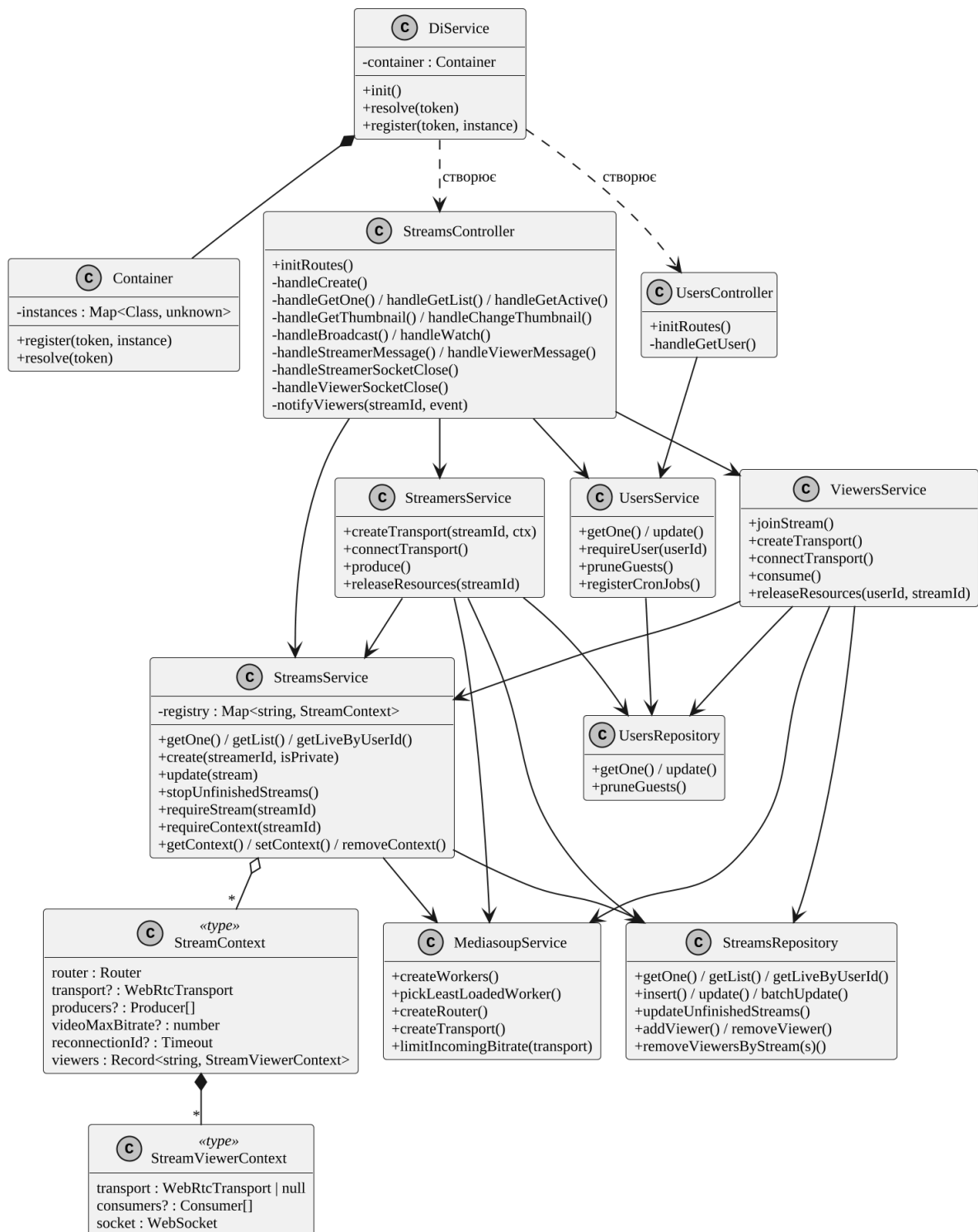


Рисунок 3.9 – Діаграма класів сервера сигналізації

Контролери реєструють маршрути Fastify: клас `StreamsController` – REST-маршрути трансляцій і обидва WebSocket-канали, клас `UsersController` – маршрут отримання користувача. Сервіси містять бізнес-логіку: `StreamsService` – трансляції та реєстр контекстів, `StreamersService` і `ViewersService` – транспорти, продюсери й

споживачі стримера та глядачів, UsersService – користувачів і очищення гостей, MediasoupService – процеси-обробники, маршрутизатори та транспорти. Репозиторії інкапсулюють запити Drizzle. Залежності передаються через конструктори: замість DI-фреймворку клас Container зберігає словник «клас → екземпляр», а клас DiService, доступний як декоратор Fastify, під час запуску створює об'єкти в порядку залежностей. Граф об'єктів явний і статично типізований.

Сторінка настільного застосунку працює з увімкненою ізоляцією контексту і не має прямого доступу до API Node.js та Electron. Шар міжпроцесної взаємодії (у коді – conveyor) наведено на рис. 3.10.

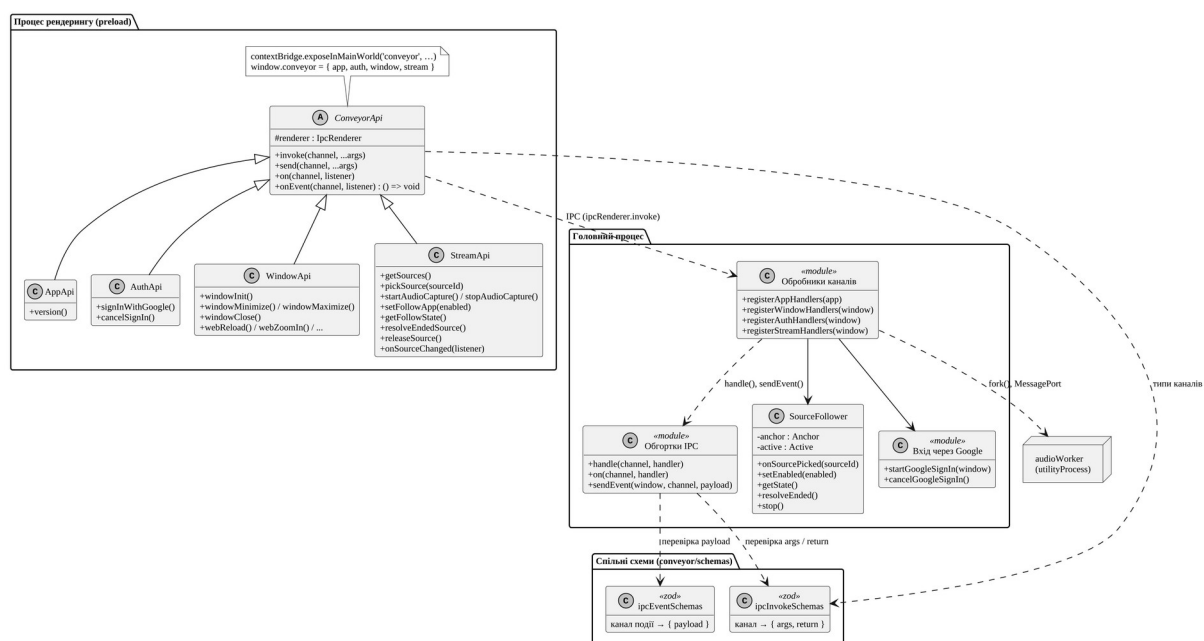


Рисунок 3.10 – Діаграма класів міжпроцесної взаємодії настільного застосунку

Абстрактний клас ConveyorApi обгортає модуль ipcRenderer; його нащадків AppApi, AuthApi, WindowApi і StreamApi сценарій preload публікує як об'єкт window.conveyor через contextBridge. У головному процесі обробники каналів реєструються спільною обгорткою, через яку надсилаються їм події. Обидві сторони спираються на схеми zod ipcInvokeSchemas (аргументи й результат каналу) та ipcEventSchemas (дані подій): сторона рендерингу отримує з них типи, а головний процес перевіряє дані під час виконання. Захоплення звуку виконує

утилітарний процес [28], який передає дані сторінці напряму через пару портів повідомлень.

3.5 Проєктування інтерфейсу користувача

Головна сторінка відвідувачу без входу показує опис сервісу та кнопку завантаження настільного застосунку, а користувачу – перелік активних публічних трансляцій. Кнопка «Start stream» у шапці (для гостей прихована) веде на сторінку трансляції, а сторінка перегляду відкривається з картки трансляції або за посиланням. Написи інтерфейсу англійські, тому на макетах їх наведено мовою оригіналу.

Макет сторінки трансляції, побудований засобами PlantUML Salt, наведено на рис. 3.11.



Рисунок 3.11 – Макет сторінки трансляції

Під попереднім переглядом з індикатором стану розташовано панель керування: вибір джерела «Pick source» / «Change source», групи якості (360p–1440p, «Source») і частоти кадрів (5, 30, 60) (ФВ-10), перемикачі звуку (ФВ-13) і приватності (ФВ-11), кнопка початку чи зупинки трансляції та поле посилання з кнопкою копіювання (ФВ-12). У настільному застосунку вибір джерела відкриває модальне вікно з вкладками «Applications» і «Entire Screen» (ФВ-07), а панель

містить перемикач слідування й напис «Following: ...» (ФВ-09). За наявності незавершеної трансляції замість панелі показано кнопку «Reconnect».

Макет сторінки перегляду наведено на рис. 3.12.

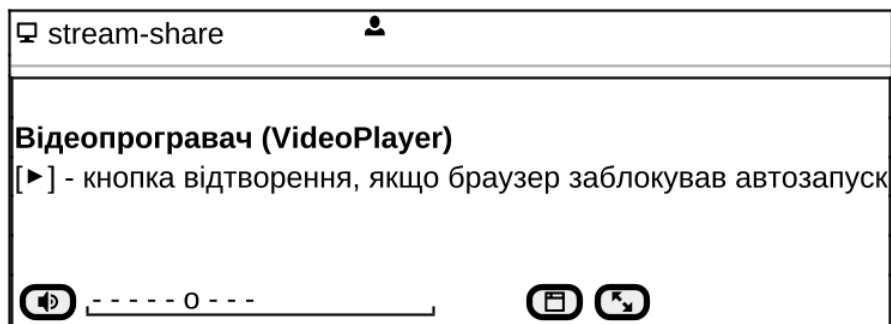


Рисунок 3.12 – Макет сторінки перегляду трансляції

Сторінка містить відеопрогравач з керуванням звуком і гучністю, режимом «картинка в картинці» та повноекранним режимом, доступними й клавішами M, P, F (ФВ-14). Вибору якості на боці глядача немає. Після завершення трансляції програвач замінюється повідомленням «The stream has ended».

Висновки до розділу 3

Спроектована система складається з вебзастосунку, сервера сигналізації з SFU-пересиланням медіапотоків і настільного застосунку-оболонки, які спираються на спільні модулі схем даних, протоколу та конфігурації. Систему розгортають як набір контейнерів Docker Compose за зворотним проксі Caddy; медіатрафік WebRTC іде напряму до сервера сигналізації через виділений діапазон портів.

Діаграми процесів формалізують публікацію й перегляд трансляції, вхід у настільному застосунку, слідування за вікнами та відновлення після втрати з'єднання. Модель даних з п'яти таблиць узгоджено з Auth.js, класи сервера поділено на контролери, сервіси та репозиторії, а міжпроцесну взаємодію типізовано схемами zod. Ці рішення є основою програмної реалізації системи.

4 ПРОГРАМНА РЕАЛІЗАЦІЯ СИСТЕМИ

4.1 Вибір інструментів розробки

Критеріями вибору інструментів розробки були єдина мова для всіх модулів, спільний код клієнта й сервера та підтримка WebRTC на кожній стороні. Склад технологічного стеку з версіями, зафіксованими у файлі блокування залежностей, наведено в табл. 4.1.

Таблиця 4.1 – Технології розробки програмної системи

Технологія	Версія	Призначення в системі
TypeScript, pnpm (workspaces)	6.0.3 (web), 7.0.2; 12.3.4	мова всіх модулів; монорепозиторій із трьох застосунків і спільних модулів
Next.js, React, Tailwind CSS	16.2.6, 19.2.4, 4.3.0	вебзастосунок
Auth.js (next-auth)	5.0.0-beta.31	вхід через Google, гостьові сеанси
Fastify, @fastify/websocket	5.8.5, 11.2.0	REST- і WebSocket- інтерфейси сервера сигналізації
mediasoup, mediasoup-client	3.20.5, 3.21.0	сервер SFU та його клієнт у браузері
PostgreSQL, Drizzle ORM, drizzle- kit	16, 1.0.0-beta.22	база даних, схеми, міграції
zod	4.4.3	перевірка конфігурації, повідомлень IPC, параметрів запитів
Electron, electron-native- screenshare	35.7.5, 1.2.0	настільний застосунок, звук окремого процесу
electron-vite, electron-builder, electron-updater	5.0.0, 26.8.1, 6.8.9	збирання, інсталятори, автоматичне оновлення
Docker Compose, Caddy, GitHub Actions	–, 2, –	розгортання, HTTPS, збирання й випуск

Основною мовою обрано TypeScript, яка виявляє помилки типів до виконання програми. Оскільки всі модулі працюють у середовищах JavaScript,

протокол WebSocket, розрахунок бітрейту, схеми бази даних і конфігурацію винесено в спільні пакети робочого простору npm, а схеми zod перевіряють дані під час виконання й водночас задають їхні статичні типи.

Для вебзастосунку обрано Next.js, що поєднує сторінки React та API-маршрути, а його механізм Proху перевіряє доступ до сторінок ще до їх формування (ФВ-04). Стратегія сеансу JWT в Auth.js зберігає сеанс у зашифрованому файлі cookie без звернень до бази даних, ціною неможливості відкликати його [29], і цей файл cookie розшифровує й сервер сигналізації.

Сервер сигналізації та пересилання медіапотоків побудовано на Fastify: плагін @fastify/websocket обслуговує маршрути WebSocket тими самими хуками Fastify, що й HTTP-маршрути, тому сеанс перевіряється однаково. Як сервер SFU обрано mediasoup – модуль Node.js, у якому медіадані пересилають процеси-обробники мовою C++, а сигнальний протокол застосунку не нав'язується [12]. Вбудований у процес сервера сигналізації mediasoup дозволяє керувати трансляцією та повторним підключенням стримера з одного коду, а його програмний інтерфейс маршрутизаторів, транспортів і споживачів дає змогу задати порядок кодеків і межі бітрейту, визначені нефункціональними вимогами [30]. Платою за це є власний сигналізаційний протокол (табл. 3.1) з клієнтом mediasoup-client.

Для настільного застосунку визначальними є вимоги ФВ-07–ФВ-09, які потребують доступу до вікон і процесів операційної системи, тому його побудовано на платформі Electron. Її модуль desktopCapturer перелічує екрани й вікна, а сесія дозволяє відповісти на запит захоплення сторінки обраним джерелом [13, 14]. Настільний застосунок відкриває розгорнутий вебзастосунок, тож вбудований рушій Chromium виконує той самий клієнтський код WebRTC, що й браузер, а Node.js в основному процесі дозволяє запустити бібліотеку electron-native-screenshare [5] в окремому утилітарному процесі, призначеному для компонентів, схильних до збоїв [28].

Для зберігання даних обрано реляційну СКБД PostgreSQL. Схеми Drizzle ORM є звичайними модулями TypeScript, тому їх зі спільного модуля доступу до бази даних імпортують і адаптер Auth.js, і сервер сигналізації.

Сервіси системи розгортаються за єдиним описом Docker Compose, а Caddy автоматично отримує сертифікати TLS, і вебзастосунок працює через HTTPS. Інсталятори збирає electron-builder, оновлення з випусків GitHub отримує electron-updater.

4.2 Реалізація захоплення окремих застосунків

Через ізоляцію контексту вебзастосунку в настільному застосунку доступний лише об'єкт `window.conveyor` сценарію `preload`. Методи класу `StreamApi` відповідають каналам трансляції основного процесу (рис. 3.10), для кожного з яких описано схеми `zod` аргументів і результату.

Перелік екранів і вікон із мініатюрами та піктограмами застосунків (ФВ-07) формує обробник каналу отримання джерел, а підміну системного вибору джерела – обробник запитів захоплення сеансу (лістинг 4.1).

Лістинг 4.1 – Перелік джерел, визначення PID вікна та відповідь на запит захоплення

```
function pidForSource(sourceId: string): number {
  if (!sourceId.startsWith('window:')) return 0;
  const hwnd = Number(sourceId.split(':')[1]);
  return Number.isFinite(hwnd) ? getPidFromWindowHandle(hwnd) : 0;
}
// ...
handle('stream:getSources', async () => {
  const [width, height] = mainWindow.getSize();
  const sources = await desktopCapturer.getSources({
    types: ['screen', 'window'],
    thumbnailSize: { width: width ?? 320, height: height ?? 180 },
    fetchWindowIcons: true,
  });
  // ...
});
// ...
session.defaultSession.setDisplayMediaRequestHandler(async (request, callback)
=> {
  // ...
  const sources = await desktopCapturer.getSources({ types: ['window', 'screen']
});
```

```
const chosen = sources.find((s) => s.id === selectedSourceId);
// ...
callback({ video: chosen ?? sources[0] });
});
```

Функція `pidForSource` виділяє з ідентифікатора джерела-вікна його дескриптор і визначає ідентифікатор процесу (PID) функцією `getPidFromWindowHandle` модуля `electron-native-screenshare` [5]; для екранів PID дорівнює 0. Обраний користувачем ідентифікатор джерела зберігається в основному процесі, а сторінка викликає стандартний метод `getDisplayMedia` [4] з бажаною роздільністю 3840×2160. Системне вікно вибору не відкривається: обробник, установлений методом `setDisplayMediaRequestHandler` [14], відповідає збереженим джерелом, а за його відсутності – першим у списку.

Захоплення звуку окремого застосунку потребує окремого механізму. Обробник запитів захоплення Electron повертає як звук лише значення `loopback` – звук усієї системи і лише у Windows [14], тому вимогу ФВ-08 реалізовано модулем `electron-native-screenshare` в утилітному процесі, призначеному, зокрема, для компонентів, схильних до аварійного завершення [28]. Обробник каналу запуску захоплення звуку створює цей процес і передає йому PID джерела та один кінець каналу повідомлень (лістинг 4.2).

Лістинг 4.2 – Запуск захоплення звуку процесу в утилітному процесі

```
process.parentPort.once('message', (e) => {
  const { processId } = e.data as AudioWorkerInit;
  const port = e.ports[0];
  // ...
  try {
    const startCaptureCallback: AudioDataCallback = (audioData, meta) => {
      const ab = audioData.buffer.slice(
        audioData.byteOffset,
        audioData.byteOffset + audioData.byteLength,
      );
      port.postMessage({ ab, meta });
    };
    const args: [number, boolean, AudioDataCallback] = processId
      ? [processId, true, startCaptureCallback]
      : [process.pid, false, startCaptureCallback];

    port.start();
    const started = startCapture(...args);
    if (!started) throw new Error('startCapture() returned false');
    report({ type: 'ready' });
  } catch (err) {
```

```

    report({ type: 'error', error: err instanceof Error ? err.message :
String(err) });
  }
});

```

Для відомого PID функція startCapture працює в режимі включення й захоплює звук лише цільового процесу, а для екрана – у режимі виключення самого утилітного процесу, тобто захоплює весь системний звук. Буфери РСМ з метаданими формату йдуть портом, другий кінець якого основний процес передає сторінці лише після повідомлення про готовність. На сторінці міст звуку створює вузол AudioWorklet, процесор якого виконується в окремому потоці Web Audio: він передискретизує відліки до частоти контексту, буферизує не більше 1 с звуку, а за нестачі даних видає тишу. Отримана звукова доріжка додається до захопленого відеопотоку; якщо будь-який крок не вдається, трансляція продовжується лише з відео з повідомленням користувачу (ФВ-08). [ПОТРЕБУЄ УТОЧНЕННЯ: операційні системи, на яких фактично перевірено захоплення звуку окремого застосунку.]

Режим слідування за вікнами застосунку (ФВ-09) реалізує клас SourceFollower за алгоритмом рис. 3.6. Після вибору вікна клас фіксує якір: PID, виконуваний файл, корінь сімейства процесів і час вибору. Коренем сімейства є найвищий предок, що не належить до оболонки (Провідник Windows, launchd, systemd тощо), а батько, створений більш ніж на 2 с пізніше за нащадка, не враховується, що захищає від повторного використання PID. Кожні 1500 мс клас перевіряє нові вікна (лістинг 4.3).

Лістинг 4.3 – Вибір вікна для слідування в класі SourceFollower

```

const candidates: (PidWindow & { createdAt: Date })[] = [];
for (const w of others) {
  const info = table.get(w.pid);
  if (!info) continue;
  if (!isInFamily(table, w.pid, anchor.familyRoot)) continue;
  if (info.createdAt <= anchor.startedAt) continue;

  const sameExe = info.name.toLowerCase() === anchor.exe.toLowerCase();
  if (sameExe && !anchorAlive) {
    // The anchor process was replaced (League's "close client during game").
    this.anchor = { ...anchor, sourceId: w.id, pid: w.pid };
    // ...
  }
}

```

```

        return;
    }
    if (sameExe) continue;
    candidates.push({ ...w, createdAt: info.createdAt });
}
if (candidates.length === 0) return;

candidates.sort((a, b) => b.createdAt.getTime() - a.createdAt.getTime());
const winner = candidates[0]!;
// ...
this.deps.setSelectedSourceId(winner.id);
// ...
this.deps.emit({ reason: 'follow', sourceId: winner.id, name: winner.name });

```

Обирається найновіше вікно процесу того самого сімейства, запущеного після вибору якоря, з іншим виконуваним файлом, а процес із тим самим файлом означає перезапуск застосунку, і якір переноситься на нього. Після закриття вікна, за яким слідували, клас повертається до якоря, а якщо той не з'являється за 20 с, надсилає подію `lost`, і трансляція зупиняється. На події `follow` і `return` сторінка знову викликає метод `getDisplayMedia`, запускає захоплення звуку нового процесу і замінює доріжки методом `replaceTrack`, що не потребує повторного узгодження сеансу [6], тому глядачі не перепідключаються.

Взаємодію описаних механізмів захоплення з публікацією на сервері показано на рис. 4.1.

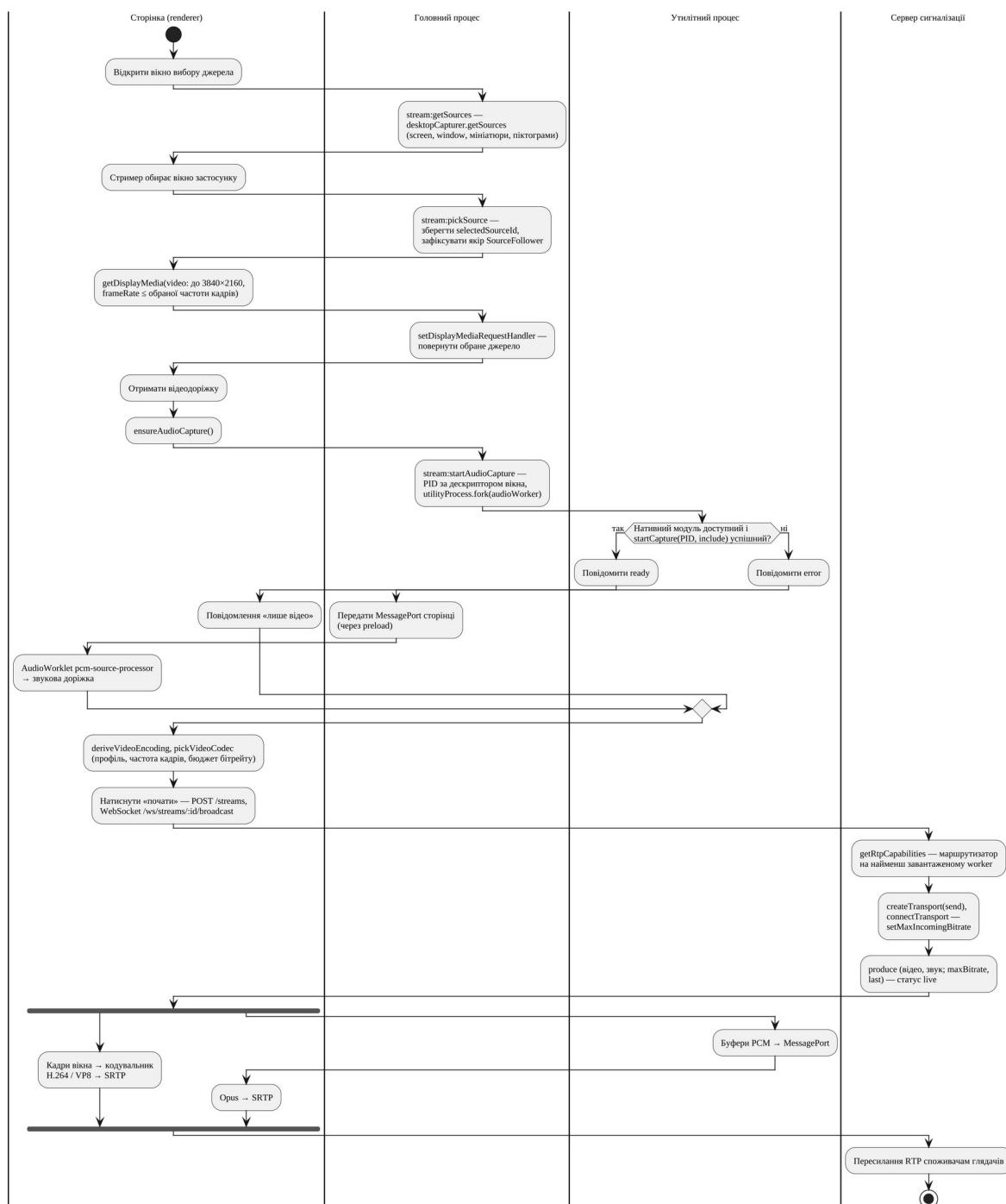


Рисунок 4.1 – Діаграма діяльності конвеєра захоплення й публікації трансляції

Відео й звук ідуть паралельними шляхами: кадри вікна кодує стек WebRTC Chromium, а відліки PCM надходять на сторінку через порт повідомлень і кодуються в Opus; розгалуження відповідає режиму «лише відео».

4.3 Реалізація потокової передачі на основі WebRTC та SFU

Сервер сигналізації запускає задану змінною оточення MEDIASOUP_NUM_WORKERS кількість процесів worker mediasoup (типово – за кількістю ядер процесора) – підпроцесів C++, що обробляють ICE, DTLS і RTP [12], з портами RTC 40000–49999. Кожна трансляція отримує власний маршрутизатор на найменш завантаженому worker; його обирає метод pickLeastLoadedWorker (лістинг 4.4).

Лістинг 4.4 – Вибір найменш завантаженого процесу worker

```
async pickLeastLoadedWorker(): Promise<Worker> {
  if (!this.workers.length) {
    throw new Error('Workers is empty');
  }

  const usages = await Promise.all(
    this.workers.map(async (w) => ({
      entry: w,
      usage: await w.getResourceUsage(),
    })),
  );
  usages.sort((a, b) => a.usage.ru_utime - b.usage.ru_utime);

  return usages[0]!.entry;
}
```

Мірою навантаження є час виконання в режимі користувача (поле ru_utime статистики ресурсів процесу) [30]; він накопичується від запуску процесу, тож вибір враховує сумарне, а не поточне навантаження.

Маршрутизатор кожної трансляції оголошує Opus з частотою 48 кГц і двома каналами, H.264 у двох профілях і VP8 (лістинг 4.5).

Лістинг 4.5 – Кодеки відео маршрутизатора mediasoup

```
const router = await worker.createRouter({
  mediaCodecs: [
    // ...
    {
      kind: 'video',
      mimeType: 'video/H264',
      clockRate: 90000,
      parameters: {
        'packetization-mode': 1,
      }
    }
  ]
});
```

```

        'profile-level-id': '42001f',
        'level-asymmetry-allowed': 1,
        'x-google-start-bitrate': START_BITRATE_KBPS,
    },
},
// ...
/** Software fallback for endpoints without H.264. */
{
    kind: 'video',
    mimeType: 'video/VP8',
    // ...

```

За коментарями в кодї VP8 є програмним резервним кодеком для клієнтів без H.264, а H.264 стоїть першим, бо лише його кодують апаратно всі цільові клієнти, що робить стійкими режими 1080p60, 1440p60 і 4K, а Baseline (профіль 42001f) передує Constrained Baseline (42e01f), бо останній у Chromium надає лише програмний OpenH264. Коментар посилається на перевірку розробником у Chrome 152 з NVENC; власних вимірювань у роботі не виконувалося. Параметр packetization-mode зі значенням 1 задає неперемежований режим пакетизації, параметр level-asymmetry-allowed дозволяє рівні, вищі за оголошений (1440p60 потребує рівня 5.1).

Параметри кодування відео обчислює функція deriveVideoEncoding (лістинг 4.6) за розмірами доріжки та обраними профілем і частотою кадрів (ФВ-10; типово 720p і 30 кадр/с).

Лістинг 4.6 – Обчислення параметрів кодування відео

```

export function deriveVideoEncoding(
    track: { width: number; height: number },
    settings: VideoSettings,
): DerivedVideoEncoding {
    const { quality } = settings;
    const fps = clampVideoSettings(settings, track.height).fps;
    const scaleResolutionDownBy = getScaleResolutionDownBy(track.height, quality);
    const targetWidth = track.width / scaleResolutionDownBy;
    const targetHeight = track.height / scaleResolutionDownBy;
    const maxBitrate = videoBitrateBudget(targetWidth * targetHeight, fps);
    const isHighFps = fps > HIGH_FPS_THRESHOLD;

    return {
        encoding: { scaleResolutionDownBy, maxFramerate: fps, maxBitrate },
        degradationPreference: isHighFps ? 'maintain-framerate' : 'maintain-
resolution',
        contentHint: isHighFps ? 'motion' : 'detail',
        frameRate: { ideal: fps, max: fps },
        startBitrateKbps: Math.round(startBitrateFor(maxBitrate) / 1000),
    };
}

```

```
};
}
```

Коефіцієнт зменшення роздільності (поле `scaleResolutionDownBy`) – відношення висоти доріжки до висоти профілю, не менше 1, тож зображення не збільшується; за вихідної висоти понад 1440 пікселів частоту обмежено 30 кадр/с. Бюджет бітрейту обчислює спільна функція `videoBitrateBudget` за формулою (4.1).

$$B = \min \left(\max \left(N \cdot 30 \cdot 0,07 \cdot \left(\frac{f}{30} \right)^{\log_2 1,5}; 3 \cdot 10^5 \right); 1,8 \cdot 10^7 \right), \quad (4.1)$$

де B – бюджет бітрейту, біт/с; N – кількість пікселів вихідного кадру; f – частота кадрів, кадр/с. Показник $\log_2 1,5$ відображає закладене в коді припущення, що подвоєння частоти кадрів потребує в 1,5 раза більшого бітрейту. Початковий бітрейт кодувальника становить половину бюджету. За 60 кадр/с обираються пріоритет деградації `maintain-framerate` і підказка вмісту `motion`, інакше – `maintain-resolution` і `detail`, тобто за нестачі пропускної здатності зберігається відповідно плавність або чіткість. Обчислені значення для джерела 3840×2160 наведено в табл. 4.2.

Таблиця 4.2 – Параметри кодування відео для джерела 3840×2160, обчислені за кодом

Профіль	Вихідна роздільність	Коефіцієнт зменшення	Максимальний бітрейт, Мбіт/с (5 / 30 / 60 кадр/с)	Початковий бітрейт, Мбіт/с (30 / 60 кадр/с)
360p	640×360	6	0,30 / 0,48 / 0,73	0,24 / 0,36
480p	853×480	4,5	0,30 / 0,86 / 1,29	0,43 / 0,65
720p	1280×720	3	0,68 / 1,94 / 2,90	0,97 / 1,45
1080p	1920×1080	2	1,53 / 4,35 / 6,53	2,18 / 3,27
1440p	2560×1440	1,5	2,71 / 7,74 / 11,61	3,87 / 5,81
Source	3840×2160	1	6,11 / 17,42 / –	8,71 / –

Значення є розрахунковими верхніми межами; фактичний бітрейт визначає оцінювання пропускної здатності WebRTC. Зміна профілю чи частоти під час трансляції не потребує повторного узгодження: метод `applyConstraints` змінює

частоту захоплення, а метод `setParameters` відправника RTP – параметри кодування [6].

На боці сервера бітрейт обмежується з обох напрямів. Після з'єднання транспорту стримера сервер обмежує вхідний бітрейт транспорту [30] типовим значенням 21,6 Мбіт/с – максимальний бюджет 18 Мбіт/с із запасом 1,2 на Opus, повторні передачі й заголовки. Транспорти глядачів створюються з початковою доступною вихідною швидкістю, що дорівнює половині бюджету відео, але не менше 4 Мбіт/с, тому для трансляції 4K оцінювання починається з 8,71 Мбіт/с замість типових для mediasoup 600 кбіт/с [30].

Методи та події сигналізації (табл. 3.1) передаються через WebSocket як JSON-конверти запиту, відповіді з ознакою успіху і події. Їхні типи виводяться зі спільного для клієнта й сервера опису протоколу, проте на відміну від IPC під час виконання схемами не перевіряються. Клієнтську частину реалізує клас `WsClient` (лістинг 4.7).

Лістинг 4.7 – Типізований запит клієнта сигналізації

```
async request<M extends WsResponse['method']>(
  msg: Extract<WsRequest, { method: M }>,
): Promise<Extract<WsResponse, { method: M }>> {
  if (this.openPromise) await this.openPromise;
  const requestId = this.nextId++;
  return new Promise((resolve, reject) => {
    this.pending.set(requestId, {
      resolve: resolve as (msg: WsResponse) => void,
      reject,
    });
    this.ws.send(constructRequest({ ...msg, requestId }));
  }) as Promise<Extract<WsResponse, { method: M }>>;
}
```

Відповідь знаходить свою обіцянку за ідентифікатором запиту, а закриття з'єднання відхиляє всі очікувані запити. Послідовність публікації (рис. 3.3) хук `useStreamer` реалізує засобами `mediasoup-client`, передаючи в запиті `produce` бюджет бітрейту.

На боці глядача хук `useViewer` реалізує послідовність рис. 3.4: після запиту `joinStream` створюються приймальний транспорт і споживач для кожного продюсера, причому сервер перевіряє, чи може маршрутизатор передати потік із

можливостями клієнта [30]. Якщо автовідтворення заблоковано, показується кнопка запуску. Подія `streamerReconnected` (рис. 3.7) замінює споживачів без перезавантаження сторінки. Мініатюру трансляції сторінка стримера знімає одразу і далі кожні 5 хв як JPEG з якістю 0,7.

Обмеженням реалізації є єдине кодування без `simulcast` і `SVC`: `simulcast` у `mediasoup-client` потребує кількох записів кодування, тому всі глядачі отримують однакову якість.

4.4 Реалізація автентифікації та авторизації

Автентифікацію у вебзастосунку реалізовано трьома провайдерами `Auth.js`. Провайдер Google (ФВ-01) передає параметр `prompt` зі значенням `select_account`, тож вибір облікового запису пропонується під час кожного входу. Два провайдери типу `Credentials`, у яких рішення ухвалює власна функція `authorize`, обслуговують гостьовий сеанс (провайдер `guest`, ФВ-03) і завершення входу з настільного застосунку (провайдер `desktop`, ФВ-02). Параметри сеансу наведено в лістингу 4.8.

Лістинг 4.8 – Стратегія сеансу та параметри файлу `cookie` в конфігурації `Auth.js`

```
const config: NextAuthConfig = {
  adapter,
  session: {
    strategy: 'jwt',
  },
  useSecureCookies: useSecureAuthCookies,
  cookies: {
    sessionToken: {
      name: SESSION_COOKIE_NAME,
      options: {
        httpOnly: true,

        sameSite: 'lax',
        path: '/',
        secure: useSecureAuthCookies,
      },
    },
  },
  providers: [
    // ...
  ],
}
```

Сеанс зберігається в зашифрованому токени у файлі cookie [29]. Атрибут `HttpOnly` закриває cookie від сценаріїв сторінки, `SameSite=Lax` обмежує його надсилання з інших сайтів, а `Secure` дозволяє передавання лише через HTTPS. Ознака `useSecureAuthCookies` зі спільних констант істинна лише у виробничому середовищі, і тоді ім'я cookie отримує префікс `__Secure-`, з яким браузер приймає його тільки від захищеної сторінки. Гостьовий провайдер створює запис із роллю `guest` і випадковим іменем, ідентифікатор якого сторінка перегляду зберігає в локальному сховищі браузера для повторного використання.

Обмеження доступу до сторінок (ФВ-04) виконує проміжний обробник `Next.js (Proxy)`. Він не вважає гостьовий сеанс входом і перенаправляє такого відвідувача на сторінку входу з усіх сторінок, крім головної, сторінки входу та сторінки повернення до настільного застосунку. Шаблон шляхів обробника виключає з перевірки API-маршрути, статичні файли й сторінки перегляду трансляцій, тому перегляд доступний без облікового запису Google.

Сервер сигналізації самостійно розшифровує той самий cookie (лістинг 4.9). Ключ розшифрування бібліотека `Auth.js` виводить із секрету та «солі» – імені cookie, тому сервер бере це ім'я зі спільних констант, а значення змінної оточення `AUTH_SECRET` обох сервісів збігаються.

Лістинг 4.9 – Перевірка сеансового cookie на сервері сигналізації

```
export const NEXT_AUTH_SALT = SESSION_COOKIE_NAME;

export async function decodeAuthToken(rawToken: string): Promise<JWT | null> {
  const payload = await decode({
    salt: NEXT_AUTH_SALT,
    secret: env.AUTH_SECRET,
    token: rawToken,
  });
  if (!payload?.sub) return null;
  return payload;
}
// ...
export const validateWsJwt = async (
  req: FastifyRequest, reply: FastifyReply): Promise<void> => {
  const rawToken = req.cookies[NEXT_AUTH_SALT];
  // ...
  const payload = await decodeAuthToken(rawToken);

  if (!payload) {
    reply.code(401).send({ error: 'Invalid token' });
    return;
  }
}
```

```

    }
    req.context.userId = payload.sub;
  };

```

Функція `validateWsJwt` підключена як хук попередньої перевірки до групи REST-маршрутів трансляцій і обох маршрутів `WebSocket`; відповідь 401 з хука зупиняє обробку запиту. Належність трансляції перевіряє лише маршрут завантаження мініатюри (відповідь 403), а сокет публікації покладається на непередбачуваність `UUID` трансляції. Приватну трансляцію (ФВ-11) маршрут пошуку виключає з переліку примусовим фільтром за полем `isPrivate`, а переглянути її може кожен, хто має посилання.

Вхід у настільному застосунку спирається на послідовність, наведену на рис. 3.5, а її початок у головному процесі `Electron` – у лістингу 4.10.

Лістинг 4.10 – Параметри РКСЕ і запуск входу в системному браузері

```

const verifier = base64url(randomBytes(32));
const challenge = createHash('sha256').update(verifier).digest('base64url');
const state = base64url(randomBytes(16));
// ...
const port = await listen(server);
// ...
const url = new URL('/api/desktop-auth/start', __WEB_URL__);
url.searchParams.set('port', String(port));
url.searchParams.set('state', state);
url.searchParams.set('challenge', challenge);

await shell.openExternal(url.toString());

```

Локальний сервер слухає адресу 127.0.0.1 на порту, призначеному операційною системою [24], і відхиляє запити з чужим параметром `state`. Вебзастосунок будує адресу повернення за фіксованим шаблоном локальної адреси, що виключає відкрите перенаправлення, а запит і одноразовий код шифруються з різними «солями», тож один тип токена не можна підставити замість іншого. Провайдер `desktop` порівнює SHA-256 від перевірного рядка (`verifier`) з викликом (`challenge`) коду функцією сталого часу `timingSafeEqual` [25] і відхиляє повторно використаний ідентифікатор коду. Реєстр використаних

ідентифікаторів зберігається в пам'яті процесу, тому за кількох екземплярів вебзастосунок одноразовість не гарантована.

Вхід за паролем, обмеження спроб входу та журнал аудиту не реалізовано: стовпці password_hash, failed_login_count, locked_until і таблиця audit_log лише зарезервовані у схемі.

4.5 Реалізація рівня даних і програмного інтерфейсу сервера

Таблиці бази даних (рис. 3.8) описано в модулі доступу до бази даних засобами Drizzle ORM разом зі зв'язками трансляції зі стримером і глядачами. Типи рядків виводяться зі схем, тому вебзастосунок і сервер сигналізації використовують одні типи без генерації коду. Окремих міграцій немає: контейнер migrate виконує команду push утиліти drizzle-kit у примусовому режимі, яка застосовує відмінності схеми безпосередньо до бази даних.

Репозиторії сервера сигналізації інкапсулюють запити Drizzle, а контролери перетворюють помилки на коди 404, 400 чи 500 (рис. 3.9). Маршрути наведено в табл. 4.3; ззовні вони доступні через Caddy з префіксом /sfu.

Таблиця 4.3 – Маршрути програмного інтерфейсу сервера сигналізації

Метод і шлях	Сеанс	Призначення
POST /streams	так	створення трансляції (ознака приватності)
POST /streams/search	так	сторінка публічних трансляцій з фільтрами й сортуванням
GET /streams/:userId/active	так	активна трансляція користувача
GET, POST /streams/:streamId/thumbnail	так (POST – власник)	отримання та збереження мініатюри
GET /streams/:streamId	ні	трансляція зі стримером і глядачами
GET /users/:userId	ні	відомості про користувача
GET /ws/streams/:streamId/broadcast	так	WebSocket стримера

Метод і шлях	Сеанс	Призначення
GET /ws/streams/:streamId/watch	так, зокрема гостьовий	WebSocket глядача

Метод `registerCronJobs` класу `UserService` реєструє в плагіні планувальника `Fastify` щотижнєве завдання (понеділок, 00:00), яке видаляє гостей без активності понад 7 діб, що не дивляться жодної трансляції (ФВ-19).

Модуль перевірки змінних оточення перевіряє їх під час запуску схемою `zod` і в разі помилок зупиняє запуск із переліком усіх некоректних змінних; зокрема, змінна оточення `AUTH_SECRET` має містити не менше 32 символів, а у виробничому середовищі змінна оточення `AUTH_URL` має бути адресою `HTTPS`.

4.6 Реалізація інтерфейсу користувача та розгортання

Основні сторінки вебзастосунку мають спільні шапку й підвал, а серверні компоненти отримують дані з сервера сигналізації через внутрішню мережу `Docker` відповідно до архітектури системи. Головна сторінка відвідувачу показує кнопку завантаження настільного застосунку для операційної системи із заголовка `User-Agent` (ФВ-18), а користувачу – по 12 карток активних трансляцій на сторінці (ФВ-05).

Кольори світлої й темної тем задано змінними `CSS`, які директива `theme` перетворює на змінні теми `Tailwind CSS`. Компонент `ThemeProvider` бібліотеки `next-themes` перемикає тему класом кореневого елемента й типово обирає її за налаштуванням операційної системи; окремого перемикача теми немає. Головну сторінку в обох темах наведено на рис. 4.2.

[ПОТРЕБУЄ УТОЧНЕННЯ: РИСУНОК: скріншот головної сторінки зі списком активних трансляцій у світлій і темній темах – замінити заглушку рисунка]

[РИСУНОК: скріншот]

Головна сторінка вебзастосунку для користувача, що увійшов, зі списком активних трансляцій (картки з мініатюрами, перемикач сторінок):
а - світла тема операційної системи; б - темна тема (два знімки поруч)

Заглушка: замінити файл thesis/screenshots/scr-home-themes.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 4.2 – Головна сторінка вебзастосунку: а – світла тема; б – темна тема

Теми різняться лише значеннями змінних кольорів, тому компонент, що використовує кольори теми, підтримує обидві без додаткового коду.

Збирання та розгортання автоматизовано засобами GitHub Actions. За тегом версії сервісу робочий процес збирає образи web, signaling і migrate у GitHub Container Registry та оновлює їх на сервері через SSH засобами Docker Compose згідно з діаграмою розгортання (рис. 3.2), а за тегом версії настільного застосунку – інсталятори NSIS, DMG і AppImage засобами electron-builder для випуску GitHub. Автоматичне оновлення, як і передбачено нефункціональними вимогами, працює лише у Windows і Linux.

Висновки до розділу 4

Систему реалізовано мовою TypeScript зі спільними модулями. Захоплення окремого застосунку з його звуком і слідування за його вікнами виконує настільний застосунок Electron, а потокову передачу – SFU mediasoup з H.264 як основним кодеком.

Автентифікація поєднує вхід через Google, гостьові сеанси та вхід настільного застосунку через системний браузер за схемою PKCE, а зашифрований сеанс JWT у cookie перевіряє й сервер сигналізації. Обмеженнями є відсутність перевірки власника для сокета публікації та неможливість відкликати сеанс до завершення строку дії.

5 ТЕСТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

5.1 Принципи вибору тестових прикладів

Об'єктом тестування є програмна система у складі вебзастосунку, сервера сигналізації й пересилання медіапотоків, настільного застосунку і спільних модулів у версії від 25.09.2026 з основної гілки репозиторію. Репозиторій продукту не містить автоматизованих модульних, інтеграційних чи наскрізних тестів, а конвеєри безперервної інтеграції лише збирають інсталятори настільного застосунку та образи контейнерів. Тому перевірку проведено на трьох рівнях: статична перевірка вихідного коду, функціональні перевірки серверних компонентів і логіки настільного застосунку сценаріями, підготовленими для цього розділу, та ручні перевірки сценаріїв, які потребують реального захоплення екрана.

Тестові приклади виведено з функціональних вимог ФВ-01...ФВ-19 (табл. 1.3) і сценаріїв використання 1.1 – 1.6. Для кожного сценарію обрано позитивний випадок основного потоку та негативні випадки, що відповідають альтернативним потокам і межам доступу. Для логіки з часовими параметрами (повторні спроби відновлення, інтервал опитування вікон) перевірено фактичні інтервали спрацювання.

Автоматизовані перевірки виконано 05.10.2026 на стенді з Debian 13 (aarch64), Node.js 24.21.0 і PostgreSQL 18.4; використано mediasoup 3.20.5 і Next.js 16.2.6 із файлу блокування залежностей. Схему бази даних створено засобами Drizzle Kit, сервери запущено у виробничому режимі. Сценарій перевірки серверних компонентів звертається до запущених серверів через HTTP і WebSocket тими самими повідомленнями протоколу, що й клієнтський код. Сеанс користувача Google, для якого потрібен зовнішній постачальник, сформовано функцією encode бібліотеки Auth.js з тим самим секретом і сіллю, якими користується система [29], а гостьовий сеанс отримано через справжню кінцеву точку входу. Сценарій перевірки режиму слідування збирає клас SourceFollower

настільного застосунку (лістинг А.1) без Electron і подає йому змодельований перелік вікон і таблицю процесів. Обидва сценарії не захоплюють, не кодують і не відтворюють медіадані, тому відповідні перевірки віднесено до ручних.

Результати статичної перевірки наведено в табл. 5.1.

Таблиця 5.1 – Результати статичної перевірки та збирання

Перевірка	Команда	Фактичний результат
Встановлення залежностей	<code>pnpm install --frozen-lockfile</code>	успішно, 2 хв 12 с
Статичний аналіз ESLint	<code>pnpm lint</code>	6 із 6 пакетів без помилок, 1 хв 22 с
Перевірка типів TypeScript	<code>pnpm typecheck</code>	6 із 6 пакетів без помилок, 35 с
Збирання спільних модулів і сервера сигналізації	<code>pnpm -r build (tsup)</code>	успішно
Збирання вебзастосунку без змінних оточення	<code>next build</code>	зупинка з переліком відсутніх змінних DATABASE_URL, AUTH_SECRET, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET
Збирання вебзастосунку, як в образі контейнера	<code>SKIP_ENV_VALIDATION=1 next build</code>	успішно: компіляція 70 с, перевірка типів 27,6 с, 10 статичних сторінок
Збирання настільного застосунку	<code>electron-vite build</code>	успішно за 3,4 с: головний процес 33,6 кБ, скрипт попереднього завантаження 3,4 кБ, процес захоплення звуку 1,2 кБ
Пакування інсталяторів	<code>electron-builder</code>	не виконувалося: пакування виконує CI під час публікації тегу

Перевірка типів виконується в суворому режимі TypeScript, а конфігурація ESLint забороняє необроблені проміси і пряме звертання до змінних оточення поза модулями конфігурації. Зупинка збирання без змінних оточення є очікуваною: образ контейнера вимикає цю перевірку на етапі збирання й виконує її під час запуску.

5.2 Функціональне тестування

Тестові приклади згруповано за сценаріями використання: автентифікація (А), трансляція з браузера (Б), настільний застосунок (Д), перегляд (П), завершення та відновлення трансляції (З). Автоматизовані приклади виконано 05.10.2026; основні з них, зокрема всі, що виявили невідповідності, наведено в табл. 5.2.

Таблиця 5.2 – Тестові приклади функціонального тестування

ID	Сценарій	Вхідні дані	Очікуваний результат	Фактичний результат
A7	REST без сеансу та з підробленим токеном	POST /streams; без cookie, потім з cookie forged.token	Статус: 401 в обох випадках	Не пройдено: без cookie – 401; з підробленим токеном – 500
Б5	Гість створює трансляцію	POST /streams; cookie гостя	Відмова	Не пройдено: 201, трансляцію створено
Д3	Нове вікно застосунку	У процесу застосунку з'являється дочірній процес із вікном	Перемикання на нове вікно	Пройдено: перемикання через 1,5 с
Д5	Закриття нового вікна	Вікно й процес із Д3 зникають	Повернення до вихідного вікна	Пройдено: повернення через 1,3 с
Д7	Вихідне вікно не повернулося	Закрито і вихідне, і нове вікно	Подія lost після очікування	Пройдено: подія lost через 22,3 с
32	Стример не повернувся	Очікування після розриву WebSocket стримера	Завершення після 5 спроб по 6 с, причина timeout	Пройдено: завершено через 36,0–36,1 с, endReason = timeout
34	Сторонній користувач	Гість відкриває сокет трансляції іншого користувача і надсилає endStream	Відмова	Не пройдено: сокет відкрито, трансляцію завершено

Приклади автентифікації перевіряють вимоги ФВ-01 – ФВ-04 і перевірку сеансу сервером сигналізації. Обмеження доступу до сторінок (ФВ-04) і гостьовий вхід (ФВ-03) відповідають вимогам, cookie сеансу має атрибути, визначені вимогами до безпеки. Приклад А7 виявив дефект: перевірка токена сервером сигналізації не перехоплює виняток JWInvalid, тому довільне значення cookie дає відповідь 500 замість 401, хоча доступу до даних не надає. Сторінка політики

конфіденційності, на яку посилається сторінка входу, для анонімного відвідувача переспрямовує на вхід, що відповідає ФВ-04, але робить правові сторінки недоступними до входу.

Приклади трансляції з браузера охоплюють створення трансляції та медіаз'єднання стримера (ФВ-06, ФВ-10 – ФВ-13, ФВ-17). Сигнальна частина потоку 1.4 виконується повністю: маршрутизатор пропонує кодеки в порядку лістингу 4.5, публікація останнього потоку переводить трансляцію в стан live, мініатюру змінює лише власник. Приклад Б5 показує, що заборону трансляцій для гостей реалізовано лише маршрутами вебзастосунку: кінцева точка створення трансляції не перевіряє роль сеансу.

Перевірки настільного застосунку стосуються режиму слідування (ФВ-09) і вибору джерела (ФВ-07, ФВ-08) на змодельованій таблиці процесів. Інтервали в Д3, Д5 і Д7 узгоджуються з параметрами модуля: опитування раз на 1,5 с і очікування вихідного вікна 20 с плюс цикл опитування. Ці перевірки підтверджують логіку вибору вікна, але не саме захоплення, тому перелік джерел, звук застосунку і слідування в реальному застосунку перевіряються вручну у Windows.

Приклади перегляду охоплюють перелік трансляцій, приватні трансляції та підключення глядача (ФВ-05, ФВ-11, ФВ-14). Сервер приховує приватну трансляцію з переліку й водночас надає її за ідентифікатором, а гість отримує медіапотік без входу через Google; відтворення в браузері перевіряється вручну.

Приклади завершення трансляції перевіряють вимоги ФВ-15 і ФВ-16 та право керувати трансляцією. Тривалість у 32 складається з п'яти інтервалів по 6 с і ще одного, на якому таймер фіксує вичерпання спроб, тобто 36 с, а не 30 с. Приклад 34 виявив найсерйозніший дефект: обробник WebSocket стримера перевіряє наявність сеансу, але не належність трансляції, тому будь-який сеанс може завершити чужу трансляцію.

5.3 Нефункціональне тестування

Методи перевірки нефункціональних вимог до продуктивності, сумісності та безпеки і їх результати наведено в табл. 5.3. Величини продуктивності залежать від обладнання й мережі і не можуть бути отримані на стенді без графічного середовища, тому їх позначено як дані, що має надати автор.

Таблиця 5.3 – Результати нефункціонального тестування

Характеристика	Метод перевірки	Очікуваний результат	Фактичний результат
Затримка доставки	екранний секундомір у вікні, що транслюється; знімок екрана з вікнами стримера й глядача; 10 вимірювань	[ПОТРЕБУЄ УТОЧНЕННЯ: Я: цільове значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: середнє та максимальне значення, мережа]
Бітрейт і частота кадрів	сторінка chrome://webrtc-internals , вихідний потік: поля targetBitrate, framesPerSecond, frameWidth, qualityLimitationReason	для 1080p60 – бюджет 6,53 Мбіт/с, старт 3,27 Мбіт/с (табл. 4.2)	[ПОТРЕБУЄ УТОЧНЕННЯ: значення для 720p30, 1080p60, 1440p60, вихідної]
Завантаження клієнта	диспетчер завдань Windows, процеси настільного застосунку під час трансляції 1080p60	[ПОТРЕБУЄ УТОЧНЕННЯ: Я: допустиме значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: CPU, GPU, модель процесора й відеокarti, кодер encoderImplementation]
Кількість глядачів	N вкладок або пристроїв на одній трансляції; навантаження процесу сервера (top)	відтворення без зупинок	[ПОТРЕБУЄ УТОЧНЕННЯ: N, CPU сервера, freezeCount у глядачів]
Узгодженість лімітів бітрейту	змінна оточення MEDIASOUP_MAX_INCOMING_BITRATE = 10 000 000	попередження про обмеження	попередження з порогом 21 600 000 біт/с
Сумісність	Windows, macOS, Linux × Chrome, Edge, Firefox, Safari: трансляція й перегляд	робота в підтримуваних поєднаннях	[ПОТРЕБУЄ УТОЧНЕННЯ: матриця результатів з версіями]
Обмеження доступу	запити закритих сторінок і WebSocket без сеансу та із сеансом гостя	закриті маршрути недоступні без входу	відповідає
Захист сеансу	гостьовий вхід, заголовок Set-Cookie	атрибути HttpOnly,	відповідає

Характеристика	Метод перевірки	Очікуваний результат	Фактичний результат
		Secure, SameSite, префікс __Secure-	
Авторизація сервера сигналізації	приклади А7, Б5, 34	відмова сторонньому чи недійсному сеансу	не відповідає: 500, 201, завершення чужої трансляції

У статистиці потоку WebRTC поле `qualityLimitationReason` – причина зниження якості, поле `freezeCount` – кількість зупинок відтворення в глядача. Значення слід записувати після 60 с трансляції динамічного вмісту, оскільки кодер починає з половини бюджету бітрейту; затримку – як різницю показів секундоміра на одному знімку з вікнами стримера й глядача.

5.4 Виправлені дефекти та аналіз результатів

Частину дефектів виявлено й виправлено до цього тестування; їхній перелік наведено в табл. 5.4 разом із дефектами, виявленими під час функціонального та нефункціонального тестування.

Таблиця 5.4 – Виявлені дефекти та їх стан

Дефект	Причина	Виправлення	Стан
Блочність зображення 1080p60 у динамічних сценах	транспорт стартував з вихідного бітрейту 1 Мбіт/с; ліміт сервера 10 Мбіт/с був нижчим за стелю відправника, тож найвищі профілі мовчки обмежувалися	спільний модуль розрахунку бітрейту: стеля 18 Мбіт/с, ліміт сервера 1,2 × стеля, старт – половина бюджету, підказка типу вмісту motion	виправлено 20.09.2026
Невідповідність гідратації React	ОС відвідувача і фокус вікна обчислювалися в браузері під час	ОС визначається на сервері із заголовка User-Agent; фокус – через підписку	виправлено 19.09.2026

Дефект	Причина	Виправлення	Стан
	рендерингу	Реаг на зовнішнє сховище; придушення попередження про гідратацію лише для елемента body	
Кожен другий вхід у настільному застосунку зависав	ядро Auth.js оновлює cookie адреси повернення лише за зміни значення, а виклики виходу та входу передавали різні адреси	однакова адреса повернення в обох викликах, обробка відмови, сторожовий таймер, відсутність витoku обробника	виправлено 19.09.2026
Низький контраст світлої теми	токени поверхонь світлої теми мали темні значення; стиль елемента body посилався на неіснуючу CSS-змінну	світла палітра, наявний токен кольору для тексту, колір небезпеки темної теми 3,6:1 → рівень AA	виправлено 19.09.2026
Відсутній аватар	зображення аватара за замовчуванням не існувало, оптимізатор зображень повертав 400	компонент аватара: ініціали або піктограма, заміна при помилці завантаження	виправлено 19.09.2026
Помилки типів після чистого встановлення	збій перезбирання нативного модуля без заголовків PipeWire/X11 переривав встановлення залежностей	перезбирання обов'язкове лише в CI, явний тип конфігурації, збирання залежностей перед збиранням вебзастосунку	виправлено 19.09.2026
Підроблений токен → 500	неперехоплений виняток JWInvalid	перехоплення з відповіддю 401	відкрито (A7)
Гість створює трансляцію	REST-ендпоінт створення не перевіряє роль	відмова для ролі guest	відкрито (B5)
Керування чужою трансляцією	WebSocket стримера не перевіряє власника	перевірка власника під час підключення	відкрито (34)

Виправлення входу в настільному застосунку перевірялося під час розроблення допоміжним стендом, що до репозиторію не входить, тому передбачено повторну ручну перевірку. Три відкриті дефекти мають спільну причину: сервер сигналізації автентифікує сеанс, але не авторизує дії, покладаючись на обмеження маршрутів вебзастосунку, які не діють при прямих зверненнях до його API. Крім того, під час зупинки стенда процес сервера сигналізації не завершився протягом 10 с після сигналу SIGTERM, а обробників сигналів у його коді немає; наслідки для перезапуску контейнера пом'якшує закриття незавершених трансляцій під час запуску сервера.

Зі 30 автоматизованих прикладів 27 дали очікуваний результат: 20 із 23 у сценарії перевірки серверних компонентів і 7 із 7 у сценарії перевірки режиму слідування. Ручні перевірки входу через Google, захоплення, відтворення й відновлення трансляції та вимірювання табл. 5.3 потребують реального обладнання та є умовою остаточної оцінки ефективності функціонування й сумісності.

Висновки до розділу 5

Перевірку побудовано на статичному аналізі та збиранні, функціональних сценаріях проти запущених серверів і змодельованого оточення настільного застосунку та ручних прикладах. Статична перевірка і збирання всіх пакетів пройшли успішно, а автоматизовані приклади підтвердили обмеження доступу, гостьовий вхід, захищені атрибути cookie сеансу, публікацію трансляції, приватні трансляції, підключення глядача, завершення трансляції після втрати з'єднання за 36 с і логіку режиму слідування.

Тестування виявило три дефекти авторизації сервера сигналізації (табл. 5.4), які слід усунути перевітками ролі та власника на сервері. Шість дефектів, виявлених під час розроблення, виправлено. Результати ручних прикладів і вимірювань продуктивності має надати автор роботи.

6 ЕКСПЛУАТАЦІЯ ПРОГРАМНОЇ СИСТЕМИ

6.1 Контрольний приклад застосування програмної системи

Контрольний приклад відтворює сценарії 1.5, 1.3 та 1.6 на розгорнутому сервісі за участю стримера у Windows і незареєстрованого глядача на іншому пристрої [ПОТРЕБУЄ УТОЧНЕННЯ: адреса сервісу, версії ОС, браузера і застосунку].

Відвідувач, який не увійшов, бачить на головній сторінці кнопку «Download for» з логотипом своєї ОС, що завантажує інсталятор останнього випуску GitHub (рис. 6.1).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-download.png]

[РИСУНОК: скріншот]

Головна сторінка у браузері для відвідувача, який не увійшов: кнопка «Download for» з логотипом ОС і кнопка «Get started»

Заглушка: замінити файл thesis/screenshots/scr-download.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 6.1 – Головна сторінка для відвідувача з кнопкою завантаження настільного застосунку

Інсталятор NSIS встановлює застосунок для поточного користувача з вибором каталогу. Вікно застосунку відкриває той самий вебінтерфейс без кнопки завантаження, а меню в системному лотку містить пункти «Open», «Check for updates...» і «Quit».

У настільному застосунку кнопка «Log in» відкриває сторінку входу, а «Continue with Google» – вхід у системному браузері; застосунок показує стан очікування з кнопкою «Cancel» (рис. 6.2).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-desktop-sign-in.png]

[РИСУНОК: скріншот]

а - вікно настільного застосунку «Waiting for browser sign-in...» з кнопкою «Cancel»;
б - вкладка системного браузера «Signed in. You can close this tab...» (два знімки поруч)

*Заглушка: замінити файл thesis/screenshots/scr-desktop-sign-in.png
реальним скріншотом (PNG, ширина не менше 1600 px)*

Рисунок 6.2 – Вхід у настільному застосунку: а – очікування входу в браузері; б – сторінка завершення входу в браузері

Після вибору облікового запису вкладка браузера пропонує повернутися до застосунку, а він обмінює одноразовий код на сеанс і відкриває перелік трансляцій.

Кнопка «Start stream» у шапці відкриває сторінку трансляції, а «Pick source» – вікно вибору джерела з вкладками «Applications» і «Entire Screen» (рис. 6.3).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-source-picker.png]

[РИСУНОК: скріншот]

Вікно вибору джерела в настільному застосунку, вкладка «Applications»: мініатюри вікон з піктограмами застосунків

*Заглушка: замінити файл thesis/screenshots/scr-source-picker.png
реальним скріншотом (PNG, ширина не менше 1600 px)*

Рисунок 6.3 – Вікно вибору джерела трансляції в настільному застосунку

Вікна показано мініатюрами з піктограмами застосунків. Після вибору вікна починається попередній перегляд зі станом «PREVIEW» і захоплення звуку лише цього застосунку; якщо звук недоступний, з'являється повідомлення про передавання тільки відео. Режим слідування увімкнено за замовчуванням.

Для запуску приватної трансляції стример обирає якість і частоту кадрів (типово 720p і 30 кадр/с), вмикає приватність кнопкою із замком і натискає «Start stream». Стан змінюється на «LIVE», а поле «Stream link» показує посилання на сторінку перегляду трансляції з кнопкою «Сору» (рис. 6.4).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-broadcast-live.png]

[РИСУНОК: скріншот]

Сторінка трансляції в настільному застосунку: стан «LIVE», увімкнений замок (приватна), кнопка режиму слідування, поле «Stream link» з посиланням

Заглушка: замінити файл thesis/screenshots/scr-broadcast-live.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 6.4 – Сторінка активної приватної трансляції в настільному застосунку

Приватна трансляція не потрапляє до переліку на головній сторінці, а змінити приватність після запуску не можна.

Глядач відкриває посилання без входу, система створює гостьовий сеанс із випадковим іменем і відтворює трансляцію (рис. 6.5, а).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-watch-guest.png]

[РИСУНОК: скріншот]

а - сторінка перегляду в браузері глядача-гостя з елементами керування;
б - сторінка «The stream has ended» (два знімки поруч)

Заглушка: замінити файл thesis/screenshots/scr-watch-guest.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 6.5 – Сторінка перегляду в браузері глядача: а – під час трансляції; б – після завершення трансляції

Глядач керує гучністю, режимом «картинка в картинці» і повноекранним режимом кнопками або клавішами M, P і F.

Коли застосунок, що транслюється, відкриває нове вікно (наприклад, програма запуску – вікно гри), джерело перемикається без дій стримера з повідомленням «Switched to ...» і написом «Following: ...» (рис. 6.6).

[ПОТРЕБУЄ УТОЧНЕННЯ: скріншот screenshots/scr-follow.png]

[РИСУНОК: скріншот]

Сторінка трансляції з повідомленням «Switched to ...» і написом «Following: ...»; поруч - вікно глядача з новим вікном застосунку

Заглушка: замінити файл thesis/screenshots/scr-follow.png реальним скріншотом (PNG, ширина не менше 1600 px)

Рисунок 6.6 – Перемикання трансляції на нове вікно застосунку в режимі слідування

Глядач бачить нове вікно без перепідключення. Після його закриття трансляція повертається до початкового вікна («Back to ...»), а якщо зникло й воно, завершується з повідомленням «The captured app closed».

Кнопка «Stop stream» завершує трансляцію, а глядач бачить сторінку «The stream has ended» з кнопкою повернення до переліку (рис. 6.5, б).

6.2 Експлуатаційні випробування програмної системи

Критерієм досягнення мети є автоматичне, без дій стримера, перемикання трансляції на нове вікно застосунку протягом 1,5 с. Його підтверджують тестові приклади ДЗ, у якому трансляція перемкнулася на нове вікно через 1,5 с, і Д5, у якому повернення до вихідного вікна відбулося через 1,3 с (табл. 5.2), а також ручна перевірка слідування в реальному застосунку після її виконання.

Затримку доставки зображення глядачеві оцінюють як експлуатаційну характеристику, а не як критерій досягнення мети. Для цього у вікні застосунку працює екранний секундомір, а різницю його показів у вікнах стримера й глядача

фіксують на одному знімку 10 разів для кожних умов. Результати випробувань наведено в табл. 6.1.

Таблиця 6.1 – Результати експлуатаційних випробувань

№	Умови	Затримка, мс (сер. / макс.)	Відповідність меті
1	[ПОТРЕБУЄ УТОЧНЕННЯ: мережа, якість, кількість глядачів]	[ПОТРЕБУЄ УТОЧНЕННЯ: значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: так / ні]
2	[ПОТРЕБУЄ УТОЧНЕННЯ: мережа, якість, кількість глядачів]	[ПОТРЕБУЄ УТОЧНЕННЯ: значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: так / ні]
3	[ПОТРЕБУЄ УТОЧНЕННЯ: мережа, якість, кількість глядачів]	[ПОТРЕБУЄ УТОЧНЕННЯ: значення]	[ПОТРЕБУЄ УТОЧНЕННЯ: так / ні]

[ПОТРЕБУЄ УТОЧНЕННЯ: висновок за табл. 6.1; факти використання сервісу, лише якщо вони є]

Висновки до розділу 6

Контрольний приклад охоплює повний шлях користувача від встановлення настільного застосунку і входу через системний браузер до приватної трансляції вікна застосунку, перегляду гостем, автоматичного перемикавання на нове вікно та завершення трансляції. Усі кроки виконуються засобами інтерфейсу.

Мети досягнуто: трансляція без дій стримера перемикається на нове вікно застосунку за 1,5 с і повертається до вихідного вікна за 1,3 с, а затримку доставки визначено як експлуатаційну характеристику з окремою методикою вимірювання.

ЗАГАЛЬНІ ВИСНОВКИ

У кваліфікаційній роботі розроблено вебсервіс stream-share для потокової передачі медіаконтенту з функцією захоплення окремих застосунків. Система складається з вебзастосунку, сервера сигналізації з медіасервером SFU та настільного застосунку для захоплення і розгорнута в мережі Інтернет. Усі поставлені задачі розв'язано.

Аналіз предметної області показав, що для інтерактивної трансляції від одного стримера багатьом глядачам доцільно використовувати WebRTC у топології SFU, а вебсторінка не може програмно обрати вікно, отримати звук лише одного застосунку чи перейти до його нового вікна. Порівняння чотирьох найвідоміших наявних рішень не виявило жодного, яке поєднує захоплення окремого застосунку з його власним звуком, слідування за вікнами цього застосунку та перегляд у браузері без приєднання до зустрічі чи голосового каналу. На цій основі визначено дійових осіб «Глядач» і «Стример», шість варіантів використання зі сценаріями, сформульовано 19 функціональних вимог і нефункціональні вимоги до надійності, продуктивності, безпеки, сумісності, супроводжуваності та зручності використання.

Під час планування трудомісткість оцінено методом точок варіантів використання: близько 109,8 точки, тобто близько 3074 люд.-год за коефіцієнта продуктивності 28 люд.-год. Роботи над проєктом сплановано як дев'ять послідовних етапів. Реєстр містить дев'ять ризиків, і для ключових технічних ризиків заходи реагування реалізовано в коді.

Спроектовано архітектуру з трьох застосунків і спільних модулів протоколу, схем даних і конфігурації, що розгортаються контейнерами Docker Compose за зворотним проксі Caddy. Взаємодію описано діаграмами послідовності, діяльності та станів, модель даних – п'ятьма таблицями, узгодженими з Auth.js, а сервер сигналізації поділено на контролери, сервіси та репозиторії.

У реалізації настільний застосунок Electron перелічує вікна засобами модуля desktopCapturer, визначає процес-власник вікна і захоплює звук лише цього процесу в окремому службовому процесі, а клас SourceFollower кожні 1 500 мс перевіряє появу нових вікон застосунку та перемикає на них трансляцію. Потокову передачу виконує mediasoup: процеси-обробники за кількістю ядер процесора, H.264 як основний кодек і VP8 як резервний, бюджет бітрейту до 18 Мбіт/с. Автентифікація поєднує вхід через Google, гостьові сеанси та вхід настільного застосунку через системний браузер за схемою PKCE; сеанс JWT перевіряють і вебзастосунок, і сервер сигналізації.

Репозиторій не містить автоматизованих тестів, тому перевірку виконано статичним аналізом, перевіркою типів, збиранням і окремими сценаріями. Усі шість пакетів пройшли статичну перевірку, 27 із 30 автоматизованих функціональних прикладів дали очікуваний результат. Три невідповідності стосуються авторизації дій на сервері сигналізації. Шість дефектів, виявлених під час розроблення, виправлено.

Контрольний приклад показав повний шлях користувача від встановлення настільного застосунку до приватної трансляції вікна застосунку, перегляду гостем і автоматичного перемикання на нове вікно. Мету роботи досягнуто: глядачі переглядають у браузері зображення та власний звук лише обраного застосунку, а трансляція без жодних дій стримера перемикається на нове вікно цього застосунку за 1,5 с і повертається до вихідного вікна за 1,3 с, що підтверджено тестовими прикладами Д3 і Д5.

Практичне значення роботи полягає в тому, що сервіс доступний за адресою <https://streamshare.space>, а інсталятори настільного застосунку для Windows, macOS і Linux доступні для завантаження з головної сторінки сервісу. Подальший розвиток передбачає усунення виявлених дефектів авторизації перевітками ролі та власника трансляції, впровадження автоматизованих тестів у конвеєр CI, підписання збірки для macOS з автоматичним оновленням і розподіл однієї трансляції між кількома маршрутизаторами mediasoup методом pipeToRouter для збільшення кількості глядачів [30].

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Pantos R., May W. HTTP Live Streaming : RFC 8216. IETF, 2017. URL: <https://www.rfc-editor.org/rfc/rfc8216> (дата звернення: 05.10.2026).
2. Present during a video meeting. Google Meet Help. URL: <https://support.google.com/meet/answer/9308856> (дата звернення: 05.10.2026).
3. Go Live and Screen Share. Discord Support. URL: <https://support.discord.com/hc/en-us/articles/360040816151-Go-Live-and-Screen-Share> (дата звернення: 05.10.2026).
4. Screen Capture : W3C Working Draft 27 August 2026 / ed. by J.-I. Bruaroey, E. Alon. URL: <https://www.w3.org/TR/screen-capture/> (дата звернення: 05.10.2026).
5. electron-native-screenshare : Cross-platform native audio capture for Electron screen sharing with process-level isolation. GitHub. URL: <https://github.com/CilginSinek/electron-native-screenshare> (дата звернення: 05.10.2026).
6. WebRTC: Real-Time Communication in Browsers : W3C Recommendation 13 March 2025 / ed. by C. Jennings et al. URL: <https://www.w3.org/TR/webrtc/> (дата звернення: 05.10.2026).
7. Alvestrand H. Overview: Real-Time Protocols for Browser-Based Applications : RFC 8825. IETF, 2021. URL: <https://www.rfc-editor.org/rfc/rfc8825> (дата звернення: 05.10.2026).
8. Rescorla E. WebRTC Security Architecture : RFC 8827. IETF, 2021. URL: <https://www.rfc-editor.org/rfc/rfc8827> (дата звернення: 05.10.2026).
9. Westerlund M., Wenger S. RTP Topologies : RFC 7667. IETF, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7667> (дата звернення: 05.10.2026).
10. Video Broadcast. Twitch Developers. URL: <https://dev.twitch.tv/docs/video-broadcast/> (дата звернення: 05.10.2026).
11. Comparative Study of WebRTC Open Source SFUs for Video Conferencing / E. Andre et al. 2018 Principles, Systems and Applications of IP Telecommunications (IPTComm). IEEE, 2018. P. 1–8. DOI: 10.1109/IPTCOMM.2018.8567642.

12. Design. mediasoup documentation. URL:
<https://mediasoup.org/documentation/v3/mediasoup/design/> (дата звернення: 05.10.2026).
13. desktopCapturer. Electron Documentation. URL:
<https://www.electronjs.org/docs/latest/api/desktop-capturer> (дата звернення: 05.10.2026).
14. session. Electron Documentation. URL:
<https://www.electronjs.org/docs/latest/api/session> (дата звернення: 05.10.2026).
15. Window Capture Sources. OBS Knowledge Base. URL:
<https://obsproject.com/kb/window-capture-sources> (дата звернення: 05.10.2026).
16. Application Audio Capture Guide. OBS Knowledge Base. URL:
<https://obsproject.com/kb/application-audio-capture-guide> (дата звернення: 05.10.2026).
17. 2k Streaming on Twitch. Twitch Help. URL:
<https://help.twitch.tv/s/article/stream-quality> (дата звернення: 06.10.2026).
18. Join a meeting. Google Meet Help. URL:
<https://support.google.com/meet/answer/9303069> (дата звернення: 06.10.2026).
19. Freigeben des Bildschirms oder Desktops in Zoom. Zoom Support. URL:
https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0060609 (дата звернення: 05.10.2026).
20. Joining a Zoom meeting without an account. Zoom Support. URL:
https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0059553 (дата звернення: 06.10.2026).
21. OMG Unified Modeling Language (OMG UML) : Version 2.5.1. Object Management Group, 2017. URL: <https://www.omg.org/spec/UML/2.5.1> (дата звернення: 05.10.2026).
22. Cockburn A. Writing Effective Use Cases. Boston : Addison-Wesley, 2001. 270 p.
23. ISO/IEC 25010. ISO 25000 Portal. URL: <https://iso25000.com/en/iso-25000-standards/iso-25010> (дата звернення: 05.10.2026).

24. Denniss W., Bradley J. OAuth 2.0 for Native Apps : RFC 8252 (BCP 212). IETF, 2017. URL: <https://www.rfc-editor.org/rfc/rfc8252> (дата звернення: 05.10.2026).

25. Sakimura N., Bradley J., Agarwal N. Proof Key for Code Exchange by OAuth Public Clients : RFC 7636. IETF, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7636> (дата звернення: 05.10.2026).

26. Azad B. Upcoming security changes to Google's OAuth 2.0 authorization endpoint in embedded webviews. Google Developers Blog. 2021. 29 June. URL: <https://developers.googleblog.com/en/upcoming-security-changes-to-googles-oauth-20-authorization-endpoint-in-embedded-webviews/> (дата звернення: 05.10.2026).

27. Azzeh M., Nassif A. B. Analyzing the Relationship between Project Productivity and Environment Factors in the Use Case Points Method : preprint. arXiv:1705.09920. 2017. 24 p. URL: <https://arxiv.org/abs/1705.09920> (дата звернення: 05.10.2026).

28. Process Model. Electron Documentation. URL: <https://www.electronjs.org/docs/latest/tutorial/process-model> (дата звернення: 05.10.2026).

29. Session strategies. Auth.js. URL: <https://authjs.dev/concepts/session-strategies> (дата звернення: 05.10.2026).

30. mediasoup :: API. mediasoup documentation. URL: <https://mediasoup.org/documentation/v3/mediasoup/api/> (дата звернення: 05.10.2026).

ДОДАТОК А

Лістинг програми

У додатку наведено вихідний код ключових модулів програмної системи stream-share у версії від 25.09.2026. Пропущені фрагменти (імпорти, допоміжний код, а також код, уже наведений в основній частині роботи) позначено рядком коментаря з трикрапкою. Вихідні тексти всіх діаграм PlantUML, наведених у роботі, зберігаються в репозиторії проєкту поруч із відповідними зображеннями.

Далі наведено клас слідування за вікнами застосунку в настільному застосунку (лістинг А.1).

Лістинг А.1 – Клас SourceFollower настільного застосунку

```
// ...
export const POLL_INTERVAL_MS = 1500;
export const RETURN_GRACE_MS = 4000;
export const RETURN_WAIT_MS = 20_000;

export type WindowSource = { id: string; name: string };

export type FollowerDeps = {
  listWindows: () => Promise<WindowSource[]>;
  listProcesses: () => Promise<ProcessTable>;
  resolvePid: (sourceId: string) => number;
  setSelectedSourceId: (sourceId: string) => void;
  emit: (payload: SourceChanged) => void;
};

type Anchor = {
  sourceId: string;
  pid: number;
  exe: string;
  familyRoot: number;
  startedAt: Date;
};

type Active = { sourceId: string; pid: number; name: string };
type PidWindow = WindowSource & { pid: number };

export class SourceFollower {
  private enabled = true;
  private anchor: Anchor | null = null;
  private active: Active | null = null;
  private knownWindowIds = new Set<string>();
  private needsReseed = false;
  private pendingReturnSince: number | null = null;
  private lastReturnAt = 0;
  private lastError: string | null = null;
  private timer: NodeJS.Timeout | null = null;
  private ticking = false;
  private generation = 0;
```

```

constructor(private readonly deps: FollowerDeps) {}

getState(): FollowState {
  return {
    enabled: this.enabled,
    following: this.active !== null,
    activeName: this.active?.name ?? null,
    lastError: this.lastError,
  };
}

setEnabled(enabled: boolean): void {
  this.enabled = enabled;
  if (!enabled) {
    this.stopPolling();
    this.active = null;
    this.pendingReturnSince = null;
    return;
  }
  this.lastError = null;
  if (this.anchor) {
    this.needsReseed = true;
    this.startPolling();
  }
}

async onSourcePicked(sourceId: string): Promise<void> {
  this.stop();
  const gen = this.generation;
  if (!sourceId.startsWith('window:')) return;

  const pid = this.deps.resolvePid(sourceId);
  if (!pid) {
    log.warn('[follower] could not resolve a pid for', sourceId);
    return;
  }

  try {
    const table = await this.deps.listProcesses();
    if (gen !== this.generation) return;
    if (isShellProcess(table, pid)) {
      log.info('[follower] not following a shell process window',
table.get(pid)?.name);
      return;
    }
    const familyRoot = findFamilyRoot(table, pid);
    if (familyRoot === null) {
      log.warn('[follower] pid not in process table', pid);
      return;
    }
    const windows = await this.deps.listWindows();
    if (gen !== this.generation) return;
    this.knownWindowIds = new Set(windows.map((w) => w.id));
    this.anchor = {
      sourceId,
      pid,
      exe: table.get(pid)?.name ?? '',
      familyRoot,
      startedAt: new Date(),
    };
    log.info('[follower] anchored', { sourceId, pid, familyRoot:
table.get(familyRoot)?.name });
    if (this.enabled) this.startPolling();
  } catch (err) {

```

```

    if (gen === this.generation) this.fail(err);
  }
}

async resolveEnded(): Promise<SourceChanged> {
  const anchor = this.anchor;
  if (!this.enabled || !anchor) return { reason: 'lost' };
  const gen = this.generation;
  let windows: WindowSource[] = [];
  try {
    windows = await this.deps.listWindows();
  } catch (err) {
    log.warn('[follower] listWindows failed in resolveEnded', err);
  }
  if (gen !== this.generation) return { reason: 'noop' };
  this.knownWindowIds = new Set(windows.map((w) => w.id));

  if (this.active) return this.beginReturn(windows);
  if (this.pendingReturnSince !== null) return { reason: 'noop' };
  const anchorListed = windows.some((w) => w.id === anchor.sourceId);
  if (anchorListed && Date.now() - this.lastReturnAt < RETURN_GRACE_MS) return {
reason: 'noop' };

  log.info('[follower] captured window closed, stopping');
  this.stop();
  return { reason: 'lost' };
}

stop(): void {
  this.generation += 1;
  this.stopPolling();
  this.anchor = null;
  this.active = null;
  this.pendingReturnSince = null;
  this.knownWindowIds.clear();
}

private startPolling(): void {
  if (this.timer) return;
  this.timer = setInterval(() => void this.tick(), POLL_INTERVAL_MS);
}

private stopPolling(): void {
  if (this.timer) clearInterval(this.timer);
  this.timer = null;
}

private async tick(): Promise<void> {
  if (this.ticking || !this.anchor) return;
  this.ticking = true;
  const gen = this.generation;
  try {
    const windows = await this.deps.listWindows();
    if (gen !== this.generation) return;
    const ids = new Set(windows.map((w) => w.id));
    if (this.needsReseed) {
      this.knownWindowIds = ids;
      this.needsReseed = false;
      return;
    }
  }
  const appeared = windows.filter((w) => !this.knownWindowIds.has(w.id));
  const gone = [...this.knownWindowIds].filter((id) => !ids.has(id));
  this.knownWindowIds = ids;
}

```

```

    if (this.active && gone.includes(this.active.sourceId)) {
      const result = this.beginReturn(windows);
      if (result.reason === 'return') this.deps.emit(result);
    }

    if (appeared.length > 0) await this.considerNewWindows(appeared, gen);
    if (gen !== this.generation) return;

    if (
      this.pendingReturnSince !== null &&
      Date.now() - this.pendingReturnSince > RETURN_WAIT_MS
    ) {
      log.info('[follower] anchor did not come back, giving up');
      this.stop();
      this.deps.emit({ reason: 'lost' });
    }
  } catch (err) {
    if (gen === this.generation) this.fail(err);
  } finally {
    this.ticking = false;
  }
}

private beginReturn(windows: WindowSource[]): SourceChanged {
  this.active = null;
  const target = this.findReturnTarget(windows);
  if (target) return this.completeReturn(target);
  if (this.pendingReturnSince === null) {
    this.pendingReturnSince = Date.now();
    log.info('[follower] anchor not visible, waiting for it to come back');
  }
  return { reason: 'noop' };
}

private findReturnTarget(windows: WindowSource[]): WindowSource | undefined {
  const anchor = this.anchor;
  if (!anchor) return undefined;
  return (
    windows.find((w) => w.id === anchor.sourceId) ??
    windows.find((w) => this.deps.resolvePid(w.id) === anchor.pid)
  );
}

private completeReturn(target: WindowSource): SourceChanged {
  this.pendingReturnSince = null;
  this.active = null;
  this.deps.setSelectedSourceId(target.id);
  if (this.anchor) this.anchor = { ...this.anchor, sourceId: target.id };
  this.lastReturnAt = Date.now();
  log.info('[follower] returned to', target.name, target.id);
  return { reason: 'return', sourceId: target.id, name: target.name };
}

private async considerNewWindows(appeared: WindowSource[], gen: number):
Promise<void> {
  const anchor = this.anchor;
  if (!anchor) return;

  const withPid: PidWindow[] = appeared
    .map((w) => ({ ...w, pid: this.deps.resolvePid(w.id) }))
    .filter((w) => w.pid !== 0);

  const ownWindow = withPid.find((w) => w.pid === anchor.pid);
  if (ownWindow && this.pendingReturnSince !== null) {

```

```

    this.deps.emit(this.completeReturn(ownWindow));
    return;
}

const others = withPid.filter((w) => w.pid !== anchor.pid);
if (others.length === 0) return;

const table = await this.deps.listProcesses();
if (gen !== this.generation) return;
const anchorAlive = table.has(anchor.pid);

const candidates: (PidWindow & { createdAt: Date })[] = [];
for (const w of others) {
    const info = table.get(w.pid);
    if (!info) continue;
    if (!isInFamily(table, w.pid, anchor.familyRoot)) continue;
    if (info.createdAt <= anchor.startedAt) continue;

    const sameExe = info.name.toLowerCase() === anchor.exe.toLowerCase();
    if (sameExe && !anchorAlive) {
        // The anchor process was replaced (League's "close client during game").
        this.anchor = { ...anchor, sourceId: w.id, pid: w.pid };
        log.info('[follower] anchor relaunched as', w.name, w.id);
        if (this.pendingReturnSince !== null)
            this.deps.emit(this.completeReturn(w));
        return;
    }
    if (sameExe) continue;
    candidates.push({ ...w, createdAt: info.createdAt });
}
if (candidates.length === 0) return;

candidates.sort((a, b) => b.createdAt.getTime() - a.createdAt.getTime());
const winner = candidates[0]!;
if (this.active?.sourceId === winner.id) return;

this.pendingReturnSince = null;
this.active = { sourceId: winner.id, pid: winner.pid, name: winner.name };
this.deps.setSelectedSourceId(winner.id);
log.info('[follower] following', winner.name, winner.id);
this.deps.emit({ reason: 'follow', sourceId: winner.id, name: winner.name });
}

private fail(err: unknown): void {
    const message = err instanceof Error ? err.message : String(err);
    log.error('[follower] disabled after error:', message);
    this.stop();
    this.enabled = false;
    this.lastError = message;
    this.deps.emit({ reason: 'error', message });
}
}

```

Далі наведено сервіс сервера сигналізації, що створює процеси-обробники, маршрутизатори й транспорти mediasoup та обмежує їхній бітрейт (лістинг А.2).

Лістинг А.2 – Клас MediasoupService сервера сигналізації (скорочено)

```
// ...
export type TransportRole = 'send' | 'recv';

const RTC_MIN_PORT = env.MEDIASOUP_RTC_MIN_PORT;
const RTC_MAX_PORT = env.MEDIASOUP_RTC_MAX_PORT;
const NUM_WORKERS = env.MEDIASOUP_NUM_WORKERS;
const ANNOUNCED_IP = env.MEDIASOUP_ANNOUNCED_IP;
const INITIAL_OUTGOING_BITRATE = env.MEDIASOUP_INITIAL_OUTGOING_BITRATE;
const MAX_INCOMING_BITRATE = env.MEDIASOUP_MAX_INCOMING_BITRATE;
/** `x-google-start-bitrate` is kbit/s, so the same start as BWE expressed in its
unit. */
const START_BITRATE_KBPS = Math.round(INITIAL_OUTGOING_BITRATE / 1000);

export class MediasoupService {
  private readonly workers: Worker[] = [];
  constructor() {}

  // ...
  async createWorkers(): Promise<Worker[]> {
    for (let i = 0; i < NUM_WORKERS; i++) {
      const worker = await mediasoup.createWorker({
        logLevel: 'warn',
        rtcMinPort: RTC_MIN_PORT,
        rtcMaxPort: RTC_MAX_PORT,
      });
      this.workers.push(worker);
    }

    return this.workers;
  }

  async createRouter(): Promise<Router> {
    const worker = await this.pickLeastLoadedWorker();

    const router = await worker.createRouter({
      mediaCodecs: [
        {
          kind: 'audio',
          mimeType: 'audio/opus',
          clockRate: 48000,
          channels: 2,
        },
      ],
    });
    /**
     * H.264 first: it is the only codec every browser and Electron encode
     * in hardware, and hardware encoding is what makes 1080p60, 1440p60
     * and 4K source streams sustainable. Chrome's software VP8 (libvpx)
     * cannot hold those modes and silently degrades them.
     *
     * Baseline (42001f) before Constrained Baseline (42e01f), deliberately:
     * Chromium's hardware encoder factory only advertises Baseline, Main
     * and High, while Constrained Baseline comes solely from its OpenH264
     * software encoder. Offering 42e01f first therefore forces software
     * encoding in Chrome, Edge and Electron (verified against Chrome 152
     * with NVENC). Baseline is still decodable by the Chromium family and
     * Firefox; 42e01f stays as the fallback for endpoints without it.
     *
     * `level-asymmetry-allowed` lets endpoints encode above level 3.1
     * (1440p60 is level 5.1) without renegotiation.
     */
    {
      kind: 'video',
    }
  }
}
```

```

        mimeType: 'video/H264',
        clockRate: 90000,
        parameters: {
            'packetization-mode': 1,
            'profile-level-id': '42001f',
            'level-asymmetry-allowed': 1,
            'x-google-start-bitrate': START_BITRATE_KBPS,
        },
    },
    {
        kind: 'video',
        mimeType: 'video/H264',
        clockRate: 90000,
        parameters: {
            'packetization-mode': 1,
            'profile-level-id': '42e01f',
            'level-asymmetry-allowed': 1,
            'x-google-start-bitrate': START_BITRATE_KBPS,
        },
    },
    /** Software fallback for endpoints without H.264. */
    {
        kind: 'video',
        mimeType: 'video/VP8',
        clockRate: 90000,
        parameters: {
            'x-google-start-bitrate': START_BITRATE_KBPS,
        },
    },
],
});

return router;
}

async createTransport(
    router: Router,
    direction: TransportRole,
    videoMaxBitrate?: number,
): Promise<WebRtcTransport> {
    const transport = await router.createWebRtcTransport({
        listenIps: [{ ip: '0.0.0.0', announcedIp: ANNOUNCED_IP }],
        enableUdp: true,
        enableTcp: true,
        preferUdp: true,
        initialAvailableOutgoingBitrate:
this.initialOutgoingBitrate(videoMaxBitrate),
        appData: { direction },
    });

    return transport;
}

/**
 * Where bandwidth estimation starts on a transport (bit/s). When the stream's
 * video budget is known – viewer transports, created after the broadcaster
 * produced – it is the same fraction of that budget the sender's encoder opens
 * at, so a 4K stream does not start a viewer at a 1080p estimate. Never below
 * the configured floor, never above what we accept from the broadcaster.
 */
private initialOutgoingBitrate(videoMaxBitrate?: number): number {
    if (!videoMaxBitrate || !Number.isFinite(videoMaxBitrate) || videoMaxBitrate
<= 0) {
        return INITIAL_OUTGOING_BITRATE;
    }
}

```

```

    }
    const proportional = startBitrateFor(Math.min(videoMaxBitrate,
MAX_VIDEO_BITRATE));
    return Math.max(proportional, INITIAL_OUTGOING_BITRATE);
  }

  /**
   * Caps what the remote endpoint is told it may send us (REMB / transport-cc).
   * Only meaningful once the transport is connected, so callers apply it there.
   */
  async limitIncomingBitrate(transport: WebRtcTransport): Promise<void> {
    await transport.setMaxIncomingBitrate(MAX_INCOMING_BITRATE);
  }

  /**
   * The invariant of ADR 0001: the cap we advertise to the broadcaster must stay
   * at or above the sender's own ceiling plus overhead, or we silently clamp a
   * mode the app lets people pick. Lowering it is a legitimate operator choice on
   * a thin uplink, so this warns instead of refusing to start.
   */
  static bitrateConfigWarning(): string | null {
    if (MAX_INCOMING_BITRATE >= MIN_SERVER_INCOMING_BITRATE) return null;
    return (
      `MEDIASOUP_MAX_INCOMING_BITRATE=${MAX_INCOMING_BITRATE} is below ` +
      `${MIN_SERVER_INCOMING_BITRATE} (the ${MAX_VIDEO_BITRATE} bit/s sender
ceiling plus ` +
      'overhead): the SFU will clamp broadcasters in the highest quality modes.'
    );
  }
}

```

Далі наведено фрагмент контролера трансляції сервера сигналізації з маршрутами та обробниками повідомлень WebSocket стримера й глядача (лістинг А.3).

Лістинг А.3 – Маршрути та обробники WebSocket класу StreamsController (фрагмент)

```

// ...

this.app.register(
  (app) => {
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    app.decorateRequest('context', null as any);
    app.addHook('onRequest', async (req) => {
      req.context = {};
    });

    app.get<{ Params: { streamId: string } }>(
      '/broadcast',
      {
        websocket: true,
        preValidation: validateWsJwt,
      },
      this.handleBroadcast,
    );
  }
);

```

```

    app.get<{ Params: { streamId: string } }>(
      '/watch',
      {
        websocket: true,
        preValidation: async (req, reply) => {
          await validateWsJwt(req, reply);
          req.context.userId ??= crypto.randomUUID();
        },
      },
      this.handleWatch,
    );
  },
  { prefix: '/ws/streams/:streamId' },
);
}
// ...
private handleBroadcast = (
  socket: WebSocket,
  req: FastifyRequest<{ Params: { streamId: string } }>,
) => {
  const streamId = req.params.streamId;
  let status: StreamStatus.Connecting | StreamStatus.Reconnecting =
StreamStatus.Connecting;
  const streamContext = this.streamsService.getContext(streamId);
  if (streamContext?.reconnectionId) {
    status = StreamStatus.Reconnecting;
    clearInterval(streamContext.reconnectionId);
    this.streamsService.setContext(streamId, { ...streamContext, reconnectionId:
null });
  }
  void this.streamsService.update({ id: streamId, status });

  socket.on('message', (message) => {
    this.app.log.info(message.toString());
    void this.handleStreamerMessage({
      message: message.toString(),
      socket,
      streamId,
    });
  });

  socket.on('close', () => {
    this.app.log.info(`Streamer socket of 'stream:${streamId}' closed`);
    void this.handleStreamerSocketClose(streamId);
  });
};

private handleWatch = (
  socket: WebSocket,
  req: FastifyRequest<{ Params: { streamId: string } }>,
) => {
  const userId = req.context.userId as string;
  const streamId = req.params.streamId;
  const context = { viewerId: userId, streamId };

  socket.on('message', (message) => {
    this.app.log.info(message.toString());
    void this.handleViewerMessage({
      message: message.toString(),
      socket,
      context,
    });
  });
};

```

```

        socket.on('close', () => {
            this.app.log.info(
                `Viewer 'user:${context.viewerId}' socket of 'stream:${context.streamId}'
closed`,
            );
            void this.handleViewerSocketClose(context);
        });
    };

    private handleStreamerMessage = async (data: {
        message: string;
        socket: WebSocket;
        streamId: string;
    }) => {
        const { message, socket, streamId } = data;
        const { method, params, requestId } = parseMessage(message) as
WsRequestEnvelope;

        let streamContext;
        let stream;
        try {
            stream = await this.streamsService.requireStream(streamId);
            streamContext = await this.streamsService.requireContext(streamId);
        } catch (error) {
            if (error instanceof EntityNotFoundError) {
                socket.send(constructErrorResponse(method, { code: 404, msg: error.message
}, requestId));
            } else if (error instanceof Error) {
                socket.send(constructErrorResponse(method, { code: 404, msg: error.message
}, requestId));
            }
            this.app.log.error(error);
            return;
        }

        try {
            switch (method) {
                case StreamerActions.GetRtpCapabilities: {
                    return socket.send(
                        constructSuccessResponse(method, streamContext.router.rtpCapabilities,
requestId),
                    );
                }
                case CommonActions.CreateTransport: {
                    const transport = await this.streamerService.createTransport(streamId,
streamContext);
                    return socket.send(
                        constructSuccessResponse(
                            method,
                            {
                                id: transport.id,
                                iceParameters: transport.iceParameters,
                                dtlsParameters: transport.dtlsParameters,
                                iceCandidates: transport.iceCandidates,
                            },
                            requestId,
                        ),
                    );
                }
                case CommonActions.ConnectTransport: {
                    await this.streamerService.connectTransport(streamContext,
params.dtlsParameters);
                    return socket.send(constructSuccessResponse(method, null, requestId));
                }
            }
        }
    }

```

```

        case StreamerActions.Produce: {
            const { last, ...restParams } = params;
            const producer = await this.streamerService.produce(streamId,
streamContext, restParams);
            if (last) {
                await this.streamsService.update({ id: streamId, status:
StreamStatus.Live });
            }

            if (stream.status === 'reconnecting') {
                this.notifyViewers(streamId, {
                    type: 'event',
                    name: WsEvents.StreamerReconnected,
                    data: { producerId: producer.id },
                });
            }

            return socket.send(
                constructSuccessResponse(method, { producerId: producer.id },
requestId),
            );
        }
        case StreamerActions.EndStream: {
            await this.streamsService.update({
                id: streamId,
                status: StreamStatus.Ended,
                endReason: StreamEndReason.StreamerStop,
                endedAt: new Date(),
            });
            return socket.send(constructSuccessResponse(method, null, requestId));
        }
    } catch (error) {
        if (error instanceof Error) {
            socket.send(
                constructErrorResponse(method, { code: 'unknown', msg: error.message },
requestId),
            );
            this.app.log.error(error);
        }
    }
};

// ...
private handleStreamerSocketClose = async (streamId: string): Promise<void> => {
    let reconnectingAttempts = 0;
    let stream;
    try {
        stream = await this.streamsService.getOne(streamId);
    } catch {
        return this.streamerService.releaseResources(streamId);
    }
    if (stream.status === StreamStatus.Ended) {
        this.notifyViewers(streamId, { type: 'event', name: WsEvents.StreamEnd,
data: null });
        return this.streamerService.releaseResources(streamId);
    }

    const reconnectionId = setInterval(() => {
        if (reconnectingAttempts++ >= StreamsController.MAX_ATTEMPTS) {
            void this.streamsService
                .update({
                    id: streamId,
                    status: StreamStatus.Ended,

```

```

        endReason: StreamEndReason.Timeout,
        endedAt: new Date(),
    })
    .then(() => this.streamerService.releaseResources(streamId));
    this.notifyViewers(streamId, { type: 'event', name: WsEvents.StreamEnd,
data: null });

    return clearInterval(reconnectionId);
}
}, 6000);
const streamContext = this.streamsService.getContext(streamId);
if (streamContext) {
    this.streamsService.setContext(streamId, { ...streamContext,
reconnectionId });
}

this.notifyViewers(streamId, { type: 'event', name:
WsEvents.StreamerDisconnect, data: null });
};

private handleViewerSocketClose = async (data: {
    viewerId: string;
    streamId: string;
}): Promise<void> => {
    const { viewerId, streamId } = data;
    return this.viewerService.releaseResources(viewerId, streamId);
};

private notifyViewers(streamId: string, event: WsEvent) {
    const streamContext = this.streamsService.getContext(streamId);
    if (!streamContext) return;
    Object.values(streamContext.viewers).forEach(({ socket }) => {
        socket.send(constructEvent(event));
    });
}
}
}

```

Далі наведено хук сторінки трансляції вебзастосунку, що керує захопленням і публікацією медіапотоків стримера (лістинг А.4).

Лістинг А.4 – Хук useStreamer сторінки трансляції (скорочено)

```

// ...
const DEFAULT_VIDEO_SETTINGS: VideoSettings = { quality: StreamQuality.HD, fps: 30
};

export function useStreamer() {
// ...
    const applyVideoSettings = useCallback(
        async (track: MediaStreamTrack, producer?: Producer) => {
            const derived = deriveEncodingForTrack(track);
            track.contentHint = derived.contentHint;
            try {
                await track.applyConstraints({ frameRate: derived.frameRate });
            } catch (e) {
                console.warn('[applyVideoSettings] applyConstraints failed:', e);
            }

            if (!producer?.rtpSender) return;

```

```

    const parameters = producer.rtpSender.getParameters();
    await producer.rtpSender.setParameters({
      ...parameters,
      degradationPreference: derived.degradationPreference,
      encodings: [{ ...parameters.encodings[0], ...derived.encoding }],
    });
  },
  [deriveEncodingForTrack],
);

const createProducer = useCallback(
  async (track: MediaStreamTrack) => {
    let producerOptions: ProducerOptions = { track };
    if (track.kind === 'video') {
      const derived = deriveEncodingForTrack(track);
      producerOptions = {
        ...producerOptions,
        codec: pickVideoCodec(deviceRef.current?.rtpCapabilities.codecs),
        encodings: [derived.encoding],
        codecOptions: { videoGoogleStartBitrate: derived.startBitrateKbps },
      };
    }
    const producer = await transportRef.current!.produce(producerOptions);
    if (track.kind === 'audio' && isMuted) {
      producer.pause();
    }
    if (track.kind === 'video') await applyVideoSettings(track, producer);
    producersRef.current.push(producer);
  },
  [applyVideoSettings, deriveEncodingForTrack, isMuted],
);

const replaceStream = useCallback(
  async (stream: MediaStream) => {
    return Promise.all(
      stream.getTracks().map(async (track) => {
        const producer = producersRef.current.find((p) => p.kind ===
track.kind);
        if (!producer) return createProducer(track);

        if (!track) return producer.pause();

        await producer.replaceTrack({ track: track });
        if (producer.kind === 'video') await applyVideoSettings(track,
producer);
        if (producer.kind !== 'audio' || !isMuted) {
          producer.resume();
        }
      })
    );
  },
  [applyVideoSettings, createProducer, isMuted],
);
// ...
const connectToStream = useCallback(
  async (stream: Stream) => {
    if (!mediaStreamRef.current) {
      toast.error('Stream is not created yet');
      return;
    }
    const isTrackByKindSent = Object.fromEntries(
      mediaStreamRef.current.getTracks().map((t) => [t.kind, false]),
    );

```

```

    wsClientRef.current = new WsClient(signalingWsUrl(`/ws/streams/${
{stream.id}/broadcast`));
    wsClientRef.current.ws.addEventListener('close', handleSocketClose);

    const { result: rtpCapabilities } = await wsClientRef.current.request({
      type: 'req',
      method: StreamerActions.GetRtpCapabilities,
      params: null,
    });

    const device = new Device();
    await device.load({ routerRtpCapabilities: rtpCapabilities });
    deviceRef.current = device;

    const { result: transport } = await wsClientRef.current.request({
      type: 'req',
      method: StreamerActions.CreateTransport,
      params: { direction: 'send' },
    });

    transportRef.current = device.createSendTransport(transport);

    transportRef.current.on('connect', ({ dtlsParameters }, callback,
errorCallback) => {
      wsClientRef
        .current!.request({
          type: 'req',
          method: StreamerActions.ConnectTransport,
          params: { dtlsParameters },
        })
        .then(() => callback())
        .catch(errorCallback);
    });

    transportRef.current.on('produce', ({ kind, rtpParameters }, callback,
errorCallback) => {
      isTrackByKindSent[kind] = true;
      const videoTrack = kind === 'video' ?
mediaStreamRef.current?.getVideoTracks()[0] : null;
      wsClientRef
        .current!.request({
          type: 'req',
          method: StreamerActions.Produce,
          params: {
            rtpParameters,
            kind,
            last: Object.values(isTrackByKindSent).every((e) => e === true),
            maxBitrate: videoTrack
              ? deriveEncodingForTrack(videoTrack).encoding.maxBitrate
              : undefined,
          },
        })
        .then((res) => callback({ id: res.result.producerId }))
        .catch(errorCallback);
    });

    await Promise.all(mediaStreamRef.current.getTracks().map(createProducer));
    setStatus(StreamStatus.Live);

    return stream;
  },
  [createProducer, deriveEncodingForTrack, handleSocketClose],
);

```

```

const broadcast = useCallback(async () => {
  const stream = await createStream(isPrivate);
  if (!stream) return;
  await connectToStream(stream);
  setCurrentStream(stream);
  stopCapturingThumbnailRef.current = startCapturingThumbnail(stream.id);
}, [connectToStream, isPrivate, startCapturingThumbnail]);
// ...

```

Далі наведено модуль входу через Google у системному браузері головного процесу настільного застосунку (лістинг А.5).

Лістинг А.5 – Вхід через Google у системному браузері в настільному застосунку (скорочено)

```

// ...
const SIGN_IN_TIMEOUT_MS = 5 * 60 * 1000;

export type GoogleSignInResult =
  | { status: 'success'; code: string; verifier: string }
  | { status: 'cancelled' }
  | { status: 'timeout' }
  | { status: 'error'; message: string };

type Pending = {
  server: Server;
  settle: (result: GoogleSignInResult) => void;
  /** Handed to any later caller that arrives while this attempt is running. */
  result: Promise<GoogleSignInResult>;
};

let pending: Pending | null = null;

const base64url = (bytes: Buffer) => bytes.toString('base64url');
// ...
export function startGoogleSignIn(window: BrowserWindow):
Promise<GoogleSignInResult> {
  if (pending) return pending.result;

  const verifier = base64url(randomBytes(32));
  const challenge = createHash('sha256').update(verifier).digest('base64url');
  const state = base64url(randomBytes(16));

  const server = createServer();

  let settleAttempt: (result: GoogleSignInResult) => void = () => {};

  const attempt = new Promise<GoogleSignInResult>((resolve) => {
    let done = false;
    const settle = (result: GoogleSignInResult) => {
      if (done) return;
      done = true;
      clearTimeout(timer);
      if (pending?.server === server) pending = null;
      server.close();
      server.closeAllConnections();
      resolve(result);
    };

```

```

};

const timer = setTimeout(() => settle({ status: 'timeout' }),
SIGN_IN_TIMEOUT_MS);

server.on('error', (error) =>
  settle({ status: 'error', message: error.message || 'The sign-in listener
failed' })),
);

server.on('request', (req, res) => {
  const url = new URL(req.url ?? '/', 'http://127.0.0.1');
  if (url.pathname !== '/callback') {
    respond(res, 404, 'Not found.');
```

return;

```
  }

  if (url.searchParams.get('state') !== state) {
    respond(res, 400, 'This sign-in link does not belong to this app.');
```

return;

```
  }

  const error = url.searchParams.get('error');
  const code = url.searchParams.get('code');
  if (error || !code) {
    respond(res, 200, 'Sign-in was not completed. You can close this tab and
try again.', () =>
      settle({
        status: 'error',
        message:
          error === 'access_denied'
            ? 'The browser did not finish signing in to a Stream Share
account.'
            : 'The browser could not complete the sign-in.',
      })),
    );
    return;
  }

  bringWindowToFront(window);

  respond(res, 200, 'Signed in. You can close this tab and return to Stream
Share.', () =>
    settle({ status: 'success', code, verifier })),
  );
});

settleAttempt = settle;

void (async () => {
  try {
    const port = await listen(server);

    if (done) {
      server.close();
      return;
    }

    const url = new URL('/api/desktop-auth/start', __WEB_URL__);
    url.searchParams.set('port', String(port));
    url.searchParams.set('state', state);
    url.searchParams.set('challenge', challenge);

    await shell.openExternal(url.toString());
  }
});

```

```

    } catch (error) {
      settle({
        status: 'error',
        message: error instanceof Error ? error.message : 'Could not open the
browser',
      });
    }
  })();
});

pending = { server, settle: settleAttempt, result: attempt };

return attempt;
}

export function cancelGoogleSignIn(): void {
  pending?.settle({ status: 'cancelled' });
  pending = null;
}

```